Top-down = Bottom-up

Sound and Complete Characterisations of Liveness by Multiparty Global Protocols

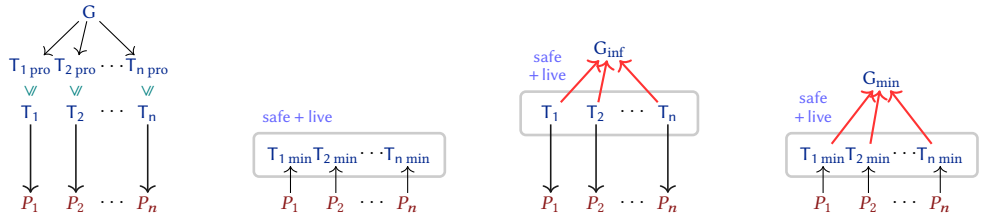
ANONYMOUS AUTHOR(S)

Multiparty session types (MPST) are a type discipline for concurrent and distributed systems, designed to ensure not only type-safety and deadlock-freedom, but also *liveness* of typed communicating processes. Two main MPST methodologies, top-down and bottom-up, have been proposed and are integrated into a wide range of programming languages and tools. The *top-down* strategy starts by specifying the overall choreography of the protocol (called a global type), from which a set of local types that satisfy safety and liveness are generated by end-point projection (EPP). Once each participant is type-checked against a generated local type, liveness of the set of typed processes is automatically ensured *by construction*. The *bottom-up* strategy directly checks whether local types inferred from processes satisfy liveness in order to enforce liveness of processes. Since the top-down strategy depends on global types and the EPP algorithms, it has often been considered that the top-down system offers *strictly less typability* than the bottom-up system. Our paper negates this belief. We prove that, using the precise subtyping for the subsumption rule, the top-down strategy offers *exactly the same typability* as the bottom-up system. More precisely, a multiparty session M is typable and verified to be live by the bottom-up typing system if and only if M is typable by the top-down typing system. The key for the proofs is to develop a *principal global type inference algorithm* which builds a *principal* global type from an arbitrary set of live local types. We have implemented the global type inference algorithm together with projection, process type checking and local type inference algorithms, and built a toolchain for both the top-down and bottom-up strategies. We evaluated our toolchain with representative examples from the literature, confirming that the top-down approach is more efficient than the bottom-up approach.

1 INTRODUCTION

Multiparty Session Types (MPST) [55, 57] are a type framework for distributed services and agents, designed to describe and verify communication protocols between *multiple* distributed participants. The original theory takes the *top-down* strategy (Figure 1a): a protocol designer first prescribes the overall choreography (who sends what to whom) as a *global type* G. Then an end-point projection (EPP) algorithm uses this global type to generate a set of local types $\{\mathsf{T}_i\}_{i\in I}$ which constrain how each individual participant P_i can behave. Type-checking guarantees that typed processes $\{P_i\}_{i\in I}$ satisfy *communication safety* (there are no unexpected type and label mismatches), *deadlock-freedom* and *session fidelity* (each process adheres to the declared global protocol). In the top-down system, typability can ensure a stronger property, *liveness*, i.e., "something good eventually happens". Here "something good" means that a request eventually gets a response, and each participant in the protocol is able to make *progress* without stalling indefinitely. The top-down strategy has been integrated into many programming languages, taking various forms such as code generation, API generation, API inference, event-driven programming, static type checking, protocol compliance, real-time systems and runtime monitoring. Some notable examples include integration with languages such as Rust [30, 58, 70, 114], Java [17, 59, 60, 69], Scala [11, 28, 39, 65, 98, 115], Go [23], TypeScript [45, 85], PureScript [68], OCaml [62, 118], MPI-C [81, 90], F★ [119], F♯ [86], Python [34, 88, 89], Erlang [36, 41, 87], C [92, 93], C♯ [67], and domain-specific actor languages [25, 26, 42, 52]. See [116] for a survey. Mechanisations of the top-down system have been developed across different frameworks to ensure correctness of the top-down system [21, 22, 37, 64, 79, 109, 110].

The shortcoming of the top-down approach is that *typability is limited* since it depends on the EPP algorithm being used. If G is not projectable by the chosen EPP algorithm, either G is not *implementable* (there exists no correct set of local types) or this particular EPP algorithm rejects some implementable G. If the EPP is too limited, many safe and live processes become untypable.



(a) top-down (b) bottom-up (c) global type inference (d) principal type inf.

Fig. 1. Multiparty session type frameworks. We have $\mathsf{G}_{\mathsf{min}} \subseteq \mathsf{G}_{\mathsf{inf}} \subseteq \mathsf{G}$, where $\subseteq$ denotes trace inclusion.

As expected, computationally cheaper EPP algorithms tend to give fewer typable processes than more expensive ones [112].

The bottom-up strategy proposed in [99] aims to solve this issue. It directly verifies that a set of local types (either prescribed by the programmer or inferred from a set of processes) satisfies some desired property φ (Figure 1b). The benefit of this approach is the ability to verify a wide variety of properties of typed processes. For instance, it is possible to check whether processes satisfy type-safety but not liveness. Since local types resemble processes (such as CCS [83] and CSP [54]) or state machines, this approach can directly use off-the-shelf model-checkers (such as mCRL2 [105] or SPIN [106]) to verify properties. The bottom-up system is integrated into Scala [10, 100], OCaml [61], Go [71, 91] and Rust [30, 70], and certified using Rocq [38, 53] and Why3 [49].

The shortcoming of this approach is that it is *computationally expensive*. Verifying safety, deadlock-freedom and liveness of a set of local types is PSPACE-complete [112] in the synchronous semantics, and is *undecidable* in the asynchronous semantics [18]. By contrast, the top-down approach requires only projection and subtyping, both of which are polynomial in the size of the types [112].

This paper gives results with unexpected conclusions related to typability of the top-down and bottom-up systems. Our focus is the *liveness* property. To explain this, let us assume a process P_i that acts as participant $\mathbf{p_i}$ (denoted by $\mathbf{p_i} :: P_i$) has session type $\mathsf{T_i}$. A *multiparty session* M is a parallel composition of participants ($M = \mathbf{p_0} :: P_0 \mid \cdots \mid \mathbf{p_n} :: P_n$). The bottom-up typing judgement is given as $\vdash^{\mathsf{bot}} M : \Delta$ where Δ is a *safe typing context* mapping each participant $\mathbf{p_i}$ to its type $\mathsf{T_i}$. "Safety" on Δ is a minimum requirement on the typing judgement $\vdash^{\mathsf{bot}}$ to ensure that M is type safe.

We define *a set of typable live sessions*, $\mathbb{P}_{\mathsf{bot}}$, as a set of multiparty sessions which are typable by the bottom-up system:

$$\mathbb{P}_{\mathsf{bot}} \;=\; \{\, M \mid \vdash^{\mathsf{bot}} M : \Delta \quad \text{such that } \Delta \text{ is live} \}$$

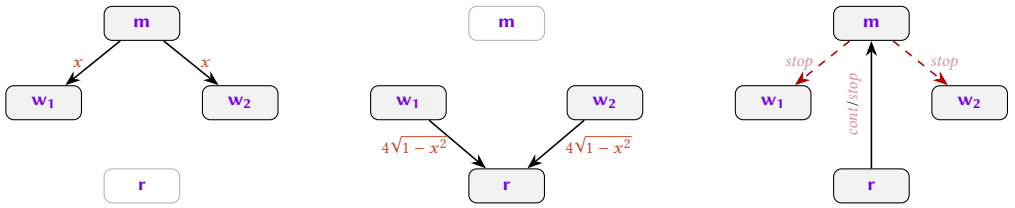
If M is typed by live Δ, then M satisfies liveness [95, 99]. This set is closed under reduction, i.e., if $M \in \mathbb{P}_{\mathsf{bot}}$ and M reduces to M', then we have $M' \in \mathbb{P}_{\mathsf{bot}}$. Thus $\mathbb{P}_{\mathsf{bot}}$ gives a **complete characterisation of liveness**, i.e., M is typable and live if $M \in \mathbb{P}_{\mathsf{bot}}$ and vice versa, and $\mathbb{P}_{\mathsf{bot}}$ is closed under reductions.

In the top-down system, each participant is typed by $\mathsf{T_i}$, which is a projection of global type G on participant $\mathbf{p_i}$ (denoted by $\mathsf{G} \upharpoonright_{\mathbf{p_i}} \mathsf{T_i}$). Assuming P_i is typed by $\mathsf{T_i}$, we define:

$$\mathbb{P}_{\mathsf{top}} \;=\; \{\, M \mid \vdash^{\mathsf{top}} M : \Delta \quad \text{such that } \mathsf{G} \upharpoonright_{\mathbf{p_i}} \mathsf{T_i} \quad \Delta = \{\mathbf{p_i}{:}\mathsf{T_i}\}_{i \in I} \text{ for some } \mathsf{G} \}$$

This implies: if $M \in \mathbb{P}_{\mathsf{top}}$, then $M \in \mathbb{P}_{\mathsf{bot}}$, i.e., $\mathbb{P}_{\mathsf{top}} \subseteq \mathbb{P}_{\mathsf{bot}}$. Depending on the EPP, we have only $\mathbb{P}_{\mathsf{top}} \subsetneq \mathbb{P}_{\mathsf{bot}}$, i.e., $M \in \mathbb{P}_{\mathsf{bot}}$ but $M \notin \mathbb{P}_{\mathsf{top}}$. This is because $\mathsf{G} \upharpoonright_{\mathbf{p_i}} \mathsf{T_i}$ might be undefined even if G is in fact implementable by safe processes satisfying liveness.

This paper shows the typability given by the top-down system is the same as that given by the bottom-up system, i.e., $\mathbb{P}_{\mathsf{top}} = \mathbb{P}_{\mathsf{bot}}$, against the common belief that the top-down strategy is incomplete. **The top-down strategy is complete with respect to liveness** if three conditions are met. First, the typing rule for a process includes the subsumption rule defined by *precise synchronous*



(a) Dispatch (b) Compute (c) Control

Fig. 2. One round of the Monte Carlo π-estimation protocol. (a) Manager sends random floats to workers. (b) Workers compute $4\sqrt{1-x^2}$ and send results to reducer. (c) Reducer sends *cont* or *stop* to manager; on *stop*, manager terminates both workers (dashed).

subtyping $\leqslant$ [47]. Secondly, G is *balanced* (i.e., any participant appearing in a subterm of G is unavoidable); and thirdly, the EPP algorithm we take is the most expressive and computationally expensive EPP algorithm among the four EPP algorithms proposed in the MPST literature [112], namely the *coinductive full-merging projection*.

To prove this, we develop a *principal global type inference algorithm* that synthesises a balanced global type from any safe and live typing context Δ (Figure 1c). The key idea is to equip contexts with behavioural semantics and follow a deterministic fair schedule, so that synthesis extracts a representative interleaving while turning repeated semantic states into syntactic μ-binders. We then show that the resulting global type is projectable by the coinductive full-merging projection, preserves exactly the traces of Δ, and is therefore a principal type for Δ.

We implement a toolchain (§8) which consists of (1) **the top-down system** with the four projection algorithms in [112], subtyping checking and process type checking; and (2) **the bottom-up system** with the minimum local type inference from a given process, safety and liveness checking of a typing context and the principal global type inference algorithm. We evaluate the toolchain on a range of examples from the literature, including particularly challenging global types from [99], [77], and [22], all of which can be projected top-down and whose global types can also be inferred bottom-up. We further show that the top-down strategy can often be more performant compared to off-the-shelf bottom-up model checkers such as mpstk [99].

Outline §2 gives an overview of our framework and results. §3 summarises the calculus and defines the properties needed in this paper. §4 defines global and local types and projection. §5 equips types with labelled transition semantics and fair scheduling. §6 is the main section: it presents the inference algorithm and proves its correctness with respect to liveness. §7 proves the completeness of the top-down typing system with respect to live processes, proving $\mathbb{P}_{\mathtt{top}} = \mathbb{P}_{\mathtt{bot}}$. §8 outlines the toolchain and benchmarks it on case studies from the literature. §9 discusses related work and concludes the paper. The supplemental materials contain the full proofs and more examples. Information about the toolchain can be found in the data availability statement (§9).

2 OVERVIEW

As a running example, we consider a *Monte Carlo protocol* for estimating π, a variation of the MapReduce protocol from [99]. Our goal is to prove that any safe and live typing context Δ has a corresponding global type G. For this, we build G from Δ which is *inferred* from a live session M.

We consider four participants: *manager* **m**, *workers* $\mathbf{w_1}$, $\mathbf{w_2}$, and *reducer* **r**. In each round, the manager selects *map* for each worker and sends a random float in $[0, 1]$, each worker computes $4\sqrt{1-x^2}$ and reports the result to the reducer, who decides whether to continue or stop. If it stops, the manager selects *stop* for both workers and the protocol terminates. The average of all reported values estimates π. Figure 2 visualises one round of this workflow.

To make the role of subsumption explicit, we let the manager be *slightly more permissive* than the protocol shown in Figure 2: besides *cont* and *stop*, it is prepared to receive an extra label *crash* from the reducer. The reducer never emits *crash*, so this branch does not affect the actual behaviour of the session, but it does appear in the inferred local type of the manager. A multiparty session M of a Monte Carlo protocol consists of four processes defined as:

$$P_{\mathbf{w_i}} = \mu X.\, \mathbf{m}\triangleright\{\mathit{map}\colon \mathbf{m}?(x).\mathbf{r}!\langle 4\sqrt{1-x^2}\rangle.X,\ \mathit{stop}\colon \mathbf{0}\} \qquad (i=1,2)$$

$$P_{\mathbf{r}} = \mu Y.\, \mathbf{w_1}?(x).\mathbf{w_2}?(y).\mathbf{if}\ g(x,y)\ \mathbf{then}\ \mathbf{m}\triangleleft\mathit{cont}.\,Y\ \mathbf{else}\ \mathbf{m}\triangleleft\mathit{stop}.\,\mathbf{0}$$

$$P_{\mathbf{m}} = \mu X.\, \mathbf{w_1}\triangleleft\mathit{map}.\,\mathbf{w_1}!\langle\mathtt{rand}()\rangle.\mathbf{w_2}\triangleleft\mathit{map}.\,\mathbf{w_2}!\langle\mathtt{rand}()\rangle.\mathbf{r}\triangleright\{\mathit{cont}\colon X,\ \mathit{crash}\colon \mathbf{0},\ \mathit{stop}\colon \mathbf{w_1}\triangleleft\mathit{stop}.\,\mathbf{w_2}\triangleleft\mathit{stop}.\,\mathbf{0}\}$$

To read this example, it is enough to know that $\mu X.\,P$ denotes guarded recursion, $\mathbf{m}\triangleright\{\ldots\}$ branches on a label received from $\mathbf{m}$, and $\mathbf{q}\triangleleft l.\,P$ is the dual selection of label l at $\mathbf{q}$; inputs, outputs, and termination are written $\mathbf{m}?(x).P$, $\mathbf{r}!\langle \mathrm{e}\rangle.P$, and $\mathbf{0}$. Thus the reducer receives both worker results and chooses between *cont* and *stop*, while the manager dispatches work to the workers and follows the reducer's verdict. Together these form the session $M = \mathbf{m} :: P_{\mathbf{m}} \mid \mathbf{w_1} :: P_{\mathbf{w_1}} \mid \mathbf{w_2} :: P_{\mathbf{w_2}} \mid \mathbf{r} :: P_{\mathbf{r}}$.

A *local type* abstracts a process to its communication structure, replacing concrete values with base types: for instance, the worker's receive $\mathbf{m}?(x)$ of a float becomes input type $\mathbf{m}?\langle\mathtt{float}\rangle$. From M, we can infer typing context $\Delta = \{\mathbf{m} : \mathsf{T}_{\mathbf{m}}, \mathbf{w_1} : \mathsf{T}_{\mathbf{w_1}}, \mathbf{w_2} : \mathsf{T}_{\mathbf{w_2}}, \mathbf{r} : \mathsf{T}_{\mathbf{r}}\}$ where:

$$\mathsf{T}_{\mathbf{w_i}} = \mu\,\mathbf{t}.\,\mathbf{m}\,\&\,\{\mathit{map}\colon \mathbf{m}?\langle\mathtt{float}\rangle;\mathbf{r}!\langle\mathtt{float}\rangle;\mathbf{t},\ \mathit{stop}\colon \mathsf{end}\} \qquad (i=1,2)$$

$$\mathsf{T}_{\mathbf{r}} = \mu\,\mathbf{t}.\,\mathbf{w_1}?\langle\mathtt{float}\rangle;\mathbf{w_2}?\langle\mathtt{float}\rangle;\mathbf{m}\oplus\{\mathit{cont}\colon \mathbf{t},\ \mathit{stop}\colon \mathsf{end}\}$$

$$\mathsf{T}_{\mathbf{m}} = \mu\,\mathbf{t}.\,\mathbf{w_1}\oplus\{\mathit{map}\colon \mathbf{w_1}!\langle\mathtt{float}\rangle;\mathbf{w_2}\oplus\{\mathit{map}\colon \mathbf{w_2}!\langle\mathtt{float}\rangle;\ \mathbf{r}\,\&\,\{\mathit{cont}\colon \mathbf{t},\ \mathit{crash}\colon \mathsf{end},\ \mathit{stop}\colon \mathbf{w_1}\oplus\{\mathit{stop}\colon \mathbf{w_2}\oplus\{\mathit{stop}\colon \mathsf{end}\}\}\}\}\}$$

$\oplus\{\,\}/\&\{\,\}$ denote the selection/branching types, and we can check Δ is safe and live.

The extra *crash* branch is exactly where subsumption enters. The reducer type $\mathsf{T}_{\mathbf{r}}$ can select only *cont* or *stop*, so the intended global protocol need not mention *crash*. Let $\mathsf{T}_{\mathbf{m},\mathrm{pro}}$ be the type obtained from $\mathsf{T}_{\mathbf{m}}$ by deleting the *crash* branch. Then $\mathsf{T}_{\mathbf{m}} \leqslant \mathsf{T}_{\mathbf{m},\mathrm{pro}}$, since a branching type with more offered labels is a subtype. Writing

$$\Delta_{\mathrm{pro}} = \{\mathbf{m} : \mathsf{T}_{\mathbf{m},\mathrm{pro}},\ \mathbf{w_1} : \mathsf{T}_{\mathbf{w_1}},\ \mathbf{w_2} : \mathsf{T}_{\mathbf{w_2}},\ \mathbf{r} : \mathsf{T}_{\mathbf{r}}\}$$

the original session M is typable by Δ_{pro} via subsumption.

Global types represent the entire choreography from a bird's-eye view. We now build *global type* $\mathsf{G}_{\mathrm{map}}$ whose projection is Δ_{pro}:

$$\mathsf{G}_{\mathrm{map}} = \mu\mathbf{t}.\ \mathbf{m}\to\mathbf{w_1}\ \Big\{\mathit{map}\colon \mathbf{m}\to\mathbf{w_1}\ \langle\mathtt{float}\rangle;\mathbf{w_1}\to\mathbf{r}\ \langle\mathtt{float}\rangle;\ \mathbf{m}\to\mathbf{w_2}\ \Big\{\mathit{map}\colon \mathbf{m}\to\mathbf{w_2}\ \langle\mathtt{float}\rangle;\mathbf{w_2}\to\mathbf{r}\ \langle\mathtt{float}\rangle;\ \mathbf{r}\to\mathbf{m}\ \big\{\mathit{cont}\colon \mathbf{t},\ \mathit{stop}\colon\ \mathbf{m}\to\mathbf{w_1}\ \{\mathit{stop}\colon \mathbf{m}\to\mathbf{w_2}\ \{\mathit{stop}\colon \mathsf{end}\}\}\big\}\Big\}\Big\}$$

where $\mathbf{p}\to\mathbf{q}\ \langle\mathsf{B}\rangle$ denotes a message with type B from participant $\mathbf{p}$ to participant $\mathbf{q}$, and $\mathbf{p}\to\mathbf{q}\ \{l_i\colon\ldots\}$ a labelled choice.

Recall our aim is to show M is typable by the top-down system. For this running example, the projected context is slightly smaller than the inferred one: it omits the manager's unused *crash* branch. This is precisely why we need subsumption in the top-down system. The global type captures only the actual *cont*/*stop* choreography, while the process typing is still discharged because $\Delta \leqslant \Delta_{\mathrm{pro}}$ (Figure 1a).

Summary. Figure 1 gives the overall picture. The *top-down* approach (Figure 1a) starts with a global type G. Then the projection generates local types $\mathsf{T}_{1\,\mathrm{pro}}, \ldots, \mathsf{T}_{n\,\mathrm{pro}}$ for each participant, guaranteed to be safe and live. We type processes P_i against T_i where $\mathsf{T}_i \leqslant \mathsf{T}_{i\,\mathrm{pro}}$ using the subsumption rule. Then $M = \mathbf{p_1} :: P_1 \mid \cdots \mid \mathbf{p_n} :: P_n$ will interact satisfying liveness (*correctness by construction*).

In the *bottom-up* approach (Figure 1b), we start with processes $P_1, \ldots, P_n$ and infer their principal local types $\mathsf{T}_{1\,\min}, \ldots, \mathsf{T}_{n\,\min}$ (by the type inference algorithm in [112]). We then check whether a set of local types satisfies safety and liveness.

The diagram in Figure 1c outlines our *inference* algorithm, which can construct a global type $\mathsf{G}_{\inf}$ from any safe and live context. Applying it to this running example, the inference starts from $\Delta = \{\mathbf{m} : \mathsf{T}_{\mathbf{m}}, \mathbf{w_1} : \mathsf{T}_{\mathbf{w_1}}, \mathbf{w_2} : \mathsf{T}_{\mathbf{w_2}}, \mathbf{r} : \mathsf{T}_{\mathbf{r}}\}$. It first follows the deterministic prefix $\mathbf{m} \to \mathbf{w_1}$, $\mathbf{m} \to \mathbf{w_2}$, $\mathbf{w_1} \to \mathbf{r}$, $\mathbf{w_2} \to \mathbf{r}$. Note that some of these interactions involve disjoint participant pairs (e.g. $\mathbf{m} \to \mathbf{w_2}$ and $\mathbf{w_1} \to \mathbf{r}$), so the synthesis must choose an order for them. As we shall see in §5, the global type semantics allow independent interactions to commute, so this choice does not affect the set of traces; the algorithm uses a fair schedule to ensure every enabled pair is eventually selected. Then it detects the choice $\mathbf{r} \to \mathbf{m}$ between *cont* and *stop*. The *cont* branch returns to the initial context; the *stop* branch has $\mathbf{m}$ select *stop* for $\mathbf{w_1}$ and $\mathbf{w_2}$ and terminates. When the initial context is revisited, synthesis binds it with recursion variable $\mathbf{t}$, yielding $\mathsf{G}_{\inf} = \mathsf{G}_{\mathrm{map}}$.

If T_i is a type of P_i as in Figure 1a, then $\mathsf{G}_{\inf} \subseteq \mathsf{G}$ (i.e., G has at least all traces of $\mathsf{G}_{\inf}$). We call $\mathsf{G}_{\inf}$ a *principal global type* of the typing context $\Delta = \{\mathbf{p_1}{:}\mathsf{T}_1, \ldots, \mathbf{p_n}{:}\mathsf{T}_n\}$.

Figure 1d shows that type $\mathsf{T}_{i\,\min}$ inferred from P_i is the minimum (principal) type of P_i [112, Sec. 5.2]. Hence a global type obtained by our inference is the *principal (minimum) global type* which can type M. Since Δ is the minimum against M, $\mathsf{G}_{\inf} = \mathsf{G}_{\mathrm{map}}$ is a principal type of M.

To illustrate the role of subtyping in more detail, consider replacing the reducer with a variant $P'_{\mathbf{r}}$ that stops after one round with its principal type $\mathsf{T}'_{\mathbf{r}\,\min}$:

$$P'_{\mathbf{r}} = \mathbf{w_1}?(x_1).\mathbf{w_2}?(y_1).\mathbf{m}\triangleleft \mathit{stop}.\,\mathbf{0} \qquad \text{with} \quad \mathsf{T}'_{\mathbf{r}\,\min} = \mathbf{w_1}?\langle \mathtt{float}\rangle;\mathbf{w_2}?\langle \mathtt{float}\rangle;\mathbf{m}\oplus\{\mathit{stop}\colon \mathsf{end}\}$$

Then we have $\mathsf{T}'_{\mathbf{r}\,\min} \leqslant \mathsf{T}_{\mathbf{r}}$. Therefore $P'_{\mathbf{r}}$ is typable by the original context Δ and by G. However, because $P'_{\mathbf{r}}$ has strictly fewer behaviours than $P_{\mathbf{r}}$, the session $M' = \mathbf{m} :: P_{\mathbf{m}} \mid \mathbf{w_1} :: P_{\mathbf{w_1}} \mid \mathbf{w_2} :: P_{\mathbf{w_2}} \mid \mathbf{r} :: P'_{\mathbf{r}}$ has fewer traces, yielding a smaller principal global type $\mathsf{G}_{\min}$ such that ($\subseteq$ here denotes trace inclusion; see Definition 5.4) $\mathsf{G}_{\min} \subseteq \mathsf{G}_{\inf} \subseteq \mathsf{G}$:

$$\mathsf{G}_{\min} = \mathbf{m}\to\mathbf{w_1}\left\{\mathit{map}\colon \mathbf{m}\to\mathbf{w_1}\,\langle \mathtt{float}\rangle;\mathbf{m}\to\mathbf{w_2}\left\{\mathit{map}\colon \mathbf{m}\to\mathbf{w_2}\,\langle \mathtt{float}\rangle;\right.\right.$$
$$\left.\left.\mathbf{w_1}\to\mathbf{r}\,\langle \mathtt{float}\rangle;\ \mathbf{w_2}\to\mathbf{r}\,\langle \mathtt{float}\rangle;\mathbf{r}\to\mathbf{m}\left\{\mathit{stop}\colon \mathbf{m}\to\mathbf{w_1}\,\{\mathit{stop}\colon \mathbf{m}\to\mathbf{w_2}\,\{\mathit{stop}\colon \mathsf{end}\}\}\right\}\right\}\right\}$$

The remainder of the paper formalises this development. We introduce the session calculus and its properties (§3), then the type language and projection (§4), followed by the operational semantics of types (§5). The core technical contribution is the synthesis algorithm and its correctness proof (§6), which together yield the completeness result (§7).

3 MULTIPARTY SESSION CALCULUS

This section introduces the multiparty synchronous session calculus from [47], which suffices for our purposes.

Syntax. *Expressions* ($\mathtt{e}, \mathtt{e}', \ldots$) are variables ($x, y, z, \ldots$), a value $\mathtt{v}$ (either a natural $\mathtt{n}$, an integer $\mathtt{i}$, a float f, or a boolean $\mathtt{true}/\mathtt{false}$), or are built by operators such as $\neg$, $\surd$, $\oplus$, $\vee$, $+$, $-$, $*$, $/$, or $\leq$. The nullary $\mathtt{rand()}$ returns a random float in $[0, 1]$. The operator $\oplus$ models non-determinism: $\mathtt{e}_1 \oplus \mathtt{e}_2$ may yield either $\mathtt{e}_1$ or $\mathtt{e}_2$.

$$\mathtt{e} \ ::= \ \mathtt{true} \mid \mathtt{false} \mid \mathtt{n} \mid \mathtt{i} \mid f \mid x \mid \mathtt{rand()} \mid \mathtt{e}\vee\mathtt{e} \mid \neg\mathtt{e} \mid \surd\mathtt{e} \mid \mathtt{e}{+}\mathtt{e} \mid \mathtt{e}{-}\mathtt{e} \mid \mathtt{e}{*}\mathtt{e} \mid \mathtt{e}/\mathtt{e} \mid \mathtt{e}\oplus\mathtt{e} \mid \mathtt{e}{\leq}\mathtt{e}$$

Processes ($P, Q, \ldots$) and *multiparty sessions* ($M, M', \ldots$) are:

$$P \ ::= \ \mathbf{0} \mid \mathbf{p}!\langle \mathtt{e}\rangle.P \mid \mathbf{p}?(x).P \mid \mathbf{p}\triangleleft l.\,P \mid \mathbf{p}\triangleright\{\,l_i : P_i\,\}_{i\in I} \mid \mathbf{if}\ \mathtt{e}\ \mathbf{then}\ P\ \mathbf{else}\ Q \mid \mu X.\,P \mid X$$
$$M \ ::= \ \mathbf{p} :: P \mid M \mid M'$$

$$
\begin{array}{lll}
\text{[R-COMM]} & \mathbf{p} :: \mathbf{q}?(x).P \mid \mathbf{q} :: \mathbf{p}!\langle \mathrm{e} \rangle.Q \mid M \longrightarrow \mathbf{p} :: P[\mathrm{v}/x] \mid \mathbf{q} :: Q \mid M & (\mathrm{e} \downarrow \mathrm{v}) \\
\text{[R-BRA]} & \mathbf{p} :: \mathbf{q} \triangleleft l_k.P \mid \mathbf{q} :: \mathbf{p} \triangleright \{l_i : P_i\}_{i \in I} \mid M \longrightarrow \mathbf{p} :: P \mid \mathbf{q} :: P_k \mid M & (k \in I) \\
\text{[T-COND]} & \mathbf{p} :: \mathbf{if}\ \mathrm{e}\ \mathbf{then}\ P\ \mathbf{else}\ Q \mid M \longrightarrow \mathbf{p} :: P \mid M & (\mathrm{e} \downarrow \mathtt{true}) \\
\text{[F-COND]} & \mathbf{p} :: \mathbf{if}\ \mathrm{e}\ \mathbf{then}\ P\ \mathbf{else}\ Q \mid M \longrightarrow \mathbf{p} :: Q \mid M & (\mathrm{e} \downarrow \mathtt{false}) \\
\text{[R-STR]} & M_1 \Rrightarrow M_2 \quad M_2 \longrightarrow M_3 \quad M_3 \Rrightarrow M_4 \implies M_1 \longrightarrow M_4 & \\
\text{[V-ERR]} & \mathbf{p} :: \mathbf{if}\ \mathrm{e}\ \mathbf{then}\ P\ \mathbf{else}\ Q \mid M \longrightarrow \mathrm{error} & (\mathrm{e} \downarrow \mathrm{v} \text{ and } \mathrm{v} \notin \{\mathtt{true}, \mathtt{false}\}) \\
\text{[C-ERR]} & \mathbf{p} :: \mathbf{q} \triangleleft l_k.P \mid \mathbf{q} :: \mathbf{p} \triangleright \{l_i : P_i\}_{i \in I} \mid M \longrightarrow \mathrm{error} & (k \notin I)
\end{array}
$$

Fig. 3. Reductions of sessions

$\mathbf{p}, \mathbf{q}, \mathbf{r}, \mathbf{s}, \ldots \in \mathbb{R} = \{\mathbf{p_0}, \ldots, \mathbf{p_n}\}$ are *participants*; $l, l', \ldots \in \mathbb{L}$ are *labels*. Nil $\mathbf{0}$ denotes termination; output $\mathbf{p}!\langle \mathrm{e} \rangle.Q$ sends the value of e to $\mathbf{p}$; input $\mathbf{p}?(x).Q$ receives from $\mathbf{p}$; selection $\mathbf{p} \triangleleft l.\, Q$ selects l at $\mathbf{p}$ (internal choice); branching $\mathbf{p} \triangleright \{l_i : P_i\}_{i \in I}$ ($|I| \geq 1$) accepts any l_i from $\mathbf{p}$ with continuation P_i (external choice). Conditionals $\mathbf{if}\ \mathrm{e}\ \mathbf{then}\ P\ \mathbf{else}\ Q$ and recursion $\mu X.\, P$ are standard. Recursion is *guarded* (e.g. $\mu X.\, X$ is invalid, but $\mu X.\, \mathbf{p}!\langle \mathrm{e} \rangle.X$ is valid). A process P playing role $\mathbf{p}$ is written $\mathbf{p} :: P$. We write $\prod_{i \in I} \mathbf{p_i} :: P_i$ for the composition of processes into a multiparty session.

Reductions. The value v of e (notation $\mathrm{e} \downarrow \mathrm{v}$) is computed by a standard big-step semantics. The full rules are given in Appendix A.1. The internal choice $\mathrm{e}_1 \oplus \mathrm{e}_2$ evaluates to either branch (see [47], Table 1). The *reduction rules of multiparty sessions* are in Figure 3.

Rule [R-COMM] is communication between an output and input; [R-BRA] is selection/branching. Rules [T-COND] and [F-COND] are standard, and [R-STR] closes reductions under *structure precongruence* $\Rrightarrow$, defined by:

$$\mathbf{p} :: \mu X.\, P \mid M \;\Rrightarrow\; \mathbf{p} :: P[\mu X.\, P/X] \mid M \qquad \prod_{i \in I} \mathbf{p_i} :: P_i \;\Rrightarrow\; \prod_{j \in J} \mathbf{p_j} :: P_j \;\; (I \text{ is a permutation of } J)$$

[V-ERR,C-ERR] define value and session errors. $\longrightarrow^*$ denotes the reflexive–transitive closure of $\longrightarrow$.

Properties. We recall properties of a session M from [99, Def. 5.1].

Definition 3.1 (Properties). A session M is:

(1) *communication safe* iff $M \longrightarrow^* M'$ implies $M' \not\Rightarrow$ error.
(2) *live* iff $M \longrightarrow^* M' \Rrightarrow M_1 \mid \mathbf{p} :: P$ implies:
  (a) If $P = \mathbf{q}?(x).P'$ then $\exists M_2, \mathrm{v}.\ M' \longrightarrow^* M_2 \mid \mathbf{p} :: P'[\mathrm{v}/x]$.
  (b) If $P = \mathbf{q}!\langle \mathrm{e} \rangle.P'$ then $\exists M_2.\ M' \longrightarrow^* M_2 \mid \mathbf{p} :: P'$.
  (c) If $P = \mathbf{q} \triangleleft l.\, P'$ then $\exists M_2.\ M' \longrightarrow^* M_2 \mid \mathbf{p} :: P'$.
  (d) If $P = \mathbf{q} \triangleright \{l_i : P_i\}_{i \in I}$ then $\exists M_2, j \in I.\ M' \longrightarrow^* M_2 \mid \mathbf{p} :: P_j$.

Thus M is *communication safe* if no errors arise, and *live* if every pending action eventually communicates. Since these properties are undecidable on processes, the bottom-up typing system checks the corresponding properties on *typing contexts* (§5), which soundly imply the process-level guarantees.

Remark 3.1 (Process liveness). *Liveness for branching (Definition 3.1 (2d)) requires that* some *branch $j \in I$ is eventually taken. Another branch may never fire. Consider:*

$$\begin{array}{lll} M_1 & = & \mathbf{p} :: \mu X.\, \mathbf{q} \triangleright \{l_1 : X,\ l_2 : \mathbf{0}\} \mid \mathbf{q} :: \mu X.\, \mathbf{p} \triangleleft l_1.X \\ M_2 & = & \mathbf{p} :: \mu X.\, \mathbf{q} \triangleright \{l_1 : X,\ l_2 : \mathbf{r}!\langle \mathrm{e} \rangle.\mathbf{0}\} \mid \mathbf{q} :: \mu X.\, \mathbf{p} \triangleleft l_1.X \mid \mathbf{r} :: \mathbf{p}?(x).\mathbf{0} \end{array}$$

M_1 is live (always takes l_1), but M_2 is not live: the l_2 branch requires $\mathbf{p}$ and $\mathbf{r}$ to interact, which never happens while $\mathbf{p}$ and $\mathbf{q}$ loop on l_1.

4 GLOBAL AND LOCAL TYPES

This section introduces global and local types and coinductive full-merging projection.

4.1 Syntax of Global and Local Types and Subtyping

Global types describe interactions between all participants.

Definition 4.1 (Global types). Base types ($\mathsf{B}, \mathsf{B}', \ldots$) and *global types* ($\mathsf{G}, \mathsf{G}', \ldots$) are defined by:

$$\mathsf{B} ::= \mathtt{bool} \mid \mathtt{nat} \mid \mathtt{int} \mid \mathtt{float} \qquad \mathsf{G} ::= \mathtt{end} \mid \mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle ; \mathsf{G}' \mid \mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I} \mid \mu \mathbf{t}.\mathsf{G} \mid \mathbf{t}$$

The constructor $\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle ; \mathsf{G}$ denotes a payload communication from $\mathbf{p}$ to $\mathbf{q}$, while $\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I}$ denotes a labelled choice where $\mathbf{p}$ selects at $\mathbf{q}$. Recursion is guarded. The set of *participants* of a global type G is defined as: $\mathrm{pt}(\mathbf{t}) = \mathrm{pt}(\mathtt{end}) = \emptyset$; $\mathrm{pt}(\mu \mathbf{t}.\mathsf{G}) = \mathrm{pt}(\mathsf{G})$; $\mathrm{pt}(\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle ; \mathsf{G}) = \mathrm{pt}(\mathsf{G}) \cup \{\mathbf{p}, \mathbf{q}\}$; $\mathrm{pt}(\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I}) = (\bigcup_{i \in I} \mathrm{pt}(\mathsf{G}_i)) \cup \{\mathbf{p}, \mathbf{q}\}$. We denote $\mathrm{ftv}(\mathsf{G})$ as the set of *free type variables* in G and if $\mathrm{ftv}(\mathsf{G}) = \emptyset$, we call G *closed*. We define $\mathrm{unfold}(\mathsf{G}) = \mathrm{unfold}(\mathsf{G}'[\mu \mathbf{t}.\mathsf{G}'/\mathbf{t}])$ if $\mathsf{G} = \mu \mathbf{t}.\mathsf{G}'$; otherwise $\mathrm{unfold}(\mathsf{G}) = \mathsf{G}$.

Local types specify individual participant behaviour; *typing contexts* abstract process behaviour.

Definition 4.2 (Local types and typing contexts). Local types ($\mathsf{T}, \mathsf{T}', \ldots$) and typing contexts ($\Delta, \Delta', \ldots$) are defined by:

$$\mathsf{T} ::= \mathtt{end} \mid \mathbf{p}!\langle \mathsf{B} \rangle ; \mathsf{T} \mid \mathbf{p}?\langle \mathsf{B} \rangle ; \mathsf{T} \mid \mathbf{p} \oplus \{l_i : \mathsf{T}_i\}_{i \in I} \mid \mathbf{p} \,\&\, \{l_i : \mathsf{T}_i\}_{i \in I} \mid \mu \mathbf{t}.\mathsf{T} \mid \mathbf{t} \qquad \Delta ::= \Delta, \Delta' \mid \mathbf{p} : \mathsf{T}$$

Local types are endpoint views of protocols: $\mathbf{p}!\langle \mathsf{B} \rangle ; \mathsf{T}$ and $\mathbf{p}?\langle \mathsf{B} \rangle ; \mathsf{T}$ are output and input, while $\mathbf{p} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$ and $\mathbf{p} \,\&\, \{l_i : \mathsf{T}_i\}_{i \in I}$ are the dual selection/branching behaviours. Recursion is guarded, and we often omit trailing $\mathtt{end}$. We use $\mathrm{ftv}(\mathsf{T})$ for the set of *free type variables* in T, and $\mathrm{unfold}(\mathsf{T})$ for unfolding. Typing contexts ($\Delta, \Delta', \ldots$) are mappings from participants $\mathbf{p_0}, \ldots, \mathbf{p_{n-1}}$ to local types, and are non-empty.

Definition 4.3 (Subtyping). The *synchronous subtyping relation* is coinductively defined as:

$$[\mathrm{S1}]\ \frac{\mathsf{T} \leqslant \mathsf{T}'}{\mathbf{p}!\langle \mathsf{B} \rangle ; \mathsf{T} \leqslant \mathbf{p}!\langle \mathsf{B} \rangle ; \mathsf{T}'} \qquad [\mathrm{S2}]\ \frac{\mathsf{T} \leqslant \mathsf{T}'}{\mathbf{p}?\langle \mathsf{B} \rangle ; \mathsf{T} \leqslant \mathbf{p}?\langle \mathsf{B} \rangle ; \mathsf{T}'} \qquad [\mathrm{S5}]\ \frac{\mathsf{T}[\mu \mathbf{t}.\mathsf{T}/\mathbf{t}] \leqslant \mathsf{T}'}{\mu \mathbf{t}.\mathsf{T} \leqslant \mathsf{T}'} \qquad [\mathrm{S6}]\ \frac{\mathsf{T} \leqslant \mathsf{T}'[\mu \mathbf{t}.\mathsf{T}'/\mathbf{t}]}{\mathsf{T} \leqslant \mu \mathbf{t}.\mathsf{T}'}$$

$$[\mathrm{S3}]\ \frac{I \subseteq J \qquad \forall i \in I : \mathsf{T}_i \leqslant \mathsf{T}'_i}{\mathbf{p} \oplus \{l_i : \mathsf{T}_i\}_{i \in I} \leqslant \mathbf{p} \oplus \{l_i : \mathsf{T}'_i\}_{i \in J}} \qquad [\mathrm{S4}]\ \frac{J \subseteq I \qquad \forall i \in J : \mathsf{T}_i \leqslant \mathsf{T}'_i}{\mathbf{p} \,\&\, \{l_i : \mathsf{T}_i\}_{i \in I} \leqslant \mathbf{p} \,\&\, \{l_i : \mathsf{T}'_i\}_{i \in J}} \qquad [\mathrm{S7}]\ \frac{}{\mathtt{end} \leqslant \mathtt{end}}$$

Rule [S3] is covariant: subtypes offer *fewer* selections ($I \subseteq J$). Rule [S4] is contravariant: subtypes offer *more* branches ($J \subseteq I$). We extend $\leqslant$ to typing contexts as: $\Delta \leqslant \Delta'$ if $\mathrm{dom}(\Delta) = \mathrm{dom}(\Delta')$ and $\forall \mathbf{p} \in \mathrm{dom}(\Delta), \Delta(\mathbf{p}) \leqslant \Delta'(\mathbf{p})$.

4.2 Merging and Projections

A participant uninvolved in a choice must anticipate different continuations. The *end-point projection* (EPP) merges these into a single compatible local type. We use *coinductive full merging*, which creates a local type that is a *subtype* of each continuation.

The coinductive full merge ($\sqcap$) *merges* local types by combining continuations in the branching and recursively merging continuations on shared labels.

Definition 4.4 (Coinductive merge relation). The coinductive merge relation between a set of local types and a local type is defined by the following rules. We use the coinductive full merge relation ($\sqcap$), where the index sets I_j in rule [M5] may differ.

$$[\text{M1}]\ \frac{}{\{\mathsf{end}\} \sqcap \mathsf{end}} \qquad [\text{M2}]\ \frac{\{\mathsf{T_j}\}_{j\in J} \sqcap \mathsf{T}}{\{\mathbf{p}!\langle \mathsf{B}\rangle;\mathsf{T_j}\}_{j\in J} \sqcap \mathbf{p}!\langle \mathsf{B}\rangle;\mathsf{T}} \qquad [\text{M3}]\ \frac{\{\mathsf{T_j}\}_{j\in J} \sqcap \mathsf{T}}{\{\mathbf{p}?\langle \mathsf{B}\rangle;\mathsf{T_j}\}_{j\in J} \sqcap \mathbf{p}?\langle \mathsf{B}\rangle;\mathsf{T}}$$

$$[\text{M4}]\ \frac{\forall i \in I : \{\mathsf{T_{i,j}}\}_{j\in J} \sqcap \mathsf{T_i}}{\{\mathbf{p} \oplus \{l_i : \mathsf{T_{i,j}}\}_{i\in I}\}_{j\in J} \sqcap \mathbf{p} \oplus \{l_i : \mathsf{T_i}\}_{i\in I}} \qquad [\text{M5}]\ \frac{\forall i \in I : \{\mathsf{T_{i,j}}\}_{j\in J|i\in I_j} \sqcap \mathsf{T_i}}{\{\mathbf{p} \,\&\, \{l_i : \mathsf{T_{i,j}}\}_{i\in I_j}\}_{j\in J} \sqcap \mathbf{p} \,\&\, \{l_i : \mathsf{T_i}\}_{i\in \bigcup_{j\in J} I_j}}$$

$$[\text{M6}]\ \frac{S \cup \{\mathsf{T}[\mu \mathsf{t}.\mathsf{T}/\mathsf{t}]\} \sqcap \mathsf{T}'}{S \cup \{\mu \mathsf{t}.\mathsf{T}\} \sqcap \mathsf{T}'} \qquad [\text{M7}]\ \frac{S \sqcap \mathsf{T}[\mu \mathsf{t}.\mathsf{T}/\mathsf{t}]}{S \sqcap \mu \mathsf{t}.\mathsf{T}}$$

Rule [M4] requires identical labels across selections; [M5] allows their union. Full merge lets uninvolved participants offer additional choices while preserving observable behaviour.

Using the coinductive merge, we define a *coinductive projection* from a global type to a local type, parameterised by the projected participant $\mathbf{p}$. To see why both *full* and *coinductive* merging are needed, consider projecting $\mathsf{G_{map}}$ onto $\mathbf{w_1}$. At the choice $\mathbf{r} \to \mathbf{m}\ \{\mathit{cont}\colon \ldots,\ \mathit{stop}\colon \ldots\}$, $\mathbf{w_1}$ is uninvolved, so rule [PR6] projects each branch independently and merges. The *cont* branch recurses, giving label set $\{\mathit{map}, \mathit{stop}\}$; the *stop* branch gives only $\{\mathit{stop}\}$. A standard merge requires identical label sets and would reject this. Full merge ([M5]) takes the union, and coinductive unfolding ([M6]) handles the μ-type, yielding $\mathsf{T_{w_1}}$ itself.

Definition 4.5 (Coinductive projection). The coinductive projection from a global type to a local type is defined by the following rules. When the merge used in rule [PR6] is the full merge ($\sqcap$), we obtain the projection relation *coinductive-full projection* ($\upharpoonright_{\mathbf{p}}$), which is the one used in this paper.

$$[\text{PR1}]\ \frac{\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T}}{\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B}\rangle;\mathsf{G} \upharpoonright_{\mathbf{p}} \mathbf{q}!\langle \mathsf{B}\rangle;\mathsf{T}} \qquad [\text{PR2}]\ \frac{\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T}}{\mathbf{q} \to \mathbf{p}\ \langle \mathsf{B}\rangle;\mathsf{G} \upharpoonright_{\mathbf{p}} \mathbf{q}?\langle \mathsf{B}\rangle;\mathsf{T}} \qquad [\text{PR3}]\ \frac{\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T} \qquad \mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}}{\mathbf{q} \to \mathbf{r}\ \langle \mathsf{B}\rangle;\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T}}$$

$$[\text{PR4}]\ \frac{\forall i \in I : \mathsf{G_i} \upharpoonright_{\mathbf{p}} \mathsf{T_i}}{\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G_i}\}_{i\in I} \upharpoonright_{\mathbf{p}} \mathbf{q} \oplus \{l_i : \mathsf{T_i}\}_{i\in I}} \qquad [\text{PR5}]\ \frac{\forall i \in I : \mathsf{G_i} \upharpoonright_{\mathbf{p}} \mathsf{T_i}}{\mathbf{q} \to \mathbf{p}\ \{l_i : \mathsf{G_i}\}_{i\in I} \upharpoonright_{\mathbf{p}} \mathbf{q} \,\&\, \{l_i : \mathsf{T_i}\}_{i\in I}}$$

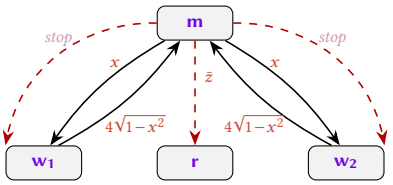
$$[\text{PR6}]\ \frac{\forall i \in I : \mathsf{G_i} \upharpoonright_{\mathbf{p}} \mathsf{T_i} \qquad \{\mathsf{T_i}\}_{i\in I} \sqcap \mathsf{T} \qquad \mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}}{\mathbf{q} \to \mathbf{r}\ \{l_i : \mathsf{G_i}\}_{i\in I} \upharpoonright_{\mathbf{p}} \mathsf{T}} \qquad [\text{PR9}]\ \frac{\mathsf{G}[\mu \mathsf{t}.\mathsf{G}/\mathsf{t}] \upharpoonright_{\mathbf{p}} \mathsf{T} \qquad \mathbf{p} \in \mathsf{pt}(\mathsf{G})}{\mu \mathsf{t}.\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T}}$$

$$[\text{PR10}]\ \frac{\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{T}[\mu \mathsf{t}.\mathsf{T}/\mathsf{t}]}{\mathsf{G} \upharpoonright_{\mathbf{p}} \mu \mathsf{t}.\mathsf{T}} \qquad [\text{PR11}]\ \frac{}{\mathsf{end} \upharpoonright_{\mathbf{p}} \mathsf{end}} \qquad [\text{PR12}]\ \frac{\mathbf{p} \notin \mathsf{pt}(\mathsf{G})}{\mu \mathsf{t}.\mathsf{G} \upharpoonright_{\mathbf{p}} \mathsf{end}}$$

These rules project communications to the sender or receiver, skip uninvolved participants, merge uninvolved branches, and handle recursion coinductively; rule [PR9] requires $\mathbf{p}$ to occur in the recursive body, while rule [PR12] projects to end when $\mathbf{p}$ is absent from it. To ensure projections are live, we need to know which participants are *unavoidable*, i.e., must eventually be involved after some bounded number of steps in any fair path.

Definition 4.6 (Subterms and Unavoidable Set). The *subterms* of a global type G are the set of (unfolded) global types that appear in G. The *unavoidable set* of G is the set of participants that must eventually be involved in any (fair) path from G. Let $\mathsf{G}' = \mathsf{unfold}(\mathsf{G})$. Then $\mathsf{sub}(\mathsf{G})$ and $\mathsf{unav}(\mathsf{G})$ are defined as the least sets satisfying the following:

G'	$\mathsf{sub}(\mathsf{G})$	$\mathsf{unav}(\mathsf{G})$
end	$\{\mathsf{end}\}$	$\emptyset$
$\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B}\rangle;\mathsf{G}''$	$\{\mathsf{G}'\} \cup \mathsf{sub}(\mathsf{G}'')$	$\{\mathbf{p}, \mathbf{q}\} \cup \mathsf{unav}(\mathsf{G}'')$
$\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G_i}\}_{i\in I}$	$\{\mathsf{G}'\} \cup \bigcup_{i\in I} \mathsf{sub}(\mathsf{G_i})$	$\{\mathbf{p}, \mathbf{q}\} \cup \bigcap_{i\in I} \mathsf{unav}(\mathsf{G_i})$



$$\mathsf{G}_{\mathsf{unbal}} = \mu \mathsf{t}.\ \mathbf{m} \to \mathbf{w}_1 \left\{ \mathit{map}\colon \mathbf{m} \to \mathbf{w}_1\ \langle \mathtt{float} \rangle;\right.$$
$$\mathbf{m} \to \mathbf{w}_2 \left\{ \mathit{map}\colon \mathbf{m} \to \mathbf{w}_2\ \langle \mathtt{float} \rangle;\right.$$
$$\mathbf{w}_1 \to \mathbf{m}\ \langle \mathtt{float} \rangle;\ \mathbf{w}_2 \to \mathbf{m}\ \langle \mathtt{float} \rangle;$$
$$\mathbf{m} \to \mathbf{w}_1 \left\{ \mathit{cont}\colon \mathbf{m} \to \mathbf{w}_2\ \{ \mathit{cont}\colon \mathbf{t} \},\right.$$
$$\left.\left. \mathit{stop}\colon \mathbf{m} \to \mathbf{w}_2\ \{ \mathit{stop}\colon \mathbf{m} \to \mathbf{r}\ \langle \mathtt{float} \rangle; \mathsf{end} \} \right\} \right\}$$

Fig. 4. Unbalanced variant: workers send results to the manager, not the reducer. Dashed arrows indicate messages that only occur on termination.

We can now define a global type as *balanced* if no reachable participant can be avoided.

Definition 4.7 (Balanced, Definition 3.3 in [47]). A global type G is *balanced* if, for all $\mathsf{G}' \in \mathsf{sub}(\mathsf{G})$, we have that $\mathsf{unav}(\mathsf{G}') = \mathsf{pt}(\mathsf{G}')$.

Without balancedness, the top-down system does not guarantee liveness. To illustrate, consider a variant of our Monte Carlo example in §2 where workers send results back to the manager instead of the reducer, and only on termination does the manager forward the aggregated result to the reducer (Figure 4).

Let G' be the subterm at the choice $\mathbf{m} \to \mathbf{w}_1\ \{\mathit{cont}\colon \ldots, \mathit{stop}\colon \ldots\}$. The reducer $\mathbf{r}$ is reachable from G' (via the *stop* branch), so $\mathbf{r} \in \mathsf{pt}(\mathsf{G}')$. However, the *cont* branch loops back without ever involving $\mathbf{r}$, so $\mathbf{r} \notin \mathsf{unav}(\mathsf{G}')$ (the unavoidable set uses $\bigcap$ over branches, and $\mathbf{r}$ is absent from the *cont* branch). Since $\mathsf{pt}(\mathsf{G}') \neq \mathsf{unav}(\mathsf{G}')$, $\mathsf{G}_{\mathsf{unbal}}$ is *not balanced*. Concretely, the projections of $\mathsf{G}_{\mathsf{unbal}}$ can type a session in which the manager and workers loop indefinitely via the *cont* branch, never reaching the *stop* branch that communicates with $\mathbf{r}$, so the reducer starves and liveness fails. In balanced $\mathsf{G}_{\mathsf{map}}$ in §2, workers send to the reducer inside the loop, ensuring every participant is unavoidable at every subterm. See also [5, Example 3.3] and [56, Example 3.9] for unbalanced types.

5 LABELLED TRANSITION SEMANTICS

As a *behavioural* type system, the framework equips global and local types with operational semantics, allowing them to evolve during the execution of the session.

Definition 5.1 (Local type and typing context semantics). We define a transition relation $\xrightarrow{\zeta}$ for local types, $\xrightarrow{\alpha}$ for one-sided transitions of contexts, and $\xrightarrow{\ell}$ for two-sided synchronous transitions in contexts, where an observable U is either a payload from B or a label l, an action ζ is a send or receive of an observable, a tagged action $\alpha = \mathbf{p} : \zeta$ is accompanied by the performing participant, and an interaction $\ell = \mathbf{pq}{:}\langle \mathsf{U} \rangle$ is an action between two participants.

$$\mathsf{U} ::= \mathsf{B} \mid l \qquad \zeta ::= \mathbf{p}!\langle \mathsf{U} \rangle \mid \mathbf{p}?\langle \mathsf{U} \rangle \qquad \alpha ::= \mathbf{p} : \zeta \qquad \ell ::= \mathbf{pq}{:}\langle \mathsf{U} \rangle$$

$$[\mathrm{LR1}]\ \frac{}{\mathbf{p}!\langle \mathsf{B} \rangle;\mathsf{T} \xrightarrow{\mathbf{p}!\langle \mathsf{B} \rangle} \mathsf{T}} \qquad [\mathrm{LR2}]\ \frac{}{\mathbf{p}?\langle \mathsf{B} \rangle;\mathsf{T} \xrightarrow{\mathbf{p}?\langle \mathsf{B} \rangle} \mathsf{T}} \qquad [\mathrm{LR3}]\ \frac{j \in I}{\mathbf{p} \oplus \{l_i : \mathsf{T}_i\}_{i \in I} \xrightarrow{\mathbf{p}!\langle l_j \rangle} \mathsf{T}_j} \qquad [\mathrm{LR4}]\ \frac{j \in I}{\mathbf{p}\,\&\,\{l_i : \mathsf{T}_i\}_{i \in I} \xrightarrow{\mathbf{p}?\langle l_j \rangle} \mathsf{T}_j}$$

$$[\mathrm{LR5}]\ \frac{\mathsf{T}[\mu \mathsf{t}.\mathsf{T}/\mathsf{t}] \xrightarrow{\zeta} \mathsf{T}'}{\mu \mathsf{t}.\mathsf{T} \xrightarrow{\zeta} \mathsf{T}'} \qquad [\mathrm{LTAG}]\ \frac{\mathsf{T} \xrightarrow{\zeta} \mathsf{T}'}{\Delta, \mathbf{p} : \mathsf{T} \xrightarrow{\mathbf{p}:\zeta} \Delta, \mathbf{p} : \mathsf{T}'} \qquad [\mathrm{LENV}]\ \frac{\Delta_1 \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{U} \rangle} \Delta_1' \qquad \Delta_2 \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{U} \rangle} \Delta_2'}{\Delta_1, \Delta_2 \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta_1', \Delta_2'}$$

Rule [LR1] permits the sending of a payload of sort B to a participant $\mathbf{p}$, and rule [LR2] is its dual for receiving. Rule [LR3] sends label l_j to $\mathbf{p}$, continuing according to the continuation T_j. Rule [LR4] is

the dual of [LR3]. Rule [LR5] unfolds a recursive type. The one-sided transition (rule [LTAG]) tags the action with the participant that performs it. The relation $\xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle}$ captures two-sided synchronous reductions in contexts by combining the one-sided transitions ([LENV]).

For example, if $\mathrm{unfold}(\mathsf{T_m}) = \mathbf{w_1}!\langle \mathtt{float} \rangle; \mathsf{T}'$, then the transition is: $\mathsf{T_m} \xrightarrow{\mathbf{w_1}!\langle \mathtt{float} \rangle} \mathsf{T}'$.

Definition 5.2 (Global type semantics). The labelled transition relation of global types, written $\xrightarrow{\ell}$, is defined by the following rules.

$$[\text{GR1}]\ \frac{}{\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G} \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \mathsf{G}} \qquad [\text{GR2}]\ \frac{\mathsf{G} \xrightarrow{\mathbf{rs}:\langle \mathsf{U} \rangle} \mathsf{G}' \qquad \{\mathbf{r},\mathbf{s}\} \cap \{\mathbf{p},\mathbf{q}\} = \emptyset}{\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G} \xrightarrow{\mathbf{rs}:\langle \mathsf{U} \rangle} \mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G}'} \qquad [\text{GR3}]\ \frac{\mathsf{G}[\mu \mathbf{t}.\mathsf{G}/\mathbf{t}] \xrightarrow{\ell} \mathsf{G}'}{\mu \mathbf{t}.\mathsf{G} \xrightarrow{\ell} \mathsf{G}'}$$

$$[\text{GR4}]\ \frac{j \in I}{\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I} \xrightarrow{\mathbf{pq}:\langle l_j \rangle} \mathsf{G}_j} \qquad [\text{GR5}]\ \frac{\forall i \in I : \mathsf{G}_i \xrightarrow{\mathbf{rs}:\langle \mathsf{U} \rangle} \mathsf{G}_i' \quad \{\mathbf{r},\mathbf{s}\} \cap \{\mathbf{p},\mathbf{q}\} = \emptyset}{\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I} \xrightarrow{\mathbf{rs}:\langle \mathsf{U} \rangle} \mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i'\}_{i \in I}}$$

Rule [GR1] exposes the head communication of a message whose payload is of sort B, while [GR4] handles selections by emitting the chosen label. Recursive protocols evolve by one unfolding step under [GR3], mirroring [LR5] on the local side.

Rules [GR2] and [GR5] are key: they allow interactions between *disjoint* participant pairs to proceed inside the continuation of another interaction. Concretely, if $\{\mathbf{p},\mathbf{q}\} \cap \{\mathbf{r},\mathbf{s}\} = \emptyset$, then $\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle;\ \mathbf{r} \to \mathbf{s}\ \langle \mathsf{B}' \rangle;\ \mathsf{G}$ and $\mathbf{r} \to \mathbf{s}\ \langle \mathsf{B}' \rangle;\ \mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle;\ \mathsf{G}$ have exactly the same set of traces. Although global types are written as a sequential chain of interactions, they therefore naturally represent concurrent behaviour: any interleaving of independent interactions is semantically equivalent. This observation is central to our synthesis algorithm (§6.2), which must choose a particular interleaving when constructing a global type from a set of local types.

We write $\Delta \longrightarrow$ to mean $\exists \Delta', \ell.\ \Delta \xrightarrow{\ell} \Delta'$, and $\Delta \longrightarrow \Delta'$ to mean $\exists \ell.\ \Delta \xrightarrow{\ell} \Delta'$. Similarly, $\Delta \xrightarrow{\mathbf{pq}}$ means $\exists \mathsf{U}, \Delta'.\ \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$, and $\Delta \xrightarrow{\mathbf{pq}} \Delta'$ means $\exists \mathsf{U}.\ \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$. We write $\Delta \not\to$ to mean $\neg(\Delta \longrightarrow)$. The same conventions apply to local types, one-sided, and global type reductions ($\longrightarrow$, $\longrightarrow$, and $\longrightarrow$).

5.1 Fair and Live Reduction Sequences

Many interesting properties are defined in terms of possible reduction sequences of contexts and global types.

Definition 5.3 (Reduction sequences). A *reduction sequence* is a finite or infinite sequence of contexts related by synchronous transitions: $\Delta_0 \xrightarrow{\ell_0} \Delta_1 \xrightarrow{\ell_1} \Delta_2 \xrightarrow{\ell_2} \ldots$. We write $\mathrm{stuck}(\Delta)$ if $\Delta \not\to$ and $\mathrm{unstuck}(\Delta)$ if $\Delta \longrightarrow$.

The context is *stuck* if it cannot take any step, and *unstuck* if it can take at least one step. We are usually interested in *maximal* reduction sequences meaning that either the sequence is infinite or the final context is stuck.

Definition 5.4 (Traces). The *traces* of a global type G (resp. a context Δ) are the (finite or infinite) sequences of interactions labelling maximal reduction sequences using $\longrightarrow$ (resp. $\longrightarrow$):

$$Tr(\mathsf{G}) = \left\{ \ell_0\, \ell_1\, \ell_2\, \cdots \;\middle|\; \mathsf{G} = \mathsf{G}_0 \xrightarrow{\ell_0} \mathsf{G}_1 \xrightarrow{\ell_1} \mathsf{G}_2 \xrightarrow{\ell_2} \cdots \text{ is maximal} \right\}.$$

$$Tr(\Delta) = \left\{ \ell_0\, \ell_1\, \ell_2\, \cdots \;\middle|\; \Delta = \Delta_0 \xrightarrow{\ell_0} \Delta_1 \xrightarrow{\ell_1} \Delta_2 \xrightarrow{\ell_2} \cdots \text{ is maximal} \right\}.$$

We write $\mathsf{G}' \subseteq \mathsf{G}$ when $Tr(\mathsf{G}') \subseteq Tr(\mathsf{G})$.

While the term *safety* has many uses in the literature, we adopt a specific definition of a *safe* context. A context is *safe* if it cannot reach a state where there is a mismatch between the expectations of the send and receive endpoints.

Definition 5.5 (Safe contexts). A context Δ is *safe* if, for all contexts $\Delta \longrightarrow^* \Delta'$ reachable from Δ, we have that: if $\Delta' \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{U} \rangle}$ and $\Delta' \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{U}' \rangle}$, then $\Delta' \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle}$.

For example, in a safe context, if **p** is trying to send a `bool` to **q** then it should not be the case that **q** is trying to receive a `int`.

We also prove inversion lemmas for safe contexts (Lemmas B.1 and B.2, see Appendix B.1), which decompose one-sided and two-sided reductions into their constituent send/receive actions.

We say a maximal reduction sequence is *fair* in the sense that enabled interactions are always eventually taken.

Definition 5.6 (Fair and Live Reduction Sequences). A maximal reduction sequence $(\Delta_i \xrightarrow{\ell_i} \Delta_{i+1})_{i \in I}$ is *fair* when $\Delta_j \xrightarrow{\mathbf{pq}}$ for some $j \in I$ implies that $\exists n \geq j . \Delta_n \xrightarrow{\mathbf{pq}} \Delta_{n+1}$.

For example, we may have a context Δ which can self-loop in two different ways $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \Delta$ and $\Delta \xrightarrow{\mathbf{rs}:\langle \mathsf{B}' \rangle} \Delta$. The reduction sequence $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \ldots$ is unfair because the interaction between **r** and **s** is enabled but never taken.

Definition 5.7 (Live paths). A maximal reduction sequence $(\Delta_i \xrightarrow{\ell_i} \Delta_{i+1})_{i \in I}$ is *live* if

(1) if $\Delta_i \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{B} \rangle}$, then there exists a $j \in I, j \geq i$ such that $\Delta_j \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{B} \rangle}$
(2) if $\Delta_i \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{B} \rangle}$, then there exists a $j \in I, j \geq i$ such that $\Delta_j \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{B} \rangle}$
(3) if $\Delta_i \xrightarrow{\mathbf{p}:\mathbf{q}!\langle l \rangle}$, then there exists a $j \in I, j \geq i$ such that $\Delta_j \xrightarrow{\mathbf{q}:\mathbf{p}?\langle l' \rangle}$
(4) if $\Delta_i \xrightarrow{\mathbf{q}:\mathbf{p}?\langle l \rangle}$, then there exists a $j \in I, j \geq i$ such that $\Delta_j \xrightarrow{\mathbf{p}:\mathbf{q}!\langle l' \rangle}$

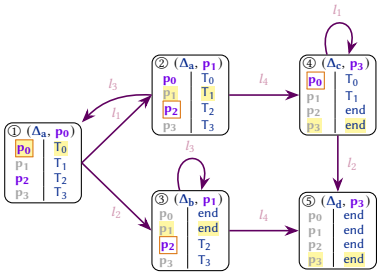
Continuing the example above, the context path $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{B} \rangle} \ldots$ is not live because the one-sided action from the perspective of **r** ($\Delta \xrightarrow{\mathbf{r}:\mathbf{s}!\langle \mathsf{B}' \rangle}$) is continuously enabled but never taken as part of the context path. We consider a liveness violation significant only if participants are starved *regardless of the scheduling of reductions*. Hence we define *live contexts* only under fair paths.

Definition 5.8 (Live contexts). A context Δ is *live* (written $\mathsf{live}(\Delta)$) if all fair reduction sequences starting from that context are live. We write $\mathsf{slive}(\Delta)$ if $\mathsf{safe}(\Delta)$ and $\mathsf{live}(\Delta)$.

Note that context liveness is a type-level analogue of the process liveness of Definition 3.1: rather than reasoning about all reachable process states, we check a semantic property of the typing context. The two notions are linked by soundness (Theorems 7.3 and 7.4): if $\mathsf{slive}(\Delta)$ and M is typed by Δ, then M is live in the sense of Definition 3.1.

5.2 Fair Scheduling

As established above, interactions between disjoint participant pairs commute in the global type semantics, so a context with several simultaneously enabled pairs admits many equivalent interleavings. To synthesise a global type that faithfully captures *all* behaviours of the local context, we must choose an interleaving that is *fair*: every enabled pair is eventually scheduled. Combined with the assumption that the context is live, a fair schedule guarantees that the synthesised global type preserves the semantics of each participant. We achieve fairness via a simple deterministic strategy, cycling through enabled senders in a fixed order, captured by a relation $\rightsquigarrow$.



(a) Reduction graph on states $(\Delta, \mathbf{p})$. Yellow = prioritised participant, grey = not enabled, orange outline = next-active sender.

$$\mathsf{T}_0 = \mu \mathsf{t}.\mathbf{p_1} \oplus \{l_1 : \mathsf{end},\ l_2 : \mathsf{t}\} \qquad \mathsf{T}_2 = \mu \mathsf{t}.\mathbf{p_3} \oplus \{l_3 : \mathsf{end},\ l_4 : \mathsf{t}\}$$
$$\mathsf{T}_1 = \mu \mathsf{t}.\mathbf{p_0} \,\&\, \{l_1 : \mathsf{end},\ l_2 : \mathsf{t}\} \qquad \mathsf{T}_3 = \mu \mathsf{t}.\mathbf{p_2} \,\&\, \{l_3 : \mathsf{end},\ l_4 : \mathsf{t}\}$$

$$\mathsf{G} = \mu \mathsf{t}_1.\ \mathbf{p_0} \to \mathbf{p_1} \left\{ \begin{array}{l} l_1 : \mu \mathsf{t}_2.\ \mathbf{p_2} \to \mathbf{p_3}\ \{l_3 : \mathsf{t}_1,\\ \quad l_4 : \mu \mathsf{t}_3.\ \mathbf{p_0} \to \mathbf{p_1}\ \{l_1 : \mathsf{t}_3,\ l_2 : \mathsf{end}\}\} \\ l_2 : \mu \mathsf{t}_4.\ \mathbf{p_2} \to \mathbf{p_3}\ \{l_3 : \mathsf{t}_4,\ l_4 : \mathsf{end}\} \end{array} \right\}$$

(b) Local types Δ (top) and synthesised global type G (bottom). The two disjoint pairs ($\mathbf{p_0}/\mathbf{p_1}$ and $\mathbf{p_2}/\mathbf{p_3}$) are interleaved by the round-robin schedule.

Fig. 5. Round-robin reduction graph with inferred global type. Nodes ① and ② share the same context but differ in priority; this distinction forces both pairs to be scheduled before the state repeats, so the synthesised global type captures all interleavings. The binder $\mu \mathsf{t}_2$ in G is redundant and can be removed in a post-pass.

We define this strategy using two functions: a set of *enabled senders*, which contains all participants who are the sender in an enabled communication pair at state Δ; the *next-active* function chooses the next participant who is the sender in an enabled communication pair from Δ.

Definition 5.9 (Enabled and next-active functions). Let n be the total number of participants and assume unstuck(Δ). When Δ can transition, we define:

$$\mathsf{enabled}(\Delta) = \{\, \mathbf{p} \mid \Delta \xrightarrow{\mathbf{pq}} \,\} \qquad \mathsf{next\text{-}active}(\mathbf{p_i}) = \arg\min_{\mathbf{p_j} \in \mathsf{enabled}(\Delta)} \left((j - i) \bmod n\right)$$

For example, if we are in a state with $\Delta \xrightarrow{\mathbf{p_0 p_1}}$ and $\Delta \xrightarrow{\mathbf{p_2 p_3}}$ then we have $\mathsf{enabled}(\Delta) = \{\mathbf{p_0}, \mathbf{p_2}\}$ and we have $\mathsf{next\text{-}active}(\mathbf{p_0}) = \mathsf{next\text{-}active}(\mathbf{p_3}) = \mathbf{p_0}$ and $\mathsf{next\text{-}active}(\mathbf{p_1}) = \mathsf{next\text{-}active}(\mathbf{p_2}) = \mathbf{p_2}$. The strategy cycles through participants, prioritising each one in turn. Hence $\rightsquigarrow$ is a relation between pairs of contexts and a prioritised participant.

Definition 5.10 (Round robin). We write $(\Delta, \mathbf{p_i}) \overset{\ell}{\rightsquigarrow} (\Delta', \mathbf{p_k})$ to show that the state Δ with participant $\mathbf{p_i}$ prioritised transitions to the state Δ' with participant $\mathbf{p_k}$ prioritised.

$$\mathbf{p_j} = \mathsf{next\text{-}active}(\mathbf{p_i}), \quad \ell = \mathbf{p_j q}{:}\langle \mathsf{U} \rangle, \quad \Delta \xrightarrow{\ell} \Delta', \quad k = (j + 1) \bmod n.$$

The two disjoint pairs in Figure 5 are truly concurrent, so the round-robin schedule must interleave them to explore all reachable states.

Example 5.11 (Round-robin execution of Figure 5). The context has two disjoint pairs ($\mathbf{p_0}/\mathbf{p_1}$ and $\mathbf{p_2}/\mathbf{p_3}$), and the execution proceeds as follows.

① Both pairs are enabled. With priority $\mathbf{p_0}$, next-active chooses $\mathbf{p_0}$, which sends l_1 or l_2 to $\mathbf{p_1}$.
② After l_1, the context is still Δ_a but the priority has advanced to $\mathbf{p_1}$. The scheduler skips to $\mathbf{p_2}$, which sends l_3 back to ① or l_4 to ④.
③ After l_2 from ①, the pair $\mathbf{p_0}/\mathbf{p_1}$ has terminated. Only $\mathbf{p_2}$ remains enabled, sending l_3 on the self-loop or l_4 to ⑤.
④ After l_4 from ②, the pair $\mathbf{p_2}/\mathbf{p_3}$ has terminated. Only $\mathbf{p_0}$ remains enabled, sending l_1 on the self-loop or l_2 to ⑤.
⑤ Terminal state: all local types are end.

The round-robin schedule guarantees fairness by construction; the synthesis algorithm in the next section relies on this to ensure every reachable interaction is captured.

6 BUILDING GLOBAL TYPES

This section shows the main result of this paper: we infer a global type from any safe and live typing context, and show that it is a principal global type of the context.

6.1 Proof Outline

The proof proceeds in several steps.

Synthesis definition and well-definedness. We define a synthesis function $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta)$ that constructs a global type from a safe and live context by following its operational semantics using fair scheduling. The environment Σ binds revisited states to recursion variables (Definition 6.1). We show termination of this procedure (Lemma 6.3) via a finiteness argument on the reachable state space (Lemmas B.10–B.13).

Structural properties of the synthesised type. We prove that the synthesised type faithfully reflects the participants in the context: every active participant appears in the result (Lemma 6.5), but not if its local type is end (Lemma B.14), and the synthesised type is balanced (Lemma 6.11).

Backward closure and association A substitution lemma (Lemma 6.4) ensures that the environment bindings are correctly resolved. We define a composite relation $\Vdash_{\mathsf{p}}$ (Definition B.5) that packages coinductive projection and subtyping into a single coinductive relation. We show that the candidate global type we synthesise is related by this composite relation to the local context we started with (Lemma 6.7), using a substitution lemma (Lemma 6.4), preservation of safety and liveness under reduction (Lemma 6.6).

Splitting the composite relation We show the composite relation can be decomposed into separate coinductive projection and subtyping derivations (Lemma 6.9), using an auxiliary result that merge preserves lower bounds (Lemma B.17), and using inversion lemmas for subtyping (Lemmas B.6–B.7). These results establish that the synthesised global type is associated with the context (Theorem 6.10).

Principal global types We show that association implies trace inclusion (Lemma 6.14) and that the synthesised type has the same traces as the context (Lemma 6.18). Combining these with the existence theorem gives the principality result: the synthesised type is the smallest global type associated with the context (Theorem 6.19).

6.2 Inferring a Global Type from a Local Context

We define a synthesis function that infers global types from safe and live contexts. It linearises parallel behaviour via fair scheduling, tracking visited states to synthesise recursive types with μ-binders.

Definition 6.1 (Synthesis environment and function). A *synthesis environment* Σ is a finite partial map from context-participant pairs to type variables: $\Sigma \;::=\; \varnothing \mid \Sigma, (\Delta, \mathbf{p_j}) \mapsto \mathbf{t}$. We write $\Sigma(\Delta, \mathbf{p_j}) = \mathbf{t}$ when $(\Delta, \mathbf{p_j}) \in \mathrm{dom}(\Sigma)$. Then we define the *synthesis function*, $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta)$, by the table below, where $\mathbf{p_j} = \mathsf{next\text{-}active}(\mathbf{p_i})$:

(1) if $\mathsf{stuck}(\Delta)$, then $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta) = \mathsf{end}$;

(2) if $\Sigma(\Delta, \mathbf{p_j}) = \mathbf{t}$, then $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta) = \mathbf{t}$;

(3) otherwise, let $\mathbf{q}$ be the unique participant such that $\Delta \xrightarrow{\mathbf{p_j q}}$, and let $\mathbf{t}$ be a fresh type variable. We case-split on the form of the enabled action:

(a) if $\mathsf{unfold}(\Delta(\mathbf{p_j})) = \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}'$ (value send), then $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta) = \mu \mathbf{t}.\ \mathbf{p_j} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G}'$ where $\Delta \xrightarrow{\mathbf{p_j q}:\langle \mathsf{B} \rangle} \Delta'$ and $\mathsf{G}' = \mathsf{synth}\big(\Sigma \cup \{(\Delta, \mathbf{p_j}) \mapsto \mathbf{t}\},\ \mathbf{p_{(j+1)\bmod n}},\ \Delta'\big)$;

(b) if $\mathrm{unfold}(\Delta(\mathbf{p_j})) = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i\in I}$ (label select), then $\mathrm{synth}(\Sigma, \mathbf{p_i}, \Delta) = \mu \mathbf{t}.\ \mathbf{p_j} \to \mathbf{q}\ \{\, l : \mathsf{G}_l \,\}_{l\in L}$ where $L = \{\, l \mid \Delta \xrightarrow{\mathbf{p_j}\,\mathbf{q}:\langle l\rangle} \Delta_l \,\}$ and $\mathsf{G}_l = \mathrm{synth}\big(\Sigma \cup \{(\Delta, \mathbf{p_j}) \mapsto \mathbf{t}\},\ \mathbf{p_{(j+1)\bmod n}},\ \Delta_l\big)$ for each $l \in L$.

Theorem (Complexity of Synthesis). *For a context $\Delta = \{\mathbf{p_i} : \mathsf{T}_{\mathbf{p_i}}\}_{i\in I}$ with $n = |I|$, computing $\mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$ takes time $O(n^2|\Delta|^n)$.*

Proof. The algorithm (see Algorithm 1 in §8) maintains an environment Σ that ensures each reachable context-participant pair is expanded at most once. The number of reachable contexts is bounded by $\prod_{i\in I} |\mathsf{T}_{\mathbf{p_i}}| \le |\Delta|^n$, so there are at most $n|\Delta|^n$ reachable context-participant pairs. Processing one such pair requires only $O(n)$ overhead to select the next active participant, determine its partner, and collect the enabled observables. Hence the total running time is $O(n \cdot n|\Delta|^n) = O(n^2|\Delta|^n)$. □

Example 6.2 (Synthesising Monte Carlo Global Type). We illustrate synthesis on the running example from §2. Starting from the initial context $\Delta = \{\mathbf{m} : \mathsf{T}_{\mathbf{m}}, \mathbf{w_1} : \mathsf{T}_{\mathbf{w_1}}, \mathbf{w_2} : \mathsf{T}_{\mathbf{w_2}}, \mathbf{r} : \mathsf{T}_{\mathbf{r}}\}$ with participant ordering $\mathbf{p_0}{=}\mathbf{m}$, $\mathbf{p_1}{=}\mathbf{w_1}$, $\mathbf{p_2}{=}\mathbf{w_2}$, $\mathbf{p_3}{=}\mathbf{r}$, we compute $\mathrm{synth}(\varnothing, \mathbf{m}, \Delta)$.

At Δ, $\mathbf{m}$ is the sole enabled sender, so synthesis applies Definition 6.1(3b), binding $(\Delta, \mathbf{m}) \mapsto \mathbf{t}$. It then follows six deterministic interactions:

(1) $\mathbf{m} \to \mathbf{w_1}\ \{\mathit{map}\}$. Only $\mathbf{m}$ is enabled; selects *map* for $\mathbf{w_1}$.
(2) $\mathbf{m} \to \mathbf{w_1}\ \langle \mathtt{float}\rangle$. Sends the payload value to $\mathbf{w_1}$.
(3) $\mathbf{w_1} \to \mathbf{r}\ \langle \mathtt{float}\rangle$. Priority advances; $\mathbf{w_1}$ forwards to $\mathbf{r}$.
(4) $\mathbf{m} \to \mathbf{w_2}\ \{\mathit{map}\}$. Round-robin returns to $\mathbf{m}$; selects *map* for $\mathbf{w_2}$.
(5) $\mathbf{m} \to \mathbf{w_2}\ \langle \mathtt{float}\rangle$. Sends the payload value to $\mathbf{w_2}$.
(6) $\mathbf{w_2} \to \mathbf{r}\ \langle \mathtt{float}\rangle$. Priority advances; $\mathbf{w_2}$ forwards to $\mathbf{r}$.

Let Δ_6 be the context after these six steps:

$$\Delta_6 = \big\{\, \mathbf{m}{:}\,\mathbf{r}\,\&\,\{\mathit{cont}{:}\,\mathsf{T}_{\mathbf{m}},\ \mathit{crash}{:}\,\mathsf{end},\ \mathit{stop}{:}\,\mathbf{w_1} \oplus \{\mathit{stop}{:}\,\mathbf{w_2} \oplus \{\mathit{stop}{:}\,\mathsf{end}\}\}\},\ \mathbf{r}{:}\,\mathbf{m} \oplus \{\mathit{cont}{:}\,\mathsf{T}_{\mathbf{r}},\ \mathit{stop}{:}\,\mathsf{end}\},\ \mathbf{w_1}{:}\,\mathsf{T}_{\mathbf{w_1}},\ \mathbf{w_2}{:}\,\mathsf{T}_{\mathbf{w_2}} \,\big\}$$

In Δ_6, the reducer chooses *cont* or *stop*. The *cont* branch restores Δ with priority $\mathbf{m}$, so rule (2) reuses $\mathbf{t}$. The *stop* branch terminates both workers and reaches a stuck context, so rule (1) returns end. The result is $\mathsf{G}_{\mathrm{inf}} = \mathsf{G}_{\mathrm{map}}$ from §2.

We first show that synthesis is a well-defined construction on safe-and-live inputs. Termination of the synthesis procedure follows directly from several standard finiteness properties (Lemmas B.10–B.13, see Appendix B.2).

Lemma 6.3 (Termination of Synthesis on Safe-and-Live Inputs). *For any context Δ satisfying* $\mathrm{slive}(\Delta)$ *and any priority $\mathbf{p_i}$, the computation $\mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$ terminates.*

Proof (full details in Appendix B.2). Let $S = \{(\Delta', \mathbf{p}) \mid \Delta \longrightarrow^{*} \Delta',\ \mathbf{p} \in \mathrm{dom}(\Delta)\}$, which is finite by Lemma B.12. Each recursive call of synth adds a fresh pair $(\Delta', \mathbf{p_j})$ to Σ, so the measure $m(\Sigma) = |S \setminus \mathrm{dom}(\Sigma)|$ strictly decreases. Since the non-recursive cases (stuck context or revisited state) terminate immediately and each context has only finitely many successors (Lemma B.13), the computation terminates. □

Lemma 6.4 (Substitution for Synth). [Proof in Appendix B.2] *For any environment Σ, contexts Δ and Δ', priorities $\mathbf{p_i}$ and $\mathbf{p_j}$, and type variable $\mathbf{t}$,*

$$\mathit{synth}\big(\Sigma \cup \{\,(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}\,\}, \mathbf{p_i}, \Delta\big)\,[\mathit{synth}(\Sigma, \mathbf{p_j}, \Delta')/\mathbf{t}] = \mathit{synth}(\Sigma, \mathbf{p_i}, \Delta).$$

Termination shows synthesis returns some global type. Now we use the liveness assumption on the context to demonstrate any active participant is in fact eventually incorporated into the synthesised type.

Lemma 6.5 (Non-Terminated Participant Appears in Synthesised Type). *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$*, any priority* $\mathbf{p_i}$*, and any participant* $\mathbf{p} \in \mathit{dom}(\Delta)$*, if* $\mathsf{unfold}(\Delta(\mathbf{p})) \neq \mathsf{end}$*, then* $\mathbf{p} \in \mathsf{pt}(\mathit{synth}(\varnothing, \mathbf{p_i}, \Delta))$.

Proof (full details in Appendix B.2). By contradiction. If $\mathbf{p}$ never appears in the synthesised type, then no interaction along any path in the recursion tree of synth involves $\mathbf{p}$, and so $\Delta(\mathbf{p})$ is unchanged throughout. Every leaf of the recursion tree is either a stuck context (case (1)) or a revisited state (case (2)). In case (1), extending the corresponding reduction to a maximal sequence yields a fair path on which $\mathbf{p}$ is never served, contradicting *liveness*. In case (2), the revisited state creates a cycle that can be repeated forever, yielding an infinite fair path that again never involves $\mathbf{p}$, contradicting liveness. □

Two further lemmas (Lemmas B.14 and B.15, Appendix B.2) give the converse: terminated participants do not appear in the synthesised type, and a stuck safe-and-live context consists entirely of end locals. The second key use of liveness comes in the backward-closure argument (Lemma 6.7), via Lemma 6.6, ensuring successor contexts remain safe and live.

At this point the remaining gap is how to show that projecting our candidate global type recovers the original context. Because full merge may widen offered branches, ordinary projection is slightly too rigid for this step. To establish association, we use a combined relation $\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}$ packaging coinductive-full projection with branch subtyping from merge (Definition B.5). The key difference from ordinary projection is at branching: [SPR5] allows branching on a superset $I \supseteq J$ of the global labels (reflecting [M5]), while [SPR4] requires an exact label match for selections.

$$\text{[SPR4]}\ \frac{\forall i \in I : \mathsf{G_i} \Vdash_{\mathsf{p}} \mathsf{T_i}}{\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G_i}\}_{i\in I} \Vdash_{\mathsf{p}} \mathbf{q} \oplus \{l_i : \mathsf{T_i}\}_{i\in I}} \qquad \text{[SPR5]}\ \frac{\forall j \in J : \mathsf{G_j} \Vdash_{\mathsf{p}} \mathsf{T_j} \qquad J \subseteq I}{\mathbf{q} \to \mathbf{p}\ \{l_j : \mathsf{G_j}\}_{j\in J} \Vdash_{\mathsf{p}} \mathbf{q} \,\&\, \{l_i : \mathsf{T_i}\}_{i\in I}} \qquad \text{[SPR6]}\ \frac{\forall i \in I : \mathsf{G_i} \Vdash_{\mathsf{p}} \mathsf{T} \qquad \mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}}{\mathbf{q} \to \mathbf{r}\ \{l_i : \mathsf{G_i}\}_{i\in I} \Vdash_{\mathsf{p}} \mathsf{T}}$$

We write $\mathsf{G} \Vdash \Delta$ when $\mathsf{dom}(\Delta) = \mathsf{pt}(\mathsf{G})$ and $\mathsf{G} \Vdash_{\mathsf{p}} \Delta(\mathbf{p})$ for all $\mathbf{p} \in \mathsf{pt}(\mathsf{G})$.

We make use of standard inversion lemmas for subtyping (Lemmas B.6 and B.7, see Appendix B.2) and that safety and liveness are preserved under reduction.

Lemma 6.6 (Preservation of Safety and Liveness under Reduction). *If* Δ *is safe and live, and* $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$*, then* Δ' *is safe and live.*

Proof (full details in Appendix B.2). Safety is inherited because every state reachable from Δ' is also reachable from Δ. Liveness is preserved because any fair reduction from Δ' can be extended by prepending the step from Δ, yielding a fair reduction from Δ which is live by assumption. □

The existence proof proceeds by coinduction. We define a candidate relation pairing the synthesised global type with each participant's local type, and show it is backward-closed under $\Vdash_{\mathsf{p}}$, establishing $\mathsf{G} \Vdash_{\mathsf{p}} \Delta(\mathbf{p})$ for every $\mathbf{p}$. For each participant $\mathbf{p}$:

$$R_{\mathbf{p}} = \{(\mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta), \Delta(\mathbf{p})) \mid \mathsf{slive}(\Delta),\ \mathbf{p} \in \mathsf{dom}(\Delta),\ \mathbf{p_i} \in \mathbb{R}\}.$$

We then define its closure $\widehat{R}_{\mathbf{p}}$ inductively by the following rules:

$$\text{[RCand]}\ \frac{(\mathsf{G}, \mathsf{T}) \in R_{\mathbf{p}}}{(\mathsf{G}, \mathsf{T}) \in \widehat{R}_{\mathbf{p}}} \qquad \text{[RUnfL]}\ \frac{(\mu \mathbf{t}.\mathsf{G}, \mathsf{T}) \in R_{\mathbf{p}}}{(\mathsf{G}[\mu \mathbf{t}.\mathsf{G}/\mathbf{t}], \mathsf{T}) \in \widehat{R}_{\mathbf{p}}} \qquad \text{[RUnfR]}\ \frac{(\mathsf{G}, \mu \mathbf{t}.\mathsf{T}) \in \widehat{R}_{\mathbf{p}}}{(\mathsf{G}, \mathsf{T}[\mu \mathbf{t}.\mathsf{T}/\mathbf{t}]) \in \widehat{R}_{\mathbf{p}}}$$

Lemma 6.7 (Backward Closure of Candidate Relation). *For any participant* $\mathbf{p}$*, the relation* $\widehat{R}_{\mathbf{p}}$ *is closed backwards under the rules defining* $\Vdash_{\mathsf{p}}$.

Proof (full details in Appendix B.2). We show that for every $(\mathsf{G}, \mathsf{T}) \in \widehat{R}_{\mathbf{p}}$ there is a rule of $\Vdash_{\mathsf{p}}$ whose conclusion is $\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}$ and whose premises all lie in $\widehat{R}_{\mathbf{p}}$. Each pair traces back to some $\mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta)$ and $\Delta(\mathbf{p})$. We case-split on the synthesis step that produced G. The key case is when synthesis chose an interaction $\Delta \xrightarrow{\mathbf{p_j q}:\langle \mathsf{U} \rangle} \Delta_l$: by Lemma 6.6, each successor Δ_l remains safe and live, so the recursive sub-calls place the premises back in $R_{\mathbf{p}}$ (via the substitution lemma). We then match the role of $\mathbf{p}$ (sender, receiver, or uninvolved) to the corresponding rule of $\Vdash_{\mathsf{p}}$. See Appendix B.2 for the full case analysis. □

6.3 Existence of Well-Formed Global Types and Principal Global Type

We can now state the first main result of the section: synthesis produces a global type that is associated with the source context. We prove two main theorems: (1) any safe and live context yields a well-formed global type (Theorem 6.10); (2) this type is *principal*, having the minimum trace set (Theorem 6.19). We use the association relation [11, 12, 95, 96] between contexts and global types.

Definition 6.8 (Association). We say typing context Δ' is *associated* with global type G, written $\Delta' \sqsubseteq \mathsf{G}$, if there exists Δ such that for all $\mathbf{p} \in \mathsf{pt}(\mathsf{G})$, $\mathsf{G} \upharpoonright_{\mathsf{p}} \Delta(\mathbf{p})$, $\mathsf{dom}(\Delta) = \mathsf{pt}(\mathsf{G})$ and $\Delta' \leqslant \Delta$.

The composite relation is designed so that pointwise projection directly yields association, linking the candidate global type back to the source context.

Lemma 6.9 (Pointwise Composite Relation Implies Association). [Proof in Appendix B.2] *If* $\mathsf{G} \Vdash \Delta$*, then* $\Delta \sqsubseteq \mathsf{G}$.

Theorem 6.10 (Existence of Associated Global Type for Safe, Live Contexts). *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$*, and any participant* $\mathbf{p_i}$*, let* $\mathsf{G} = \mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$*. Then* $\Delta \sqsubseteq \mathsf{G}$.

Proof. By Lemma 6.3, G is well-defined. For each $\mathbf{p} \in \mathsf{dom}(\Delta)$, the pair $(\mathsf{G}, \Delta(\mathbf{p})) \in R_{\mathbf{p}} \subseteq \widehat{R}_{\mathbf{p}}$. By Lemma 6.7, $\widehat{R}_{\mathbf{p}}$ is backward-closed under the rules of $\Vdash_{\mathsf{p}}$, so $\mathsf{G} \Vdash_{\mathsf{p}} \Delta(\mathbf{p})$. Then Lemma 6.9 yields $\Delta \sqsubseteq \mathsf{G}$. □

Lemma 6.11 (Synthesised Global Type is Balanced). [Proof in Appendix B.2] *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$ *and any priority* $\mathbf{p_i}$*, let* $\mathsf{G} = \mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$*. Then* G *is balanced.*

In addition to existence, we demonstrate that synthesis returns the semantically *smallest* global type compatible with the context (it does not create any extra behaviours). This notion is exactly what we call a *principal global type*.

Definition 6.12 (Principal Global Type). A global type G is a *principal global type* for a context Δ if $\Delta \sqsubseteq \mathsf{G}$ and for all global types G' such that $\Delta \sqsubseteq \mathsf{G}'$, we have $\mathsf{G} \subseteq \mathsf{G}'$. That is, G is the smallest global type associated with Δ.

Lemma 6.13 (Completeness of Association [95, Theorem 2]). *If* $\Delta \sqsubseteq \mathsf{G}$ *and* $\Delta \xrightarrow{\ell} \Delta'$*, then there exists* G' *such that* $\mathsf{G} \xrightarrow{\ell} \mathsf{G}'$ *and* $\Delta' \sqsubseteq \mathsf{G}'$.

The principality argument is semantic in that we compare associated global types by their trace sets rather than by their syntax.

Lemma 6.14 (Association Implies Trace Inclusion). *If* $\Delta \sqsubseteq \mathsf{G}$, *then* $Tr(\Delta) \subseteq Tr(\mathsf{G})$.

Proof. Let $\ell_0\,\ell_1\,\cdots \in Tr(\Delta)$, witnessed by a maximal reduction sequence $\Delta = \Delta_0 \xrightarrow{\ell_0} \Delta_1 \xrightarrow{\ell_1} \cdots$. Set $\mathsf{G}_0 = \mathsf{G}$. At each step, $\Delta_k \sqsubseteq \mathsf{G}_k$ and $\Delta_k \xrightarrow{\ell_k} \Delta_{k+1}$. By Lemma 6.13, there exists G_{k+1} with $\mathsf{G}_k \xrightarrow{\ell_k} \mathsf{G}_{k+1}$ and $\Delta_{k+1} \sqsubseteq \mathsf{G}_{k+1}$. This yields $\mathsf{G}_0 \xrightarrow{\ell_0} \mathsf{G}_1 \xrightarrow{\ell_1} \cdots$, so $\ell_0\,\ell_1\,\cdots \in Tr(\mathsf{G})$. □

Lemma 6.15 (Soundness of the Combined Relation). [Proof in Appendix B.2] *If* $\mathsf{G} \Vdash \Delta$ *and* $\mathsf{G} \xrightarrow{\ell} \mathsf{G}'$, *then there exists* Δ' *such that* $\Delta \xrightarrow{\ell} \Delta'$ *and* $\mathsf{G}' \Vdash \Delta'$.

Lemma 6.16 (Soundness Implies Trace Inclusion). *If* $\mathsf{G} \Vdash \Delta$, *then* $Tr(\mathsf{G}) \subseteq Tr(\Delta)$.

Proof. Let $\ell_0\,\ell_1\,\cdots \in Tr(\mathsf{G})$, witnessed by a maximal reduction sequence $\mathsf{G} = \mathsf{G}_0 \xrightarrow{\ell_0} \mathsf{G}_1 \xrightarrow{\ell_1} \cdots$. Set $\Delta_0 = \Delta$. At each step, $\mathsf{G}_k \Vdash \Delta_k$ and $\mathsf{G}_k \xrightarrow{\ell_k} \mathsf{G}_{k+1}$. By Lemma 6.15, there exists Δ_{k+1} with $\Delta_k \xrightarrow{\ell_k} \Delta_{k+1}$ and $\mathsf{G}_{k+1} \Vdash \Delta_{k+1}$. This yields $\Delta_0 \xrightarrow{\ell_0} \Delta_1 \xrightarrow{\ell_1} \cdots$, so $\ell_0\,\ell_1\,\cdots \in Tr(\Delta)$. □

Lemma 6.17 (Combined Relation Implies Trace Equality). *If* $\mathsf{G} \Vdash \Delta$, *then* $Tr(\Delta) = Tr(\mathsf{G})$.

Proof. By Lemma 6.9, $\Delta \sqsubseteq \mathsf{G}$, so $Tr(\Delta) \subseteq Tr(\mathsf{G})$ by Lemma 6.14. The reverse inclusion $Tr(\mathsf{G}) \subseteq Tr(\Delta)$ follows by Lemma 6.16. □

Lemma 6.18 (Trace Equivalence of Context and Synthesised Type). *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$ *and any priority* $\mathbf{p_i}$, *let* $\mathsf{G} = \mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$. *Then* $Tr(\Delta) = Tr(\mathsf{G})$.

Proof. By the proof of Theorem 6.10, we have $\mathsf{G} \Vdash \Delta$. The result follows by Lemma 6.17. □

Theorem 6.19 (Synthesised Global Type is Principal). *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$, *and any priority* $\mathbf{p_i}$, *let* $\mathsf{G} = \mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$. *Then* G *is a principal global type for* Δ.

Proof. By Definition 6.12, it suffices to show (1) $\Delta \sqsubseteq \mathsf{G}$ and (2) $\mathsf{G} \subseteq \mathsf{G}'$ for every G' with $\Delta \sqsubseteq \mathsf{G}'$.
For (1), Theorem 6.10 gives $\Delta \sqsubseteq \mathsf{G}$.
For (2), suppose $\Delta \sqsubseteq \mathsf{G}'$. By Lemma 6.14, $Tr(\Delta) \subseteq Tr(\mathsf{G}')$. By Lemma 6.18, $Tr(\mathsf{G}) = Tr(\Delta)$. Hence $Tr(\mathsf{G}) \subseteq Tr(\mathsf{G}')$, i.e. $\mathsf{G} \subseteq \mathsf{G}'$. □

With existence and principality established, we now show that the top-down approach accepts all bottom-up typable sessions.

7 COMPLETENESS OF THE TOP-DOWN TYPING SYSTEM

This section connects the synthesis result back to session typing and proves completeness of the top-down system with respect to liveness. Process typing $\Gamma \vdash P : \mathsf{T}$ is standard [112, Figure 5], including the subsumption rule [Sub]; Appendix A.2 gives the full rules. We therefore recall only the session-level top-down and bottom-up judgements.

Definition 7.1 (Top-down and bottom-up typing). We recall the session-level typing rules for a multiparty session M. Rule [T-Sess] types M from a projected global type, while rule [B-Sess] types M from a safe local context.

$$\frac{\Gamma \vdash P : \mathsf{T} \quad \mathsf{T} \leqslant \mathsf{T}'}{\Gamma \vdash P : \mathsf{T}'}\ [\text{Sub}] \qquad \frac{\forall i \in I. \quad \vdash P_i : \mathsf{T}_i \quad \mathsf{G} \upharpoonright_{\mathbf{p_i}} \mathsf{T}_i \quad \mathsf{pt}(\mathsf{G}) \subseteq \{\mathbf{p_i} \mid i \in I\}}{\vdash^{\mathrm{top}} \prod_{i \in I} \mathbf{p_i} :: P_i : \{\mathbf{p_i} : \mathsf{T}_i\}_{i \in I}}\ [\text{T-Sess}]$$

$$\frac{\forall i \in I \vdash P_i : \mathsf{T}_i \quad \mathsf{safe}(\{\mathbf{p_i} : \mathsf{T}_i\}_{i \in I})}{\vdash^{\mathrm{bot}} \prod_{i \in I} \mathbf{p_i} :: P_i : \{\mathbf{p_i} : \mathsf{T}_i\}_{i \in I}}\ [\text{B-Sess}]$$

We now define the completeness with respect to a property φ.

Definition 7.2 (Completeness wrt φ). Suppose $\vdash P_i : \mathsf{T_i}$ with $i \in I$. Let $M = \prod_{i\in I} \mathbf{p_i} :: P_i$ and $\Delta = \{\mathbf{p_i} : \mathsf{T_i}\}_{i\in I}$. Typing system $\vdash^\star$ is complete with respect to φ if (1) $\varphi(\Delta)$ implies there exists Δ' such that $\vdash^\star M : \Delta'$ with $\varphi(\Delta')$ and $\Delta \leqslant \Delta'$; and for all M' such that $M \longrightarrow^* M'$, there exists Δ'' such that $\vdash^\star M' : \Delta''$ and $\varphi(\Delta'')$.

We state the theorems which have been proven in the top-down and bottom-up systems.

Theorem 7.3 (Top-down System [95]). *Assume* $\vdash^{\mathrm{top}} M : \Delta$.

(1) (Subject Reduction) ([95, Theorem 5]) If $M \longrightarrow^* M'$, *then* $\vdash^{\mathrm{top}} M' : \Delta'$ *such that* $\Delta \longrightarrow^* \Delta'$.
(2) (Soundness) ([95, Theorem 7]) If $\vdash^{\mathrm{top}} M : \Delta$, *then (a)* $\mathsf{slive}(\Delta)$*; and (b)* M *is safe and live.*

Theorem 7.4 (Bottom-up system [99]). *Assume* $\vdash^{\mathrm{bot}} M : \Delta$ *and* $\mathsf{live}(\Delta)$.

(1) (Subject Reduction with Liveness) ([99, Theorem 15]) If $M \longrightarrow^* M'$, *then* $\vdash^{\mathrm{bot}} M' : \Delta'$ *such that* $\Delta \longrightarrow^* \Delta'$ *and* $\mathsf{live}(\Delta')$.
(2) (Liveness) ([99, Theorem 15]) If $\vdash^{\mathrm{bot}} M : \Delta$ *and* $\mathsf{live}(\Delta)$, *then* M *is safe and live.*

We now arrive at the main result:

Theorem 7.5 (Top-down = Bottom-up).
(1) If $\vdash^{\mathrm{top}} M : \Delta$, *then* $\vdash^{\mathrm{bot}} M : \Delta$ *with* $\mathsf{live}(\Delta)$.
(2) If $\vdash^{\mathrm{bot}} M : \Delta$ *with* $\mathsf{live}(\Delta)$, *then there exists* Δ' *such that* $\vdash^{\mathrm{top}} M : \Delta'$ *and* $\Delta \leqslant \Delta'$.

Proof. 7.5. Assume $\vdash^{\mathrm{top}} M : \Delta$. By Theorem 7.3(2), $\mathsf{slive}(\Delta)$, hence $\mathsf{safe}(\Delta)$ and $\mathsf{live}(\Delta)$. The premises of [T-Sess] already give $\vdash P_i : \mathsf{T_i}$ for each i, so [B-Sess] yields $\vdash^{\mathrm{bot}} M : \Delta$. Thus $\vdash^{\mathrm{bot}} M : \Delta$ with $\mathsf{live}(\Delta)$.
7.5. Assume $\vdash^{\mathrm{bot}} M : \Delta$ with $\mathsf{live}(\Delta)$. From [B-Sess] we obtain $\vdash P_i : \mathsf{T_i}$ for each i and $\mathsf{safe}(\Delta)$, hence $\mathsf{slive}(\Delta)$. By Theorem 6.10, there exists G such that $\Delta \sqsubseteq \mathsf{G}$. Unfolding Definition 6.8, choose Δ' with $\mathrm{dom}(\Delta') = \mathrm{pt}(\mathsf{G})$, $\mathsf{G} \upharpoonright_{\mathbf{p}} \Delta'(\mathbf{p})$ for all $\mathbf{p} \in \mathrm{pt}(\mathsf{G})$, and $\Delta \leqslant \Delta'$. Therefore $\Delta(\mathbf{p_i}) \leqslant \Delta'(\mathbf{p_i})$ for each i, so [Sub] gives $\vdash P_i : \Delta'(\mathbf{p_i})$. Since $\mathrm{dom}(\Delta') = \mathrm{pt}(\mathsf{G}) \subseteq \mathrm{dom}(\Delta) = \{\mathbf{p_i} \mid i \in I\}$, rule [T-Sess] applies, yielding $\vdash^{\mathrm{top}} M : \Delta'$. The same witness Δ' satisfies $\Delta \leqslant \Delta'$. □

By Theorems 7.3, 7.4 and 7.5, we have:

Corollary 7.6.

(1) $\vdash^{\mathrm{top}} M : \Delta$ *is complete with respect to the intersection of safety and liveness.*
(2) $\vdash^{\mathrm{bot}} M : \Delta$ *with* $\mathsf{live}(\Delta)$ *is complete with respect to the intersection of safety and liveness.*

In other words, for the class of live sessions, the protocol-driven top-down approach is just as expressive as the model-checking bottom-up approach.

8 IMPLEMENTATION

We implement a Haskell toolchain (GHC 9.6.6) and test it on the examples from this paper and case studies from the literature.

8.1 Implemented Features

Our toolchain implements both sides of the theory. On the **top-down** strategy (Figure 1a), it includes the four **projection algorithms** of [112] (IP, IF, CP, and CF; Definitions 4.4 and 4.5), **balancedness checking** (Definition 4.7), **synchronous subtyping** (Definition 4.3), and **process type checking** (Appendix A.2). On the **bottom-up** strategy (Figure 1b), it implements **local type inference** from processes [112, Sec. 5.2], **safety and liveness checking** of typing contexts via mpstk [99], and **global type synthesis** from safe and live contexts as in §6.

The toolchain parses the paper's surface syntax into global, local, and context automata on which the core algorithms operate. In particular, balancedness is checked on the graph representation by comparing reachable and unavoidable participants at each reachable vertex. Both sets can be computed iteratively in $O(|G|^2)$ time.

8.2 Global Type Inference

The core feature of our implementation is the synthesis algorithm (Algorithm 1), which constructs a global type from a local context. The algorithm follows the structure of the function synth(Σ, $\mathbf{p_i}$, Δ) (Definition 6.1 in §6).

Algorithm 1 Global Type Synthesis

Require: Safe and live Δ, initial priority $\mathbf{p_i}$.
Ensure: synth($\emptyset$, $\mathbf{p_i}$, Δ).
function Synth(Σ, $\mathbf{p_i}$, Δ)
  if stuck(Δ) **then return** end
  end if
  $\mathbf{p_j} \leftarrow$ next-active($\mathbf{p_i}$)
  if $(\Delta, \mathbf{p_j}) \in \mathrm{dom}(\Sigma)$ **then**
    return $\Sigma(\Delta, \mathbf{p_j})$
  end if
  Choose fresh $\mathbf{t}$
  $\Sigma' \leftarrow \Sigma \cup \{(\Delta, \mathbf{p_j}) \mapsto \mathbf{t}\}$
  Choose $\mathbf{q}$ such that $\Delta \xrightarrow{\mathbf{p_j}\,\mathbf{q}}$
  $L \leftarrow \{U \mid \Delta \xrightarrow{\mathbf{p_j}\,\mathbf{q}:\langle U\rangle}\}$
  for all $U \in L$ **do**
    Choose Δ' with
    $\Delta \xrightarrow{\mathbf{p_j}\,\mathbf{q}:\langle U\rangle} \Delta'$
    $G_U \leftarrow$ Synth(Σ', $\mathbf{p_{(j+1)\bmod n}}$, Δ')
  end for
  return $\mu \mathbf{t}.\mathbf{p_j} \to \mathbf{q}\{U : G_U\}_{U \in L}$
end function

Algorithm outline.

(1) Select the next active sender $\mathbf{p_j}$ by cycling through participants in round-robin order (§5).
(2) If the pair $(\Delta, \mathbf{p_j})$ was already visited, return the stored recursion variable to close the loop.
(3) Otherwise, record a fresh variable $\mathbf{t}$ and enumerate every observable enabled for $\mathbf{p_j}$.
(4) Recurse on each successor context; assemble all branches into a global type node.
(5) When all processes are idle (stuck), return end. Memoisation converts revisited states into μ-binders.

8.3 Evaluations

We evaluate our implementation on examples from the literature and this paper, organised around the two directions studied in the paper. Table 1 starts from global protocols and asks how much the choice of projection algorithm affects the reach of the top-down approach. Table 2 starts from local contexts and asks the converse question: once safety and liveness have been established bottom-up, can we reconstruct an associated global type, and what does that reconstruction cost? All benchmarks were run on an Apple M1 processor (8 cores) with 16 GB of RAM, running macOS 14.3 and GHC 9.6.6. Selected global types used in the evaluation are collected in Table 3 (Appendix D). The benchmark suite includes both hand-written examples from the literature and several parameterised families, including MapReduce, Independent Workers, Coinductive Full, and Binary Counter, so the evaluation covers a broad range of protocols. For every benchmark where synthesis returned a global type G_{inf}, we additionally validated the result by projecting G_{inf} back to a local context and checking pointwise that the original input context is a subtype of that projection. Thus every reported synthesised type was explicitly checked to be associated with the context from which it was inferred. Rows in Table 2 that fail safety or liveness are included as diagnostics about implementation behaviour outside the hypotheses of Sections 6 and 7. The equivalence result concerns the safe-and-live rows.

Top-down projection (Table 1). Table 1 shows that the practical reach of the top-down approach depends strongly on the projection algorithm. Coinductive full-merging projection (CF) [112]

succeeds on every benchmark, including OAuth (Table 3(b)), Two Buyer (Table 3(c)), Odd-Even (Table 3(a)), Ring (Table 3(k)), MapReduce (Table 3(d)), Independent Workers (Table 3(e)), and the Coinductive Full family, whereas IP/IF/CP fail on many rows. Notably, Odd-Even is projectable only by CF; all three weaker algorithms fail on it. CF remains inexpensive, typically in the sub-millisecond range. This suggests that, for these examples, the main limitation is the strength of the EPP algorithm rather than the top-down methodology itself.

This is particularly relevant for the "Less Is More" suite of [99]—OAuth, Recursive Two-Buyer, MapReduce, and Independent Workers—which was designed to expose projection limitations. Table 1 shows that CF already projects the entire suite. The same is true for Ring (Table 3(k)), the main example of [22]. For these benchmarks, the obstruction lies in weaker projection algorithms rather than in the absence of a suitable global specification.

Bottom-up synthesis (Table 2). Table 2 examines the reverse direction. For each safe-and-live row, the tool first checks the local context with mpstk and then runs synthesis; every returned global type passes the reprojection check, giving an implementation-level validation of the association result from Section 6. The timings also separate the cost of verification from the cost of reconstruction: in the successful rows, the safety/liveness check is usually the dominant step, while synthesis itself is much faster. Most successful syntheses also return balanced global types; the main remaining negative case is Two Buyer, which lies outside the safe-and-live fragment. Larger instances such as Independent Workers(10) and the larger Coinductive Full family eventually exhaust available resources in one part of the pipeline or the other.

The size columns also show that principality is semantic rather than purely syntactic. Coinductive Full(4) has $|G|$=63 but $|\Delta|$=1312 and $|G_{\mathsf{inf}}|$=2681: a compact global type can project to a large local context and synthesise back to an even larger principal type. The Binary Counter family (Example D.1) shows the converse: small local contexts ($|\Delta|$=50–107) induce global types that grow exponentially ($|G_{\mathsf{inf}}|$=49–605 for 2–5 bits).

Comparison of top-down and bottom-up. Taken together, the benchmarks demonstrate a clear practical advantage for the top-down approach with coinductive full-merging projection. CF succeeds on every benchmark, removes the projectability gap exhibited by weaker EPP algorithms, and is typically 10–100× faster than the mpstk-based bottom-up pipeline: CF projection completes in microseconds where mpstk safety/liveness checking alone requires hundreds of milliseconds to seconds, and in the largest Coinductive Full instances the bottom-up pipeline eventually exceeds available resources. On the bottom-up side, associated global types can still be reconstructed from verified local contexts, confirming the theoretical equivalence.

9 RELATED WORK AND CONCLUSION

Different theories and frameworks of MPST have been integrated across programming languages and tools [44, 116]. We discuss related work that is most relevant to our main results.

Synthesis and compositions of global protocols In earlier work in the context of communicating finite state machines (CFSMs) [13, 14, 40], synthesis of global protocols refers to the automatic construction of a global behaviour from local specifications represented by CFSMs. Basu and Bultan [13, 14] introduced a "send-synchronisability" problem which asks whether the send projections of its executions remain identical under both asynchronous and rendezvous communication semantics. They have claimed that send-synchronisability is decidable under mailbox and peer-to-peer communications. Finkel and Lozes [40] have shown that their proofs are flawed by establishing their undecidability in peer-to-peer systems. Recently, Delpy et al. [32] have proven undecidability of mailbox communication, closing the open problem.

Table 1. Top-down projection benchmarks: median times over 10 runs. Here $|G|$ is the size of the global type, #p is its number of participants, Balanced reports whether G is balanced (✓) or not (×), with the accompanying time giving the cost of the balancedness check, and $|\Delta_{\text{pro}}|$ is the size of the projected local context. The Citation column gives the source citation and, where available, the corresponding entry in Appendix Table 3. IP = inductive plain, IF = inductive full, CP = coinductive plain, and CF = coinductive full; each projection column shows the runtime when projection succeeds and × when that algorithm fails.

Example	Citation	$\|G\|$	#p	Balanced	$\|\Delta_{\text{pro}}\|$	IP	IF	CP	CF
Simple Travel Agency	[117, Fig. 1(a)]; Table 3(g)	9	2	✓ 13.0 µs	18	42.0 µs	40.0 µs	54.0 µs	48.0 µs
Better Travel Agency	[117, Fig. 1(b)]; Table 3(h)	12	2	✓ 15.0 µs	24	61.0 µs	51.0 µs	67.0 µs	59.0 µs
OAuth	[99, Ex. 1]; Table 3(b)	13	3	✓ 18.0 µs	27	×	80.0 µs	×	69.0 µs
Two Buyer	[99, Ex. 2]; Table 3(c)	20	3	× 32.0 µs	41	×	96.0 µs	×	100.0 µs
Inductive Plain	[112, Ex. 4.8]; Table 3(i)	10	3	✓ 11.0 µs	20	48.0 µs	47.0 µs	42.0 µs	41.0 µs
Inductive Full	[112, Ex. 4.8]; Table 3(j)	10	3	✓ 11.0 µs	22	×	50.0 µs	×	51.0 µs
Semantic Recursion	[107]; Table 3(f)	9	4	✓ 14.0 µs	20	×	×	48.0 µs	46.0 µs
Ring	[22]; Table 3(k)	15	3	✓ 16.0 µs	31	×	74.0 µs	×	57.0 µs
Odd-Even	[77, Ex. 2.1]; Table 3(a)	33	3	✓ 29.0 µs	68	×	×	×	135.0 µs
Monte Carlo Gmap	—	18	4	✓ 37.0 µs	44	×	×	×	103.0 µs
Monte Carlo Gmin	—	15	4	✓ 35.0 µs	32	70.0 µs	65.0 µs	71.0 µs	71.0 µs
Independent Pairs	—	13	4	✓ 17.0 µs	20	64.0 µs	63.0 µs	44.0 µs	44.0 µs
Company Communication	[45, Fig. 9]; Table 3(m)	30	6	✓ 73.0 µs	68	×	181.0 µs	×	157.0 µs
Online Wallet	[89, Fig. 1]; Table 3(n)	26	3	✓ 45.0 µs	53	×	157.0 µs	×	144.0 µs
Adder	[59, Fig. 1(a)]; Table 3(l)	13	2	✓ 15.0 µs	26	59.0 µs	51.0 µs	57.0 µs	57.0 µs
Distributed Logging	[70, Fig. 16]; Table 3(o)	17	2	✓ 18.0 µs	34	86.0 µs	66.0 µs	83.0 µs	88.0 µs
E-Voting	[70]; Table 3(p)	22	2	✓ 24.0 µs	44	93.0 µs	92.0 µs	106.0 µs	101.0 µs
MapReduce(5)	[99, Ex. 3]; Table 3(d)	25	5	✓ 66.0 µs	62	×	×	×	149.0 µs
MapReduce(6)	[99, Ex. 3]; Table 3(d)	31	6	✓ 107.0 µs	78	×	×	×	187.0 µs
MapReduce(7)	[99, Ex. 3]; Table 3(d)	37	7	✓ 150.0 µs	94	×	×	×	230.0 µs
Independent Workers(7)	[99, Ex. 4]; Table 3(e)	59	7	✓ 213.0 µs	91	×	×	×	326.0 µs
Independent Workers(10)	[99, Ex. 4]; Table 3(e)	215	10	✓ 1.5 ms	169	×	×	×	1.4 ms
Coinductive Full(2)	[112, Thm. 4.24]; Table 3(q)	25	3	✓ 20.0 µs	62	×	×	×	122.0 µs
Coinductive Full(3)	[112, Thm. 4.24]; Table 3(r)	42	3	✓ 32.0 µs	217	×	×	×	394.0 µs
Coinductive Full(4)	[112, Thm. 4.24]; Table 3(s)	63	3	✓ 44.0 µs	1312	×	×	×	3.1 ms
Coinductive Full(5)	[112, Thm. 4.24]; Table 3(t)	92	3	✓ 70.0 µs	14239	×	×	×	152.6 ms

In multiparty session types, *building a global type or graph* from a set of participants performs an analogous synthesis: it organises a temporal and spatial relation among participants, ensuring a well-formed global specification is built. The early works in [35, 72] build global types from CFSMs, while [73] builds more expressive *global graphs* with interleaved parallel and mixed choice constructs from CFSMs. Recent works focus on composing two or more global types or choreographic automata to generate a well-formed global type. Barbanera et al. [1, 4, 7] use *gateways* to connect two protocols via forwarding participants at once, while Stolze et al. [101] provide an alternative but restricted approach which has no recursions. Gheri and Yoshida [46] have proposed *hybrid multiparty session types* which are global types enriched with local type actions, enabling a composition of more than two subprotocols. Its successor [6] has introduced *multi-compatibility*, allowing multiple compositions of global types using the gateway approach.

Our aim in constructing the inference algorithm is to prove that there always exists a global type which can fully capture liveness properties of *communicating processes* (Theorem 7.5 and Corollary 7.6). We aim to demonstrate that the typability of the top-down system is the same as the bottom-up system. All the above previous works study the synthesis or compositions of *protocols* (types or CFSMs), but do not directly relate them to *properties of typed processes*. Our inference algorithm can build the *principal global type* (minimal with respect to trace sets) without altering the semantics of local protocols.

Top-Down and Bottom-Up Approaches Honda et al. [56, 57] propose the first end-point projection (EPP) algorithm with *inductive plain merging* from a global type with a parallel composition to local types. Later, to widen the set of well-formed global types, *inductive full merging* is proposed

Table 2. Bottom-up synthesis benchmarks: median times over 10 runs. Here $|\Delta|$ is the size of the input local context, #p is its number of participants, safe, df, and live report whether the context is safe, deadlock-free, and live respectively (✓/×), mpstk is the time taken to check these properties, $|G_{\mathsf{inf}}|$ is the size of the synthesised global type, Balanced reports whether the synthesised global type is balanced (✓/×), and Synth. is the synthesis time. The Citation column gives the source citation and, where available, the corresponding appendix reference, including Table 3 and Example D.1. (1) Java stack overflow error in mpstk. (2) Exceeded time limit.

Example	Citation	$\|\Delta\|$	#p	safe	df	live	mpstk	$\|G_{\mathsf{inf}}\|$	Balanced	Synth.
Two Buyer	[99, Ex. 2]; Table 3(c)	41	3	✓	✓	×	240.0 ms	20	×	178.0 µs
MapReduce(5)	[99, Ex. 3]; Table 3(d)	62	5	✓	✓	✓	280.0 ms	35	✓	18.1 ms
MapReduce(6)	[99, Ex. 3]; Table 3(d)	78	6	✓	✓	✓	370.0 ms	45	✓	179.6 ms
MapReduce(7)	[99, Ex. 3]; Table 3(d)	94	7	✓	✓	✓	590.0 ms	55	✓	1.66 s
OAuth	[99, Ex. 1]; Table 3(b)	27	3	✓	✓	✓	230.0 ms	13	✓	96.0 µs
Independent Workers(7)	[99, Ex. 4]; Table 3(e)	61	7	✓	✓	✓	320.0 ms	73	✓	131.5 ms
Independent Workers(10)	[99, Ex. 4]; Table 3(e)	91	10	✓	✓	✓	1.87 s	(2)	(2)	(2)
Inductive Plain	[112, Ex. 4.8]; Table 3(i)	20	3	✓	✓	✓	230.0 ms	10	✓	24.0 µs
Inductive Full	[112, Ex. 4.8]; Table 3(j)	27	3	✓	✓	✓	240.0 ms	16	✓	45.0 µs
Semantic Recursion	[107]; Table 3(f)	20	4	✓	✓	✓	230.0 ms	7	✓	21.0 µs
Ring	[22]; Table 3(k)	31	3	✓	✓	✓	240.0 ms	15	✓	95.0 µs
Odd-Even	[77, Ex. 2.1]; Table 3(a)	68	3	✓	✓	✓	240.0 ms	43	✓	535.0 µs
Monte Carlo Gmap	—	42	4	✓	✓	✓	250.0 ms	18	✓	434.0 µs
Monte Carlo Gmin	—	39	4	✓	✓	✓	250.0 ms	15	✓	407.0 µs
Independent Pairs	—	24	4	✓	✓	✓	230.0 ms	13	✓	80.0 µs
Online Wallet	[89, Fig. 1]; Table 3(n)	53	3	✓	✓	✓	280.0 ms	26	✓	810.0 µs
Adder	[59, Fig. 1(a)]; Table 3(l)	26	2	✓	✓	✓	270.0 ms	13	✓	44.0 µs
Distributed Logging	[70, Fig. 16]; Table 3(o)	34	2	✓	✓	✓	260.0 ms	17	✓	67.0 µs
E-Voting	[70]; Table 3(p)	44	2	✓	✓	✓	220.0 ms	22	✓	126.0 µs
Δ_5	[112, Fig. 4]	21	3	✓	✓	✓	230.0 ms	9	✓	45.0 µs
Δ_6	[112, Fig. 4]	10	2	×	✓	✓	240.0 ms	3	✓	26.0 µs
Δ_7	[112, Fig. 4]	17	5	×	✓	×	250.0 ms	4	✓	16.0 µs
Δ_8	[112, Fig. 4]	3	1	✓	×	×	260.0 ms	1	✓	4.0 µs
Δ_9	[112, Fig. 4]	11	3	✓	✓	×	270.0 ms	4	✓	23.0 µs
Coinductive Full(2)	[112, Thm. 4.24]; Table 3(q)	62	3	✓	✓	✓	300.0 ms	51	✓	292.0 µs
Coinductive Full(3)	[112, Thm. 4.24]; Table 3(r)	217	3	✓	✓	✓	570.0 ms	314	✓	7.1 ms
Coinductive Full(4)	[112, Thm. 4.24]; Table 3(s)	1312	3	(1)	(1)	(1)	(1)	2681	✓	142.3 ms
Coinductive Full(5)	[112, Thm. 4.24]; Table 3(t)	14239	3	(1)	(1)	(1)	(1)	(2)	(2)	(2)
Coinductive Full Optimised(2)	[112, Thm. 4.24]	40	3	✓	✓	✓	240.0 ms	25	✓	97.0 µs
Coinductive Full Optimised(3)	[112, Thm. 4.24]	61	3	✓	✓	✓	250.0 ms	42	✓	216.0 µs
Coinductive Full Optimised(4)	[112, Thm. 4.24]	86	3	✓	✓	✓	290.0 ms	63	✓	497.0 µs
Coinductive Full Optimised(5)	[112, Thm. 4.24]	119	3	✓	✓	✓	270.0 ms	92	✓	1.1 ms
Binary Counter(2)	Example D.1	50	4	✓	✓	✓	280.0 ms	49	✓	122.0 µs
Binary Counter(3)	Example D.1	69	5	✓	✓	✓	370.0 ms	117	✓	484.0 µs
Binary Counter(4)	Example D.1	88	6	✓	✓	✓	1.01 s	269	✓	4.8 ms
Binary Counter(5)	Example D.1	107	7	✓	✓	✓	4.60 s	605	✓	38.7 ms

and implemented in many tools and languages [116]. Scalas and Yoshida [99] have discovered that a proof of type safety with inductive full merging in the literature was flawed. Recent work [95] has corrected their proofs, and proved that the EPP with inductive full merging is in fact type sound. The EPP with *coinductive plain merging* was proposed in [110] and proven sound using Rocq. The sound and complete EPP algorithm with *coinductive full merging* was proposed in [112], which we applied to design our synthesis algorithm, implemented, and proved complete with respect to liveness of communicating processes.

A bottom-up approach which directly verifies a typing context to prove safety of processes was first introduced in [99]. We have implemented the first complete bottom-up tool which infers a principal global type from a set of MPST processes, implementing the global type inference in this paper and the local type inference in [112], combined with the model-checking tool `mpstk` [99].

Dagnino et al. [31] propose a typing system of an asynchronous network (which consists of a process similar to a local type and a queue) typed by a global type, and develop a non-deterministic type inference system. Their well-formedness checking of global types is undecidable, which is resolved by bounding the paths of global type trees.

In the context of CFSMs, the top-down works [77, 78, 82] have studied an *implementability (projectability) problem* for more expressive sender-driven global types in CFSMs from various angles, see [75, 103] for a comprehensive summary of their works. Recent work [51] studies realisability and complementability (a global type G admits a complementary one $\overline{\mathrm{G}}$ that gives all those behaviours that are not described by G) of global types in both peer-to-peer and synchronous CFSMs [51]. Barbanera et al. [8] have proposed a *corecursive* projection on *coinductive* global types and local types, and proved that corecursively projectable global types automatically satisfy balanced conditions.

Castro-Perez et al. [22] propose a synthetic approach that defines a typing system based on an operational correspondence between a process and a global labelled transition system, extending their earlier works [39, 66]. Their use of the term "synthetic" differs from the synthesis discussed before. They state that their framework is the first top-down system capable of projecting and typing the examples in [99, Fig. 4]. In contrast, related projections appeared prior to [22]: the coinductive full merging in [112, Definition 4.23] can project all examples in [22, Fig. 12] ([99, Fig. 4]), and the corecursive projection in [8] can project [99, Fig. 4(a,c,d)]. See "CF" in Table 3 (Appendix D) and Table 1. Notice also that the term "liveness" is used in [22] to denote "progress" (deadlock-freedom in [99, Def. 5.1]), which is weaker than the liveness used in our paper and in other works [10, 12, 70, 112]. For example, M_2 is *not* live in our paper but live in [22]. Finally, we note a potential issue with [22, Theorem 5.9]; further discussion is provided in Appendix C.

Our focus is not a global type construction but the *typability problem of end-point processes*, comparing a specification-guided (top-down) approach against a model-checking (bottom-up) one.

Conclusion This paper has proven that the *correctness-by-protocol-construction* approach (the top-down system) offers exactly the same typability as the model-checking (bottom-up) approach, which is against the common understanding in the session type literature. We have proven this by developing a novel inference algorithm that produces a principal global type from any live typing context.

We have implemented four kinds of projection algorithms, type-checking systems, and global and local type inference algorithms, and tested them on case studies from the literature. The results in [112] have shown that for most of the realistic protocols, the complexity of the top-down approach remains semi-quadratic (inductive full merging EPPs) and polynomial (coinductive EPPs), and the safety and liveness checks of a typing context Δ for the bottom-up approach are PSPACE-complete in the size of typing contexts, hence the complexity is exponential in the number of states in most cases.

Our benchmarks (Tables 1 and 2) validate these results in practice. We have tested the top-down approach with coinductive full-merging projection across a broad range of examples from the literature, including all benchmarks of [99] and [22], and verified our inference algorithm by reprojecting every synthesised global type and checking subtyping against the original context. The results confirm that the top-down approach offers the same expressiveness as the bottom-up approach (CF projection succeeds on every benchmark) while substantially outperforming the mpstk model-checker on these examples.

In the asynchronous case, the top-down system is still practically implementable when combined with synchronous subtyping from [10–12, 58, 99] or sound asynchronous subtyping algorithms [15, 16, 19, 24, 30], although the bottom-up system remains undecidable. Future work includes analysing decidability and typability in both systems for richer forms of MPSTs, including sender-driven choice [51, 75, 103] and mixed choice [94].

DATA-AVAILABILITY STATEMENT

An artifact exists for this paper. It is a Haskell (GHC 9.6.6) prototype implementing the toolchain described in §8, including parsing and checking of global and local types, four projection algorithms, balancedness check, global type inference, local type inference, and type checking for the process calculus used in this paper. We intend to submit this toolchain for artifact evaluation. An implementation is available at: https://github.com/anonymous5738/submission.

REFERENCES

[1] Franco Barbanera, Ugo de'Liguoro, and Rolf Hennicker. 2019. Connecting open systems of communicating finite state machines. *Journal of Logical and Algebraic Methods in Programming* 109 (2019), 100476. https://doi.org/10.1016/j.jlamp.2019.07.004

[2] Franco Barbanera and Mariangiola Dezani-Ciancaglini. 2019. Open Multiparty Sessions. In Proceedings 12th *Interaction and Concurrency Experience,* Copenhagen, Denmark, 20-21 June 2019 *(Electronic Proceedings in Theoretical Computer Science, Vol. 304)*, Massimo Bartoletti, Ludovic Henrio, Anastasia Mavridou, and Alceste Scalas (Eds.). Open Publishing Association, Waterloo, Australia, 77–96. https://doi.org/10.4204/EPTCS.304.6

[3] Franco Barbanera and Mariangiola Dezani-Ciancaglini. 2023. Partially Typed Multiparty Sessions. In Proceedings 16th *Interaction and Concurrency Experience,* Lisbon, Portugal, 19th June 2023 *(Electronic Proceedings in Theoretical Computer Science, Vol. 383)*, Clément Aubert, Cinzia Di Giusto, Simon Fowler, and Larisa Safina (Eds.). Open Publishing Association, Waterloo, Australia, 15–34. https://doi.org/10.4204/EPTCS.383.2

[4] Franco Barbanera, Mariangiola Dezani-Ciancaglini, and Ugo de'Liguoro. 2022. Open Compliance in Multiparty Sessions. In *Formal Aspects of Component Software (Lecture Notes in Computer Science, Vol. 13712)*, Silvia Lizeth Tapia Tarifa and José Proença (Eds.). Springer International Publishing, Cham, 222–243. https://doi.org/10.1007/978-3-031-20872-0_13

[5] Franco Barbanera, Mariangiola Dezani-Ciancaglini, and Ugo de'Liguoro. 2024. Un-projectable Global Types for Multiparty Sessions. In *Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming* (Milano, Italy) *(PPDP '24)*. Association for Computing Machinery, New York, NY, USA, Article 15, 13 pages. https://doi.org/10.1145/3678232.3678245

[6] Franco Barbanera, Mariangiola Dezani-Ciancaglini, Lorenzo Gheri, and Nobuko Yoshida. 2023. Multicompatibility for Multiparty-Session Composition. In *Proceedings of the 25th International Symposium on Principles and Practice of Declarative Programming* (Lisboa, Portugal) *(PPDP '23)*. Association for Computing Machinery, New York, NY, USA, Article 2, 15 pages. https://doi.org/10.1145/3610612.3610614

[7] Franco Barbanera, Mariangiola Dezani-Ciancaglini, Ivan Lanese, and Emilio Tuosto. 2021. Composition and decomposition of multiparty sessions. *Journal of Logical and Algebraic Methods in Programming* 119 (2021), 100620. https://doi.org/10.1016/j.jlamp.2020.100620

[8] Franco Barbanera and Rolf Hennicker. 2024. Safe Composition of Systems of Communicating Finite State Machines. In *Proceedings 17th Interaction and Concurrency Experience, ICE 2024, Groningen, The Netherlands, 21st June 2024 (EPTCS, Vol. 414)*, Clément Aubert, Cinzia Di Giusto, Simon Fowler, and Violet Ka I Pun (Eds.). 39–57. https://doi.org/10.4204/EPTCS.414.3

[9] Franco Barbanera, Ivan Lanese, and Emilio Tuosto. 2023. Composition of synchronous communicating systems. *J. Log. Algebraic Methods Program.* 135 (2023), 100890. https://doi.org/10.1016/J.JLAMP.2023.100890

[10] Adam Barwell, Alceste Scalas, Nobuko Yoshida, and Fangyi Zhou. 2022. Generalised Multiparty Session Types with Crash-Stop Failures. In *33rd International Conference on Concurrency Theory (LIPIcs, Vol. 243)*. Dagstuhl, 35:1–35:25.

[11] Adam D. Barwell, Ping Hou, Nobuko Yoshida, and Fangyi Zhou. 2023. Designing Asynchronous Multiparty Protocols with Crash-Stop Failures. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 1:1–1:30. https://doi.org/10.4230/LIPIcs.ECOOP.2023.1

[12] Adam D. Barwell, Ping Hou, Nobuko Yoshida, and Fangyi Zhou. 2025. Crash-Stop Failures in Asynchronous Multiparty Session Types. *Logical Methods in Computer Science* 21(2) (2025). Issue 2. https://doi.org/10.46298/lmcs-21(2:5)2025

[13] Samik Basu and Tevfik Bultan. 2011. Choreography Conformance via Synchronizability. In *Proceedings of the 20th International Conference on World Wide Web (WWW 2011)*. ACM, 795–804. https://ra.ethz.ch/cdstore/www2011/proceedings/p795.pdf

[14] Samik Basu and Tevfik Bultan. 2016. On deciding synchronizability for asynchronously communicating systems. *Theoretical Computer Science* 656 (2016), 60–75. https://doi.org/10.1016/j.tcs.2016.09.023

[15] Laura Bocchi, Andy King, and Maurizio Murgia. 2024. Asynchronous Subtyping by Trace Relaxation. In *TACAS'24: 30th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Luxembourg City,*

Luxembourg, 6 - 11 April 2024. Springer.

[16] Laura Bocchi, Andy King, Maurizio Murgia, and Simon Thompson. 2025. Abstract Subtyping for Asynchronous Multiparty Sessions. In *36th International Conference on Concurrency Theory, CONCUR 2025, August 26-29, 2025, Aarhus, Denmark (LIPIcs, Vol. 348)*, Patricia Bouyer and Jaco van de Pol (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 10:1–10:19. https://doi.org/10.4230/LIPICS.CONCUR.2025.10

[17] Jelle Bouma, Stijn de Gouw, and Sung-Shik Jongmans. 2023. Multiparty Session Typing in Java, Deductively. In *Tools and Algorithms for the Construction and Analysis of Systems - 29th International Conference, TACAS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Paris, France, April 22-27, 2023, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13994)*, Sriram Sankaranarayanan and Natasha Sharygina (Eds.). Springer, 19–27. https://doi.org/10.1007/978-3-031-30820-8_3

[18] Daniel Brand and Pitro Zafiropulo. 1983. On Communicating Finite-State Machines. *J. ACM* 30, 2 (April 1983), 323–342. https://doi.org/10.1145/322374.322380

[19] Mario Bravetti, Marco Carbone, Julien Lange, Nobuko Yoshida, and Gianluigi Zavattaro. 2021. A Sound Algorithm for Asynchronous Session Subtyping and its Implementation. *Logical Methods in Computer Science* Volume 17, Issue 1, Article 20 (Mar 2021). https://doi.org/10.23638/LMCS-17(1:20)2021

[20] Mario Bravetti, Marco Carbone, and Gianluigi Zavattaro. 2017. Undecidability of asynchronous session subtyping. *Inf. Comput.* 256 (2017), 300–320.

[21] David Castro-Perez, Francisco Ferreira, Lorenzo Gheri, and Nobuko Yoshida. 2021. Zooid: a DSL for certified multiparty computation: from mechanised metatheory to certified multiparty processes. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 237–251. https://doi.org/10.1145/3453483.3454041

[22] David Castro-Perez, Francisco Ferreira, and Sung-Shik Jongmans. 2026. A Synthetic Reconstruction of Multiparty Session Types. *Proc. ACM Program. Lang.* 10, POPL, Article 50 (Jan. 2026), 29 pages. https://doi.org/10.1145/3776692

[23] David Castro-Perez, Raymond Hu, Sung-Shik Jongmans, Nicholas Ng, and Nobuko Yoshida. 2019. Distributed programming using role-parametric session types in go: statically-typed endpoint APIs for dynamically-instantiated communication structures. *Proc. ACM Program. Lang.* 3, POPL (2019), 29:1–29:30. https://doi.org/10.1145/3290342

[24] David Castro-Perez and Nobuko Yoshida. 2020. CAMP: cost-aware multiparty session protocols. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 155:1–155:30. https://doi.org/10.1145/3428223

[25] Minas Charalambides, Peter Dinges, and Gul Agha. 2012. Parameterized Concurrent Multi-Party Session Types. In *Proceedings 11th International Workshop on Foundations of Coordination Languages and Self Adaptation, FOCLASA 2012, Newcastle, U.K., September 8, 2012 (EPTCS, Vol. 91)*, Natallia Kokash and António Ravara (Eds.). 16–30. https://doi.org/10.4204/EPTCS.91.2

[26] Minas Charalambides, Peter Dinges, and Gul A. Agha. 2016. Parameterized, concurrent session types for asynchronous multi-actor interactions. *Sci. Comput. Program.* 115-116 (2016), 100–126. https://doi.org/10.1016/J.SCICO.2015.10.006

[27] Tzu-Chun Chen, Mariangiola Dezani-Ciancaglini, Alceste Scalas, and Nobuko Yoshida. 2017. On the Preciseness of Subtyping in Session Types. *Logical Methods in Computer Science* 13, 2 (2017), 1–46. https://doi.org/10.23638/LMCS-13(2:12)2017

[28] Guillermina Cledou, Luc Edixhoven, Sung-Shik Jongmans, and José Proença. 2022. API Generation for Multiparty Session Types, Revisited and Revised Using Scala 3. In *36th European Conference on Object-Oriented Programming (ECOOP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:28. https://drops.dagstuhl.de/opus/volltexte/2022/16255

[29] Mario Coppo, Mariangiola Dezani-Ciancaglini, Luca Padovani, and Nobuko Yoshida. 2015. A Gentle Introduction to Multiparty Asynchronous Session Types. In *15th International School on Formal Methods for the Design of Computer, Communication and Software Systems: Multicore Programming (LNCS, Vol. 9104)*. Springer, 146–178.

[30] Zak Cutner, Nobuko Yoshida, and Martin Vassor. 2022. Deadlock-Free Asynchronous Message Reordering in Rust with Multiparty Session Types. In *27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22, Vol. abs/2112.12693)*. ACM, 261–246. https://doi.org/10.1145/3503221.3508404

[31] Francesco Dagnino, Paola Giannini, and Mariangiola Dezani-Ciancaglini. 2023. Deconfined Global Types for Asynchronous Sessions. *Logical Methods in Computer Science* Volume 19, Issue 1, Article 3 (Jan 2023). https://doi.org/10.46298/lmcs-19(1:3)2023

[32] Romain Delpy, Anca Muscholl, and Grégoire Sutre. 2025. On the Send-Synchronizability Problem for Mailbox Communication. In *36th International Conference on Concurrency Theory (CONCUR 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 348)*. Schloss Dagstuhl–Leibniz-Zentrum f"ur Informatik, 15:1–15:20. https://doi.org/10.4230/LIPIcs.CONCUR.2025.15

[33] Romain Demangeon and Kohei Honda. 2011. Full Abstraction in a Subtyped pi-Calculus with Linear Types. In *CONCUR 2011 - Concurrency Theory - 22nd International Conference, CONCUR 2011, Aachen, Germany, September*

6-9, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6901), Joost-Pieter Katoen and Barbara König (Eds.). Springer, 280–296. https://doi.org/10.1007/978-3-642-23217-6_19

[34] Romain Demangeon, Kohei Honda, Raymond Hu, Rumyana Neykova, and Nobuko Yoshida. 2015. Practical Interruptible Conversations: Distributed Dynamic Verification with Multiparty Session Types and Python. *Formal Methods Syst. Des.* 46, 3 (2015), 197–225. https://doi.org/10.1007/s10703-014-0218-8

[35] Pierre-Malo Deniélou and Nobuko Yoshida. 2013. Multiparty Compatibility in Communicating Automata: Characterisation and Synthesis of Global Session Types. In *Automata, Languages, and Programming*, Fedor V. Fomin, Rūsiņš Freivalds, Marta Kwiatkowska, and David Peleg (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 174–186.

[36] Lavinia Egidi, Paola Giannini, and Lorenzo Ventura. 2022. Multiparty-session-types Coordination for Core Erlang. In *Proceedings of the 17th International Conference on Software Technologies, ICSOFT 2022, Lisbon, Portugal, July 11-13, 2022*, Hans-Georg Fill, Marten van Sinderen, and Leszek A. Maciaszek (Eds.). SCITEPRESS, 532–541. https://doi.org/10.5220/0011316600003266

[37] Burak Ekici, Tadayoshi Kamegai, and Nobuko Yoshida. 2025. Formalising Subject Reduction and Progress for Multiparty Session Processes. In *16th International Conference on Interactive Theorem Proving, ITP 2025, September 28 to October 1, 2025, Reykjavik, Iceland (LIPIcs, Vol. 352)*, Yannick Forster and Chantal Keller (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 19:1–19:23. https://doi.org/10.4230/LIPICS.ITP.2025.19

[38] Burak Ekici and Nobuko Yoshida. 2024. Completeness of Asynchronous Session Tree Subtyping in Coq. In *15th International Conference on Interactive Theorem Proving, ITP 2024, September 9-14, 2024, Tbilisi, Georgia (LIPIcs, Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 13:1–13:20. https://doi.org/10.4230/LIPICS.ITP.2024.13

[39] Francisco Ferreira and Sung-Shik Jongmans. 2023. Oven: Safe and Live Communication Protocols in Scala, using Synthetic Behavioural Type Analysis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 1511–1514. https://doi.org/10.1145/3597926.3604926

[40] Alain Finkel and Etienne Lozes. 2017. Synchronizability of Communicating Finite State Machines is not Decidable. In *44th International Colloquium on Automata, Languages and Programming (ICALP 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 80)*. Schloss Dagstuhl–Leibniz-Zentrum f"ur Informatik, 122:1–122:14. https://doi.org/10.4230/LIPIcs.ICALP.2017.122

[41] Simon Fowler. 2016. An Erlang Implementation of Multiparty Session Actors. In *Proceedings 9th Interaction and Concurrency Experience, ICE 2016, Heraklion, Greece, 8-9 June 2016 (EPTCS, Vol. 223)*, Massimo Bartoletti, Ludovic Henrio, Sophia Knight, and Hugo Torres Vieira (Eds.). 36–50. https://doi.org/10.4204/EPTCS.223.3

[42] Simon Fowler and Raymond Hu. 2026. Spark Now: Safe Actor Programming with Multiparty Session Types. https://simonjf.com/writing/eventactors.pdf To appear in OOPSLA'26.

[43] Simon Fowler, Sam Lindley, J. Garrett Morris, and Sára Decova. 2019. Exceptional asynchronous session types: session types without tiers. *Proc. ACM Program. Lang.* 3, POPL (2019), 28:1–28:29. https://doi.org/10.1145/3290341

[44] Simon Gay and António Ravara (Eds.). 2017. *Behavioural Types: from Theory to Tools.* River Publisher.

[45] Lorenzo Gheri, Ivan Lanese, Neil Sayers, Emilio Tuosto, and Nobuko Yoshida. 2022. Design-By-Contract for Flexible Multiparty Session Protocols. In *36th European Conference on Object-Oriented Programming, ECOOP 2022, June 6-10, 2022, Berlin, Germany (LIPIcs, Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 8:1–8:28. https://doi.org/10.4230/LIPICS.ECOOP.2022.8

[46] Lorenzo Gheri and Nobuko Yoshida. 2023. Hybrid Multiparty Session Types: Compositionality for Protocol Specification through Endpoint Projection. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 79 (April 2023), 31 pages. https://doi.org/10.1145/3586031

[47] Silvia Ghilezan, Svetlana Jakšić, Jovanka Pantović, Alceste Scalas, and Nobuko Yoshida. 2019. Precise subtyping for synchronous multiparty sessions. *Journal of Logical and Algebraic Methods in Programming* 104 (2019), 127–173. https://doi.org/10.1016/j.jlamp.2018.12.002

[48] Silvia Ghilezan, Jovanka Pantović, Ivan Prokić, Alceste Scalas, and Nobuko Yoshida. 2023. Precise Subtyping for Asynchronous Multiparty Sessions. *ACM Trans. Comput. Logic* 24, 2, Article 14 (Nov. 2023), 73 pages. https://doi.org/10.1145/3568422

[49] Marco Giunti and Nobuko Yoshida. 2025. Iso-Recursive Multiparty Sessions and their Automated Verification. In *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 15694)*, Viktor Vafeiadis (Ed.). Springer, 349–378. https://doi.org/10.1007/978-3-031-91118-7_14

[50] Cinzia Di Giusto, Laetitia Laversa, and Kirstin Peters. 2024. Synchronisability in Mailbox Communication. In *Proceedings of the Combined 31st International Workshop on Expressiveness in Concurrency and 21st Workshop on Structural Operational Semantics (EXPRESS/SOS 2024) (Electronic Proceedings in Theoretical Computer Science (EPTCS),*

Vol. 412). 19–34. https://cgi.cse.unsw.edu.au/~eptcs/paper.cgi?EXPRESSSOS2024.3 Also available as arXiv:2411.14580.

[51] Cinzia Di Giusto, Pascal Urso, and Étienne Lozes. 2025. Realisability and Complementability of Multiparty Session Types. In *Proceedings of the 27th International Symposium on Principles and Practice of Declarative Programming (PPDP 2025)*. ACM, Rende, Italy, 121–133. https://doi.org/10.1145/3676459.3676473

[52] Paul Harvey, Simon Fowler, Ornela Dardha, and Simon J. Gay. 2021. Multiparty Session Types for Safe Runtime Adaptation in an Actor Language. In *35th European Conference on Object-Oriented Programming, ECOOP 2021, July 11-17, 2021, Aarhus, Denmark (Virtual Conference) (LIPIcs, Vol. 194)*, Anders Møller and Manu Sridharan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 10:1–10:30. https://doi.org/10.4230/LIPICS.ECOOP.2021.10

[53] Jonas Kastberg Hinrichsen, Jules Jacobs, and Robbert Krebbers. 2024. Multris: Functional Verification of Multiparty Message Passing in Separation Logic. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 1446–1474. https://doi.org/10.1145/3689762

[54] C. A. R. Hoare. 1985. *Communicating Sequential Processes*. Prentice-Hall, Inc.

[55] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2008. Multiparty Asynchronous Session Types. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '08)*. Association for Computing Machinery, New York, NY, USA, 273–284. https://doi.org/10.1145/1328438.1328472

[56] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *Journal of the ACM* 63, 1 (2016), 9:1–9:67. https://doi.org/10.1145/2827695

[57] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *Journal of the ACM* 63, 1 (2016), 9:1–9:67. https://doi.org/10.1145/2827695

[58] Ping Hou, Nicolas Lagaillardie, and Nobuko Yoshida. 2024. Fearless Asynchronous Communications with Timed Multiparty Session Protocols. In *38th European Conference on Object-Oriented Programming, ECOOP 2024, September 16-20, 2024, Vienna, Austria (LIPIcs, Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 19:1–19:30. https://doi.org/10.4230/LIPICS.ECOOP.2024.19

[59] Raymond Hu and Nobuko Yoshida. 2016. Hybrid Session Verification through Endpoint API Generation. In *19th International Conference on Fundamental Approaches to Software Engineering (LNCS, Vol. 9633)*. Springer, Berlin, Heidelberg, 401–418. https://doi.org/10.1007/978-3-662-49665-7_24

[60] Raymond Hu and Nobuko Yoshida. 2017. Explicit Connection Actions in Multiparty Session Types. In *FASE*. https://doi.org/10.1007/978-3-662-54494-5_7

[61] Keigo Imai, Julien Lange, and Rumyana Neykova. 2022. Kmclib: Automated Inference and Verification of Session Types from OCaml Programs. In *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13243)*, Dana Fisman and Grigore Rosu (Eds.). Springer, 379–386. https://doi.org/10.1007/978-3-030-99524-9_20

[62] Keigo Imai, Rumyana Neykova, Nobuko Yoshida, and Shoji Yuen. 2020. Multiparty Session Programming with Global Protocol Combinators. In *34th European Conference on Object-Oriented Programming (LIPIcs, Vol. 166)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 9:1–9:30. https://doi.org/10.4230/LIPIcs.ECOOP.2020.9

[63] Keigo Imai, Shoji Yuen, and Kiyoshi Agusa. 2010. Session Type Inference in Haskell. In *Proceedings Third Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software, PLACES 2010, Paphos, Cyprus, 21st March 2010 (EPTCS, Vol. 69)*, Kohei Honda and Alan Mycroft (Eds.). 74–91. https://doi.org/10.4204/EPTCS.69.6

[64] Jules Jacobs, Stephanie Balzer, and Robbert Krebbers. 2022. Multiparty GV: functional multiparty session types with certified deadlock freedom. *Proc. ACM Program. Lang.* 6, ICFP (2022), 466–495. https://doi.org/10.1145/3547638

[65] Sung-Shik Jongmans. 2025. Multiparty Session Typing, Embedded. In *Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 15696)*, Arie Gurfinkel and Marijn Heule (Eds.). Springer, 145–164. https://doi.org/10.1007/978-3-031-90643-5_8

[66] Sung-Shik Jongmans and Nobuko Yoshida. 2020. Exploring Type-Level Bisimilarity towards More Expressive Multiparty Session Types. In *29th European Symposium on Programming (LNCS, Vol. 12075)*. Springer, 251–279. https://doi.org/10.1007/978-3-030-44914-810

[67] Shunsuke Kimura and Keigo Imai. 2020. Fluent Session Programming in C#. In *Proceedings of the 12th International Workshop on Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES'2020, Dublin, Ireland, 26th April 2020 (EPTCS, Vol. 314)*, Stephanie Balzer and Luca Padovani (Eds.). 61–75. https://doi.org/10.4204/EPTCS.314.6

[68] Jonathan King, Nicholas Ng, and Nobuko Yoshida. 2019. Multiparty Session Type-safe Web Development with Static Linearity. In *Proceedings Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES@ETAPS 2019, Prague, Czech Republic, 7th April 2019 (EPTCS, Vol. 291)*, Francisco Martins and Dominic Orchard (Eds.). 35–46. https://doi.org/10.4204/EPTCS.291.4

[69] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. 2018. Typechecking protocols with Mungo and StMungo: A session type toolchain for Java. *Sci. Comput. Program.* 155 (2018), 52–75. https://doi.org/10.1016/J.SCICO.2017.10.006

[70] Nicolas Lagaillardie, Rumyana Neykova, and Nobuko Yoshida. 2022. Stay Safe Under Panic: Affine Rust Programming with Multiparty Session Types. In *36th European Conference on Object-Oriented Programming, ECOOP 2022, June 6-10, 2022, Berlin, Germany (LIPIcs, Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 4:1–4:29. https://doi.org/10.4230/LIPICS.ECOOP.2022.4

[71] Julien Lange, Nicholas Ng, Bernardo Toninho, and Nobuko Yoshida. 2018. A Static Verification Framework for Message Passing in Go using Behavioural Types. In *40th International Conference on Software Engineering*. ACM, 1137–1148.

[72] Julien Lange and Emilio Tuosto. 2012. Synthesising Choreographies from Local Session Types. In *CONCUR 2012 - Concurrency Theory - 23rd International Conference, CONCUR 2012, Newcastle upon Tyne, UK, September 4-7, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7454)*, Maciej Koutny and Irek Ulidowski (Eds.). Springer, Berlin, Heidelberg, 225–239. https://doi.org/10.1007/978-3-642-32940-1_17

[73] Julien Lange, Emilio Tuosto, and Nobuko Yoshida. 2015. From Communicating Machines to Graphical Choreographies. *POPL* 50, 1 (2015), 221–232. https://doi.org/10.1145/2775051.2676964

[74] Julien Lange and Nobuko Yoshida. 2017. On the Undecidability of Asynchronous Session Subtyping. In *FoSSaCS (Lecture Notes in Computer Science, Vol. 10203)*. 441–457.

[75] Elaine Li. 2025. *Decision Problems for Global Protocol Specifications.* Ph.D. Thesis. City University of New York, New York, NY, USA. https://efl9013.github.io/ Ph.D. Thesis, City University of New York, 2025.

[76] Elaine Li, Felix Stutz, and Thomas Wies. 2024. Deciding Subtyping for Asynchronous Multiparty Sessions. In *Programming Languages and Systems*, Stephanie Weirich (Ed.). Springer Nature Switzerland, Cham, 176–205. https://doi.org/10.1007/978-3-031-57262-3_8

[77] Elaine Li, Felix Stutz, Thomas Wies, and Damien Zufferey. 2024. Complete Multiparty Session Type Projection with Automata. arXiv:2305.17079 [cs]

[78] Elaine Li, Felix Stutz, Thomas Wies, and Damien Zufferey. 2025. Characterizing Implementability of Global Protocols with Infinite States and Data. *Proc. ACM Program. Lang.* 9, OOPSLA1 (2025), 1434–1463. https://doi.org/10.1145/3720493

[79] Elaine Li and Thomas Wies. 2025. Certified Implementability of Global Multiparty Protocols. In *16th International Conference on Interactive Theorem Proving, ITP 2025, September 28 to October 1, 2025, Reykjavik, Iceland (LIPIcs, Vol. 352)*, Yannick Forster and Chantal Keller (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 15:1–15:20. https://doi.org/10.4230/LIPICS.ITP.2025.15

[80] Sam Lindley and J. Garrett Morris. 2016. Embedding Session Types in Haskell. In *Proceedings of the 9th International Symposium on Haskell, Haskell 2016, Nara, Japan, September 22-23, 2016*, Geoffrey Mainland (Ed.). ACM, 133–145. https://doi.org/10.1145/2976002.2976018

[81] Hugo A. López, Eduardo R. B. Marques, Francisco Martins, Nicholas Ng, César Santos, Vasco Thudichum Vasconcelos, and Nobuko Yoshida. 2015. Protocol-Based Verification of Message-Passing Parallel Programs. In *2015 ACM International Conference on Object Oriented Programming Systems Languages and Applications / SPLASH '15*. ACM, 280–298.

[82] Rupak Majumdar, Madhavan Mukund, Felix Stutz, and Damien Zufferey. 2021. Generalising Projection in Asynchronous Multiparty Session Types. In *32nd International Conference on Concurrency Theory, CONCUR 2021, August 24-27, 2021, Virtual Conference (LIPIcs, Vol. 203)*, Serge Haddad and Daniele Varacca (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 35:1–35:24. https://doi.org/10.4230/LIPIcs.CONCUR.2021.35

[83] Robin Milner. 1989. *Communication and Concurrency.* Prentics Hall.

[84] Robin Milner. 1999. *Communicating and Mobile Systems: the π-Calculus.* Cambridge University Press.

[85] Anson Miu, Francisco Ferreira, Nobuko Yoshida, and Fangyi Zhou. 2021. Communication-Safe Web Programming in TypeScript with Routed Multiparty Session Types. In *International Conference on Compiler Construction (CC)*. 94–106. https://doi.org/10.1145/3446804.3446854

[86] Rumyana Neykova, Raymond Hu, Nobuko Yoshida, and Fahd Abdeljallal. 2018. A Session Type Provider: Compile-time API Generation for Distributed Protocols with Interaction Refinements in F#. In *27th International Conference on Compiler Construction*. ACM, 128–138. https://doi.org/10.1145/3178372.3179495

[87] Rumyana Neykova and Nobuko Yoshida. 2017. Let It Recover: Multiparty Protocol-Induced Recovery. In *CC*. https://doi.org/10.1145/3033019.3033031

[88] Rumyana Neykova and Nobuko Yoshida. 2017. Multiparty Session Actors. *Logical Methods in Computer Science* 13 (2017), 1–30. Issue 1. https://doi.org/10.23638/LMCS-13(1:17)2017

[89] Rumyana Neykova, Nobuko Yoshida, and Raymond Hu. 2013. SPY: Local Verification of Global Protocols. In *Runtime Verification - 4th International Conference, RV 2013, Rennes, France, September 24-27, 2013. Proceedings (Lecture Notes in*

Computer Science, Vol. 8174), Axel Legay and Saddek Bensalem (Eds.). Springer, 358–363. https://doi.org/10.1007/978-3-642-40787-1_25

[90] Nicholas Ng, Jose G.F. Coutinho, and Nobuko Yoshida. 2015. Protocols by Default: Safe MPI Code Generation based on Session Types. In *24th International Conference on Compiler Construction (LNCS, Vol. 9031)*. Springer, 212–232. https://doi.org/10.1007/978-3-662-46663-6_11

[91] Nicholas Ng and Nobuko Yoshida. 2016. Static Deadlock Detection for Concurrent Go by Global Session Graph Synthesis. In *25th International Conference on Compiler Construction*. ACM, 174–184. https://doi.org/10.1145/2892208.2892232

[92] Nicholas Ng, Nobuko Yoshida, and Kohei Honda. 2012. Multiparty Session C: Safe Parallel Programming with Message Optimisation. In *50th International Conference on Objects, Models, Components, Patterns (LNCS, Vol. 7304)*. Springer, 202–218. https://doi.org/10.1007/978-3-642-30561-0_15

[93] Nicholas Ng, Nobuko Yoshida, Xinyu Niu, Kuen Hung Tsoi, and Wayne Luk. 2012. Session Types: Towards safe and fast reconfigurable programming. *ACM SIGARCH Computer Architecture News* 40 (2012), 22–27. Issue 5. https://doi.org/10.1145/2460216.2460221

[94] Kirstin Peters and Nobuko Yoshida. 2024. Separation and Encodability in Mixed Choice Multiparty Sessions. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2024, Tallinn, Estonia, July 8-11, 2024*, Pawel Sobocinski, Ugo Dal Lago, and Javier Esparza (Eds.). ACM, 62:1–62:15. https://doi.org/10.1145/3661814.3662085

[95] Nobuko Yoshida Ping Hou and Iona Kuhn. 2024. Less is More Revisited. arXiv:2402.16741 [cs.PL] https://mrg.cs.ox.ac.uk/publications/less_is_more_revisited_association_with_global_protocols_and_multiparty_sessions/main.pdf Accepted by TCS. A short version appeared by Cliff B. Jones Festschrift Proceeding.

[96] Kai Pischke and Nobuko Yoshida. 2025. Asynchronous Global Protocols, Precisely. In *Components Operationally: Reversibility and System Engineering (LNCS, Vol. 16065)*. Springer Nature, 116–133. https://doi.org/10.1007/978-3-031-99717-4_7

[97] Kai Pischke and Nobuko Yoshida. 2025. Asynchronous Global Protocols, Precisely: Full Proofs. arXiv:2505.17676 [cs.PL] https://arxiv.org/abs/2505.17676

[98] Alceste Scalas, Ornela Dardha, Raymond Hu, and Nobuko Yoshida. 2017. A Linear Decomposition of Multiparty Sessions for Safe Distributed Programming. In *ECOOP*. https://doi.org/10.4230/LIPIcs.ECOOP.2017.24

[99] Alceste Scalas and Nobuko Yoshida. 2019. Less Is More: Multiparty Session Types Revisited. *Proceedings of the ACM on Programming Languages* 3, POPL (Jan. 2019), 1–29. https://doi.org/10.1145/3290343

[100] Alceste Scalas, Nobuko Yoshida, and Elias Benussi. 2019. Verifying message-passing programs with dependent behavioural types. In *Programming Language Design and Implementation*. ACM, 502–516.

[101] Claude Stolze, Marino Miculan, and Pietro Di Gianantonio. 2021. Composable Partial Multiparty Session Types. In *FACS 2021 Conference Proceedings (LNCS, Vol. 13077)*. Springer, Cham, 44–62. https://doi.org/10.1007/978-3-030-90636-8_3

[102] Felix Stutz. 2023. Asynchronous Multiparty Session Type Implementability is Decidable – Lessons Learned from Message Sequence Charts. arXiv:2302.11272 [cs.FL] https://arxiv.org/abs/2302.11272

[103] Felix Stutz. 2023. *The Power of Choice: Implementability of Asynchronous Communication Protocols*. Dr. rer. nat. Thesis. University of Kaiserslautern–Landau, Kaiserslautern, Germany. https://kluedo.ub.rptu.de/files/8077/Doctoral-Thesis-Felix-Stutz.pdf Reviewer: Prof. Dr. Rupak Majumdar; Defense date: 15 December 2023.

[104] Joseph Tassarotti, Ralf Jung, and Robert Harper. 2017. A Higher-Order Logic for Concurrent Termination-Preserving Refinement. In *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10201)*, Hongseok Yang (Ed.). Springer, 909–936. https://doi.org/10.1007/978-3-662-54434-1_34

[105] mCRL2 team. 2025. mCRL2 homepage. https://mcrl2.org/web/index.html.

[106] SPIN team. 2025. SPIN homepage. https://spinroot.com/spin/whatispin.html.

[107] Dawit Tirore, Jesper Bengtson, and Marco Carbone. 2023. A Sound and Complete Projection for Global Types. In *ITP*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 19 pages. https://doi.org/10.4230/LIPICS.ITP.2023.28

[108] Dawit Legesse Tirore, Jesper Bengtson, and Marco Carbone. 2023. A Sound and Complete Projection for Global Types. In *14th International Conference on Interactive Theorem Proving, ITP 2023, July 31 to August 4, 2023, Białystok, Poland (LIPIcs, Vol. 268)*, Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 28:1–28:19. https://doi.org/10.4230/LIPICS.ITP.2023.28

[109] Dawit Legesse Tirore, Jesper Bengtson, and Marco Carbone. 2025. Multiparty Asynchronous Session Types: A Mechanised Proof of Subject Reduction. In *39th European Conference on Object-Oriented Programming, ECOOP 2025, June 30 to July 2, 2025, Bergen, Norway (LIPIcs, Vol. 333)*, Jonathan Aldrich and Alexandra Silva (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 31:1–31:30. https://doi.org/10.4230/LIPICS.ECOOP.2025.31

[110] Dawit Legesse Tirore, Jesper Bengtson, and Marco Carbone. 2025. A Sound and Complete Projection for Global Types. *J. Autom. Reason.* 69, 2 (2025), 14. https://doi.org/10.1007/S10817-025-09726-9

[111] Thien Udomsrirungruang and Nobuko Yoshida. 2024. Top-Down or Bottom-Up? Complexity Analyses of Synchronous Multiparty Session Types. arXiv:2411.07452 [cs.PL] https://arxiv.org/abs/2411.07452

[112] Thien Udomsrirungruang and Nobuko Yoshida. 2025. Top-Down or Bottom-Up? Complexity Analyses of Synchronous Multiparty Session Types. *Proc. ACM Program. Lang.* 9, POPL (2025), 1040–1071. https://doi.org/10.1145/3704872

[113] Rob van Glabbeek, Peter Höfner, and Ross Horne. 2021. Assuming just enough fairness to make session types complete for lock-freedom. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science* (Rome, Italy) *(LICS '21)*. Association for Computing Machinery, New York, NY, USA, Article 8, 13 pages. https://doi.org/10.1109/LICS52264.2021.9470531

[114] Martin Vassor and Nobuko Yoshida. 2024. Refinements for Multiparty Message-Passing Protocols: Specification-Agnostic Theory and Implementation. In *38th European Conference on Object-Oriented Programming, ECOOP 2024, September 16-20, 2024, Vienna, Austria (LIPIcs, Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 41:1–41:29. https://doi.org/10.4230/LIPICS.ECOOP.2024.41

[115] Malte Viering, Raymond Hu, Patrick Eugster, and Lukasz Ziarek. 2021. A Multiparty Session Typing Discipline for Fault-Tolerant Event-Driven Distributed Programming. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 124 (Oct. 2021), 30 pages. https://doi.org/10.1145/3485501

[116] Nobuko Yoshida. 2024. Programming Language Implementations with Multiparty Session Types. In *Active Object Languages: Current Research Trends*. Springer Nature Switzerland, 147–165.

[117] Nobuko Yoshida and Lorenzo Gheri. 2020. A Very Gentle Introduction to Multiparty Session Types. In *Distributed Computing and Internet Technology (ICDCIT 2020) (LNCS, Vol. 11969)*. Springer, 73–93. https://doi.org/10.1007/978-3-030-36987-3_5

[118] Nobuko Yoshida, Fangyi Zhou, and Francisco Ferreira. 2021. Communicating Finite State Machines and an Extensible Toolchain for Multiparty Session Types. In *23rd International Symposium on Fundamentals of Computation Theory (LNCS, Vol. 12867)*. Springer, 18–35. https://doi.org/10.1007/978-3-030-86593-1_2

[119] Fangyi Zhou, Francisco Ferreira, Raymond Hu, Rumyana Neykova, and Nobuko Yoshida. 2020. Statically Verified Refinements for Multiparty Protocols. In *OOPSLA 2020: Conference on Object-Oriented Programming Systems, Languages and Applications (PACMPL, Vol. 4)*. ACM, 148:1–148:30. Issue OOPSLA. https://doi.org/10.1145/3428216

A AUXILIARY MATERIAL

A.1 Big-Step Evaluation of Expressions

To make the premises $e \downarrow v$ in Figure 3 explicit, we record a standard big-step semantics for closed expressions. Variables are eliminated by communication substitution before evaluation, so the relation is only used on closed expressions. We write $[\![\diamond]\!]$ for the usual partial meta-level operation of a unary operator $\diamond \in \{\neg, \sqrt{}\}$, and $[\![\odot]\!]$ for binary operators $\odot \in \{\vee, +, -, *, /, \leq\}$.

$$[\textsc{EVal}]\ \frac{}{v \downarrow v} \qquad [\textsc{ERand}]\ \frac{f \in [0,1]}{\mathsf{rand}() \downarrow f}$$

$$[\textsc{EUn}]\ \frac{e \downarrow v \qquad v' = [\![\diamond]\!](v)}{\diamond e \downarrow v'} \qquad [\textsc{EBin}]\ \frac{e_1 \downarrow v_1 \qquad e_2 \downarrow v_2 \qquad v = [\![\odot]\!](v_1, v_2)}{e_1 \odot e_2 \downarrow v}$$

$$[\textsc{EChoiceL}]\ \frac{e_1 \downarrow v}{e_1 \oplus e_2 \downarrow v} \qquad [\textsc{EChoiceR}]\ \frac{e_2 \downarrow v}{e_1 \oplus e_2 \downarrow v}$$

If one of the meta-level operations is undefined because the operand sorts do not match, then no evaluation rule applies. In the typed fragment this cannot occur.

A.2 Multiparty Session Typing System

We give the full typing rules for the synchronous MPST calculus. The judgment $\Gamma \vdash P : \mathsf{T}$ states that, under term-variable assumptions in Γ, the process P follows the protocol described by the local type T. Rule [T0] assigns the inactive process $\mathbf{0}$ the terminated type end, expressing that no

further communication is possible. Rule [TOut] types an output: if the expression $\mathtt{e}$ has base sort $\mathtt{B}$ and the continuation P follows $\mathtt{T}$, then sending $\mathtt{e}$ to participant $\mathbf{p}$ behaves as $\mathbf{p}!\langle \mathtt{B}\rangle;\mathtt{T}$, i.e., first output a $\mathtt{B}$-value to $\mathbf{p}$ and then continue as $\mathtt{T}$. Dually, [TIn] types an input: assuming a fresh variable $x{:}\mathtt{B}$ for the received value, the continuation P follows $\mathtt{T}$, hence receiving from $\mathbf{p}$ follows $\mathbf{p}?\langle \mathtt{B}\rangle;\mathtt{T}$. Internal choice and external choice mirror the selection/branching constructs. Given a label l and that the continuation after choosing l follows $\mathtt{T}_j$, then selecting l towards $\mathbf{p}$ follows $\mathbf{p}\oplus\{l : \mathtt{T}\}$, which offers exactly one label. Note that internal choices offering multiple labels only arise from subtyping. Conversely, [TBra] types an external choice: if for every $i \in I$ the branch process P_i follows $\mathtt{T}_i$, then offering to accept one of the labels l_i from $\mathbf{p}$ follows $\mathbf{p}\,\&\,\{l_i : \mathtt{T}_i\}_{i\in I}$ and continues accordingly once a label is selected by the peer. Rule [TIf] ensures control-flow does not alter the protocol: when the expression $\mathtt{e}$ has type $\mathtt{bool}$ and both branches follow the same session type $\mathtt{T}$, the conditional as a whole follows $\mathtt{T}$. Combined with the subsumption rule [TSub], this allows compatible but different types in each branch. Rule [TVar] retrieves the session type assigned to a process variable X from the environment, while [TRec] ties the knot: if the body P follows $\mathtt{T}$ under the assumption that X also follows $\mathtt{T}$, then the recursive process $\mu X.\,P$ follows the recursive session type $\mu \mathtt{t}.\mathtt{T}$. Finally, [TSub] provides subsumption: whenever P follows $\mathtt{T}$ and $\mathtt{T} \leqslant \mathtt{T}'$, the same process also follows the supertype $\mathtt{T}'$, allowing safe widening of behaviours such as enlarging offered branches or narrowing accepted branches.

Definition A.1 (Typing rules for processes). The typing rules for processes are as follows.

$$\text{[T0]}\ \ \Gamma \vdash \mathbf{0} : \mathtt{end} \qquad \text{[TOut]}\ \ \frac{\Gamma \vdash \mathtt{e} : \mathtt{B} \qquad \Gamma \vdash P : \mathtt{T}}{\Gamma \vdash \mathbf{p}!\langle \mathtt{e}\rangle.P : \mathbf{p}!\langle \mathtt{B}\rangle;\mathtt{T}} \qquad \text{[TIn]}\ \ \frac{\Gamma, x{:}\mathtt{B} \vdash P : \mathtt{T}}{\Gamma \vdash \mathbf{p}?(x).P : \mathbf{p}?\langle \mathtt{B}\rangle;\mathtt{T}}$$

$$\text{[TSel]}\ \ \frac{\Gamma \vdash P : \mathtt{T}}{\Gamma \vdash \mathbf{p}\triangleleft l.\,P : \mathbf{p}\oplus\{l : \mathtt{T}\}} \qquad \text{[TBra]}\ \ \frac{\forall i \in I : \Gamma \vdash P_i : \mathtt{T}_i}{\Gamma \vdash \mathbf{p}\triangleright\{l_i : P_i\}_{i\in I} : \mathbf{p}\,\&\,\{l_i : \mathtt{T}_i\}_{i\in I}}$$

$$\text{[TIf]}\ \ \frac{\Gamma \vdash \mathtt{e} : \mathtt{bool} \qquad \Gamma \vdash P_1 : \mathtt{T} \qquad \Gamma \vdash P_2 : \mathtt{T}}{\Gamma \vdash \mathbf{if}\ \mathtt{e}\ \mathbf{then}\ P_1\ \mathbf{else}\ P_2 : \mathtt{T}}$$

$$\text{[TVar]}\ \ \frac{\Gamma(X) = \mathtt{T}}{\Gamma \vdash X : \mathtt{T}} \qquad \text{[TRec]}\ \ \frac{\Gamma, X{:}\mathtt{T} \vdash P : \mathtt{T}}{\Gamma \vdash \mu X.\,P : \mu \mathtt{t}.\mathtt{T}} \qquad \text{[TSub]}\ \ \frac{\Gamma \vdash P : \mathtt{T} \qquad \mathtt{T} \leqslant \mathtt{T}'}{\Gamma \vdash P : \mathtt{T}'}$$

B PROOFS

B.1 Proofs from Section 5

Lemma B.1 (Inversion of One-Sided Reduction for Safe Contexts). *Assume* $\mathsf{safe}(\Delta)$ *and* $\Delta \xrightarrow{\mathbf{p}:\zeta} \Delta'$. *Then:*

- *If* $\zeta = \mathbf{q}!\langle \mathtt{B}\rangle$, *then* $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}!\langle \mathtt{B}\rangle;\Delta'(\mathbf{p})$.
- *If* $\zeta = \mathbf{q}?\langle \mathtt{B}\rangle$, *then* $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}?\langle \mathtt{B}\rangle;\Delta'(\mathbf{p})$.
- *If* $\zeta = \mathbf{q}!\langle l_j\rangle$, *then there exists an index set* I *and local types* $\{\mathtt{T}_i\}_{i\in I}$ *such that* $j \in I$, $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}\oplus\{l_i : \mathtt{T}_i\}_{i\in I}$, *and* $\Delta'(\mathbf{p}) = \mathtt{T}_j$.
- *If* $\zeta = \mathbf{q}?\langle l_j\rangle$, *then there exists an index set* I *and local types* $\{\mathtt{T}_i\}_{i\in I}$ *such that* $j \in I$, $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}\,\&\,\{l_i : \mathtt{T}_i\}_{i\in I}$, *and* $\Delta'(\mathbf{p}) = \mathtt{T}_j$.

Proof of Lemma B.1. Assume $\mathsf{safe}(\Delta)$ and $\Delta \xrightarrow{\mathbf{p}:\zeta} \Delta'$. By inversion on rule [LTag], there exist a context Δ_0 and local types T, T' such that

$$\Delta = \Delta_0, \mathbf{p} : \mathsf{T}, \qquad \Delta' = \Delta_0, \mathbf{p} : \mathsf{T}', \qquad \mathsf{T} \xrightarrow{\zeta} \mathsf{T}'.$$

So it is enough to analyse the derivation of $\mathsf{T} \xrightarrow{\zeta} \mathsf{T}'$. We proceed by cases on ζ, and in each case by induction on that derivation.

(1) $\zeta = \mathbf{q}!\langle \mathsf{B} \rangle$. The last rule is either [LR1] or [LR5]. For [LR1], $\mathsf{T} = \mathbf{q}!\langle \mathsf{B} \rangle;\mathsf{T}'$, so $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathsf{unfold}(\mathsf{T}) = \mathbf{q}!\langle \mathsf{B} \rangle;\mathsf{T}' = \mathbf{q}!\langle \mathsf{B} \rangle;\Delta'(\mathbf{p})$. For [LR5], write $\mathsf{T} = \mu \mathbf{t}.\mathsf{T}_0$ and use the induction hypothesis on the premise $\mathsf{T}_0[\mu \mathbf{t}.\mathsf{T}_0/\mathbf{t}] \xrightarrow{\mathbf{q}!\langle \mathsf{B} \rangle} \mathsf{T}'$; then $\mathsf{unfold}(\mathsf{T}) = \mathsf{unfold}(\mathsf{T}_0[\mu \mathbf{t}.\mathsf{T}_0/\mathbf{t}]) = \mathbf{q}!\langle \mathsf{B} \rangle;\mathsf{T}'$.

(2) $\zeta = \mathbf{q}?\langle \mathsf{B} \rangle$. Exactly symmetric to item 1, using [LR2] and [LR5], yielding $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}?\langle \mathsf{B} \rangle;\Delta'(\mathbf{p})$.

(3) $\zeta = \mathbf{q}!\langle l_j \rangle$. The last rule is either [LR3] or [LR5]. For [LR3], there is an index set I with $j \in I$ such that $\mathsf{T} = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$ and $\mathsf{T}' = \mathsf{T}_j$. Hence $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$ and $\Delta'(\mathbf{p}) = \mathsf{T}_j$. For [LR5], write $\mathsf{T} = \mu \mathbf{t}.\mathsf{T}_0$ and apply the induction hypothesis to the premise $\mathsf{T}_0[\mu \mathbf{t}.\mathsf{T}_0/\mathbf{t}] \xrightarrow{\mathbf{q}!\langle l_j \rangle} \mathsf{T}'$.

(4) $\zeta = \mathbf{q}?\langle l_j \rangle$. Symmetric to item 3, using [LR4] and [LR5]. We obtain an index set I with $j \in I$ such that $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q} \,\&\, \{l_i : \mathsf{T}_i\}_{i \in I}$ and $\Delta'(\mathbf{p}) = \mathsf{T}_j$.

□

Lemma B.2 (Inversion of Two-Sided Reduction for Safe Contexts). *Assume* $\mathsf{safe}(\Delta)$ *and* $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$. *Then:*

- *If* $\mathsf{U} = \mathsf{B}$, *then* $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}!\langle \mathsf{B} \rangle;\Delta'(\mathbf{p})$ *and* $\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p}?\langle \mathsf{B} \rangle;\Delta'(\mathbf{q})$.
- *If* $\mathsf{U} = l_j$, *then there exist index sets* I, J *and local types* $\{\mathsf{T}_i\}_{i \in I}, \{\mathsf{T}'_i\}_{i \in J}$ *such that* $j \in I \cap J$, $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$, $\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p} \,\&\, \{l_i : \mathsf{T}'_i\}_{i \in J}$, $\Delta'(\mathbf{p}) = \mathsf{T}_j$, *and* $\Delta'(\mathbf{q}) = \mathsf{T}'_j$.
- *If* $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$, *then* $\Delta(\mathbf{r}) = \Delta'(\mathbf{r})$.

Proof of Lemma B.2. Assume $\mathsf{safe}(\Delta)$ and $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$. By inversion on [LEnv], there exist contexts $\Delta_1, \Delta_2, \Delta'_1, \Delta'_2$ such that

$$\Delta = \Delta_1, \Delta_2, \qquad \Delta' = \Delta'_1, \Delta'_2,$$

and

$$\Delta_1 \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{U} \rangle} \Delta'_1, \qquad \Delta_2 \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{U} \rangle} \Delta'_2.$$

From each one-sided premise, invert [LTag] and re-apply [LTag] with the unchanged participants of Δ. This yields auxiliary one-sided reductions from the whole context:

$$\Delta \xrightarrow{\mathbf{p}:\mathbf{q}!\langle \mathsf{U} \rangle} \Delta_p, \qquad \Delta \xrightarrow{\mathbf{q}:\mathbf{p}?\langle \mathsf{U} \rangle} \Delta_q,$$

where $\Delta_p(\mathbf{p}) = \Delta'(\mathbf{p})$, $\Delta_q(\mathbf{q}) = \Delta'(\mathbf{q})$, and all other participants are unchanged.

Apply Lemma B.1 to both auxiliary reductions.

- If $\mathsf{U} = \mathsf{B}$, we get $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}!\langle \mathsf{B} \rangle;\Delta_p(\mathbf{p})$ and $\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p}?\langle \mathsf{B} \rangle;\Delta_q(\mathbf{q})$, i.e.

$$\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q}!\langle \mathsf{B} \rangle;\Delta'(\mathbf{p}), \qquad \mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p}?\langle \mathsf{B} \rangle;\Delta'(\mathbf{q}).$$

- If $\mathsf{U} = l_j$, the send-side inversion gives an index set I and local types $\{\mathsf{T}_i\}_{i \in I}$ with $j \in I$ such that

$$\mathsf{unfold}(\Delta(\mathbf{p})) = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}, \qquad \Delta_p(\mathbf{p}) = \mathsf{T}_j,$$

hence $\Delta'(\mathbf{p}) = \mathsf{T}_j$. The receive-side inversion gives an index set J and local types $\{\mathsf{T}'_i\}_{i\in J}$ with $j \in J$ such that

$$\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p}\,\&\,\{l_i : \mathsf{T}'_i\}_{i\in J}, \qquad \Delta_q(\mathbf{q}) = \mathsf{T}'_j,$$

hence $\Delta'(\mathbf{q}) = \mathsf{T}'_j$.

Finally, by the shape of [LEnv] and [LTag], only $\mathbf{p}$ and $\mathbf{q}$ are updated, so $\Delta(\mathbf{r}) = \Delta'(\mathbf{r})$ for every $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$. □

Definition B.3 (Projection of round-robin sequences). Let $\sigma = ((\Delta_j, \mathbf{p}'_j) \overset{\ell_j}{\rightsquigarrow} (\Delta_{j+1}, \mathbf{p}'_{j+1}))_{j\in I}$ be a (finite or infinite) round-robin reduction sequence. The *projection* of σ is the context reduction sequence $\pi(\sigma) = (\Delta_j \overset{\ell_j}{\longrightarrow} \Delta_{j+1})_{j\in I}$ obtained by forgetting the prioritised participant at each step. This is well-defined because $(\Delta, \mathbf{p}_i) \overset{\ell}{\rightsquigarrow} (\Delta', \mathbf{p}_k)$ implies $\Delta \overset{\ell}{\longrightarrow} \Delta'$ by the definition of round-robin.

Lemma B.4 (Projection of Round-Robin Sequences is Fair). *Any maximal reduction sequence of the round-robin relation $((\Delta_j, \mathbf{p}'_j) \overset{\ell_j}{\rightsquigarrow} (\Delta_{j+1}, \mathbf{p}'_{j+1}))_{j\in I}$, where $\mathbf{p}'_j$ denotes the prioritised participant at step j, projects (Definition B.3) onto a fair reduction sequence.*

Proof of Lemma B.4. Consider a maximal round-robin sequence $((\Delta_j, \mathbf{p}'_j) \overset{\ell_j}{\rightsquigarrow} (\Delta_{j+1}, \mathbf{p}'_{j+1}))_{j\in I}$, and its projection (Definition B.3) $\pi(\sigma) = (\Delta_j \overset{\ell_j}{\longrightarrow} \Delta_{j+1})_{j\in I}$. We show this projected sequence is fair.

Fix any index $k \in I$ and assume Δ_k has an enabled interaction between $\mathbf{p}$ and $\mathbf{q}$. Let s be the sender index for that enabled pair: if we are looking at an enabled sender, take $s = \mathbf{p}$; if we are looking at an enabled receiver, take s to be the sender index of the participant it is waiting for. So $s \in \mathsf{enabled}(\Delta_k)$.

As long as $\mathbf{p}$ is not involved in a step, it remains enabled. Indeed, if

$$\Delta_j \overset{\mathbf{rs}:\langle \mathsf{U}' \rangle}{\longrightarrow} \Delta_{j+1} \quad \text{with} \quad \mathbf{p} \notin \{\mathbf{r}, \mathbf{s}\},$$

then by Lemma B.2(3) we have $\Delta_{j+1}(\mathbf{p}) = \Delta_j(\mathbf{p})$, so the local action at $\mathbf{p}$ is unchanged and s stays in enabled.

Round-robin always chooses $\mathbf{p}_{j'} = \mathsf{next\text{-}active}(\mathbf{p}'_j)$ and then sets $\mathbf{p}'_{j+1} = \mathbf{p}_{(j'+1) \bmod n}$, meaning it cycles through enabled senders. Since s stays enabled until selected, the cycle reaches s after at most $|\mathsf{enabled}(\Delta_k)|$ steps, and $|\mathsf{enabled}(\Delta_k)| \le \lfloor n/2 \rfloor$. Hence within at most $\lfloor n/2 \rfloor$ steps from k, we take a transition whose sender is $\mathbf{p}$.

Therefore every enabled pair is eventually scheduled in the projected sequence, so the projection is fair. □

B.2 Proofs from Section 6

B.2.1 Combined Relation Definition.

Definition B.5 (Composition of Projection and Subtyping). We define a combined relation of including both projection and subtyping coinductively by the following rules.

$$[\text{SPR1}]\ \frac{\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}}{\mathbf{p} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G} \Vdash_{\mathsf{p}} \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}} \qquad [\text{SPR2}]\ \frac{\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}}{\mathbf{q} \to \mathbf{p}\ \langle \mathsf{B} \rangle; \mathsf{G} \Vdash_{\mathsf{p}} \mathbf{q}?\langle \mathsf{B} \rangle; \mathsf{T}} \qquad [\text{SPR3}]\ \frac{\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T} \qquad \mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}}{\mathbf{q} \to \mathbf{r}\ \langle \mathsf{B} \rangle; \mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}}$$

$$[\text{SPR4}]\ \frac{\forall i \in I : \mathsf{G}_i \Vdash_{\mathsf{p}} \mathsf{T}_i}{\mathbf{p} \to \mathbf{q}\ \{l_i : \mathsf{G}_i\}_{i \in I} \Vdash_{\mathsf{p}} \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}} \qquad [\text{SPR5}]\ \frac{\forall j \in J : \mathsf{G}_j \Vdash_{\mathsf{p}} \mathsf{T}_j \qquad J \subseteq I}{\mathbf{q} \to \mathbf{p}\ \{l_j : \mathsf{G}_j\}_{j \in J} \Vdash_{\mathsf{p}} \mathbf{q} \,\&\, \{l_i : \mathsf{T}_i\}_{i \in I}}$$

$$[\text{SPR6}]\ \frac{\forall i \in I : \mathsf{G}_i \Vdash_{\mathsf{p}} \mathsf{T} \qquad \mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}}{\mathbf{q} \to \mathbf{r}\ \{l_i : \mathsf{G}_i\}_{i \in I} \Vdash_{\mathsf{p}} \mathsf{T}} \qquad [\text{SPR9}]\ \frac{\mathsf{G}[\mu \mathbf{t}.\mathsf{G}/\mathbf{t}] \Vdash_{\mathsf{p}} \mathsf{T} \qquad \mathbf{p} \in \mathrm{pt}(\mathsf{G})}{\mu \mathbf{t}.\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}}$$

$$[\text{SPR10}]\ \frac{\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{T}[\mu \mathbf{t}.\mathsf{T}/\mathbf{t}]}{\mathsf{G} \Vdash_{\mathsf{p}} \mu \mathbf{t}.\mathsf{T}} \qquad [\text{SPR11}]\ \frac{}{\mathsf{end} \Vdash_{\mathsf{p}} \mathsf{end}} \qquad [\text{SPR12}]\ \frac{\mathbf{p} \notin \mathrm{pt}(\mathsf{G})}{\mu \mathbf{t}.\mathsf{G} \Vdash_{\mathsf{p}} \mathsf{end}}$$

B.2.2 *Inversion Lemmas.*

Lemma B.6 (Subtyping is invariant under unfolding). *We have that* $\mathsf{T} \leqslant \mathrm{unfold}(\mathsf{T})$ *and* $\mathrm{unfold}(\mathsf{T}) \leqslant \mathsf{T}$.

Proof. By coinduction. Define the candidate relation $R = \{(\mathsf{T}, \mathsf{T}') \mid \mathrm{unfold}(\mathsf{T}) = \mathrm{unfold}(\mathsf{T}')\}$. Since unfold() is idempotent, $(\mathsf{T}, \mathrm{unfold}(\mathsf{T})) \in R$ and $(\mathrm{unfold}(\mathsf{T}), \mathsf{T}) \in R$ for every T. It suffices to show R is consistent with the subtyping rules, i.e. for every $(\mathsf{T}, \mathsf{T}') \in R$ there is a subtyping rule whose conclusion matches $\mathsf{T} \leqslant \mathsf{T}'$ and whose premises lie in R.

Take $(\mathsf{T}, \mathsf{T}') \in R$ with $\mathrm{unfold}(\mathsf{T}) = \mathrm{unfold}(\mathsf{T}')$.

If T *is a* μ*-type*, say $\mathsf{T} = \mu \mathbf{t}.\mathsf{T}_0$. Rule [S5] applies: the premise requires $(\mathsf{T}_0[\mu \mathbf{t}.\mathsf{T}_0/\mathbf{t}], \mathsf{T}') \in R$. Since $\mathrm{unfold}(\mathsf{T}_0[\mu \mathbf{t}.\mathsf{T}_0/\mathbf{t}]) = \mathrm{unfold}(\mathsf{T}) = \mathrm{unfold}(\mathsf{T}')$, the premise is in R.

If T' *is a* μ*-type*, say $\mathsf{T}' = \mu \mathbf{t}.\mathsf{T}'_0$. Rule [S6] applies: the premise requires $(\mathsf{T}, \mathsf{T}'_0[\mu \mathbf{t}.\mathsf{T}'_0/\mathbf{t}]) \in R$. Since $\mathrm{unfold}(\mathsf{T}'_0[\mu \mathbf{t}.\mathsf{T}'_0/\mathbf{t}]) = \mathrm{unfold}(\mathsf{T}') = \mathrm{unfold}(\mathsf{T})$, the premise is in R.

If neither T *nor* T' *is a* μ*-type*, then $\mathsf{T} = \mathrm{unfold}(\mathsf{T}) = \mathrm{unfold}(\mathsf{T}') = \mathsf{T}'$. The appropriate structural rule applies: [S1] for $!\langle\rangle;$, [S2] for $?\langle\rangle;$, [S3] for $\oplus\{\}$, [S4] for $\&\{\}$, or [S7] for end. In each case, the premises are pairs $(\mathsf{T}_i, \mathsf{T}_i)$, which lie in R since $\mathrm{unfold}(\mathsf{T}_i) = \mathrm{unfold}(\mathsf{T}_i)$. □

Lemma B.7 (Inversion of Subtyping). *If* $\mathsf{T} \leqslant \mathsf{T}'$ *then:*

- *If* $\mathsf{T}' = \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}''$, *then* $\mathrm{unfold}(\mathsf{T}) = \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}_0$ *and* $\mathsf{T}_0 \leqslant \mathsf{T}''$.
- *If* $\mathsf{T}' = \mathbf{q}?\langle \mathsf{B} \rangle; \mathsf{T}''$, *then* $\mathrm{unfold}(\mathsf{T}) = \mathbf{q}?\langle \mathsf{B} \rangle; \mathsf{T}_0$ *and* $\mathsf{T}_0 \leqslant \mathsf{T}''$.
- *If* $\mathsf{T}' = \mathbf{q} \oplus \{l_i : \mathsf{T}''_i\}_{i \in I}$, *then there exists* $J \subseteq I$ *such that* $\mathrm{unfold}(\mathsf{T}) = \mathbf{q} \oplus \{l_j : \mathsf{T}_j\}_{j \in J}$, *and* $\mathsf{T}_j \leqslant \mathsf{T}''_j$ *for all* $j \in J$.
- *If* $\mathsf{T}' = \mathbf{q} \,\&\, \{l_i : \mathsf{T}''_i\}_{i \in I}$, *then there exists* $J \supseteq I$ *such that* $\mathrm{unfold}(\mathsf{T}) = \mathbf{q} \,\&\, \{l_j : \mathsf{T}_j\}_{j \in J}$, *and* $\mathsf{T}_i \leqslant \mathsf{T}''_i$ *for all* $i \in I$.
- *If* $\mathsf{T}' = \mathsf{end}$, *then* $\mathrm{unfold}(\mathsf{T}) = \mathsf{end}$.

Proof. Assume $\mathsf{T} \leqslant \mathsf{T}'$. By Lemma B.6, $\mathrm{unfold}(\mathsf{T}) \leqslant \mathsf{T} \leqslant \mathsf{T}'$, so $\mathrm{unfold}(\mathsf{T}) \leqslant \mathsf{T}'$. Since $\mathrm{unfold}(\mathsf{T})$ is not a μ-type, rule [S5] does not apply, and the derivation of $\mathrm{unfold}(\mathsf{T}) \leqslant \mathsf{T}'$ must proceed by one of the non-unfolding rules (possibly applying [S6] to T' first, but T' is given in constructor form, so [S6] does not apply either). In each case the matching rule forces the head constructor of $\mathrm{unfold}(\mathsf{T})$:

- $\mathsf{T}' = \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}''$: only [S1] applies, so $\mathrm{unfold}(\mathsf{T}) = \mathbf{q}!\langle \mathsf{B} \rangle; \mathsf{T}_0$ with $\mathsf{T}_0 \leqslant \mathsf{T}''$.
- $\mathsf{T}' = \mathbf{q}?\langle \mathsf{B} \rangle; \mathsf{T}''$: only [S2] applies, so $\mathrm{unfold}(\mathsf{T}) = \mathbf{q}?\langle \mathsf{B} \rangle; \mathsf{T}_0$ with $\mathsf{T}_0 \leqslant \mathsf{T}''$.
- $\mathsf{T}' = \mathbf{q} \oplus \{l_i : \mathsf{T}''_i\}_{i \in I}$: only [S3] applies, so $\mathrm{unfold}(\mathsf{T}) = \mathbf{q} \oplus \{l_j : \mathsf{T}_j\}_{j \in J}$ with $J \subseteq I$ and $\mathsf{T}_j \leqslant \mathsf{T}''_j$ for all $j \in J$.
- $\mathsf{T}' = \mathbf{q} \,\&\, \{l_i : \mathsf{T}''_i\}_{i \in I}$: only [S4] applies, so $\mathrm{unfold}(\mathsf{T}) = \mathbf{q} \,\&\, \{l_j : \mathsf{T}_j\}_{j \in J}$ with $J \supseteq I$ and $\mathsf{T}_i \leqslant \mathsf{T}''_i$ for all $i \in I$.

- $T' = \mathsf{end}$: only [S7] applies, so $\mathrm{unfold}(T) = \mathsf{end}$.

□

Lemma B.8 (Inversion of Global Type Reduction). *Assume* $G \xrightarrow{\mathbf{pq}:\langle U\rangle} G'$. *Then one of the following holds:*

- $U = B$, $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \langle B\rangle; G_0$, *and* $G' = G_0$.
- $U = l_j$, *there exists an index set* I *with* $j \in I$ *such that* $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \{l_i : G_i\}_{i\in I}$ *and* $G' = G_j$.
- $\mathrm{unfold}(G) = \mathbf{r} \to \mathbf{s}\ \langle B\rangle; G_0$ *with* $\{\mathbf{p},\mathbf{q}\} \cap \{\mathbf{r},\mathbf{s}\} = \emptyset$, $G_0 \xrightarrow{\mathbf{pq}:\langle U\rangle} G_0'$, *and* $G' = \mathbf{r} \to \mathbf{s}\ \langle B\rangle; G_0'$.
- $\mathrm{unfold}(G) = \mathbf{r} \to \mathbf{s}\ \{l_i : G_i\}_{i\in I}$ *with* $\{\mathbf{p},\mathbf{q}\} \cap \{\mathbf{r},\mathbf{s}\} = \emptyset$, $G_i \xrightarrow{\mathbf{pq}:\langle U\rangle} G_i'$ *for all* $i \in I$, *and* $G' = \mathbf{r} \to \mathbf{s}\ \{l_i : G_i'\}_{i\in I}$.

Proof of Lemma B.8. By induction on the derivation of $G \xrightarrow{\mathbf{pq}:\langle U\rangle} G'$.

- [GR1]: $G = \mathbf{p} \to \mathbf{q}\ \langle B\rangle; G_0$, $U = B$, $G' = G_0$. Since G is not a μ-type, $\mathrm{unfold}(G) = G$. Case 1.
- [GR4]: $G = \mathbf{p} \to \mathbf{q}\ \{l_i : G_i\}_{i\in I}$, $U = l_j$, $j \in I$, $G' = G_j$. Since G is not a μ-type, $\mathrm{unfold}(G) = G$. Case 2.
- [GR3]: $G = \mu \mathbf{t}.G_1$ and $G_1[\mu \mathbf{t}.G_1/\mathbf{t}] \xrightarrow{\mathbf{pq}:\langle U\rangle} G'$. By the induction hypothesis, one of cases 1–4 holds for $G_1[\mu \mathbf{t}.G_1/\mathbf{t}]$. Since $\mathrm{unfold}(G) = \mathrm{unfold}(G_1[\mu \mathbf{t}.G_1/\mathbf{t}])$, the same case holds for G.
- [GR2]: $G = \mathbf{r} \to \mathbf{s}\ \langle B\rangle; G_0$, $\{\mathbf{p},\mathbf{q}\} \cap \{\mathbf{r},\mathbf{s}\} = \emptyset$, $G_0 \xrightarrow{\mathbf{pq}:\langle U\rangle} G_0'$, $G' = \mathbf{r} \to \mathbf{s}\ \langle B\rangle; G_0'$. Since G is not a μ-type, $\mathrm{unfold}(G) = G$. Case 3.
- [GR5]: $G = \mathbf{r} \to \mathbf{s}\ \{l_i : G_i\}_{i\in I}$, $\{\mathbf{p},\mathbf{q}\} \cap \{\mathbf{r},\mathbf{s}\} = \emptyset$, $G_i \xrightarrow{\mathbf{pq}:\langle U\rangle} G_i'$ for all $i \in I$, $G' = \mathbf{r} \to \mathbf{s}\ \{l_i : G_i'\}_{i\in I}$. Since G is not a μ-type, $\mathrm{unfold}(G) = G$. Case 4. □

Lemma B.9 (Inversion of the Combined Relation). *Assume* $G \Vdash_{\mathbf{p}} T$. *Then:*

- *If* $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \langle B\rangle; G_0$, *then* $\mathrm{unfold}(T) = \mathbf{q}!\langle B\rangle; T_0$ *and* $G_0 \Vdash_{\mathbf{p}} T_0$.
- *If* $\mathrm{unfold}(G) = \mathbf{q} \to \mathbf{p}\ \langle B\rangle; G_0$, *then* $\mathrm{unfold}(T) = \mathbf{q}?\langle B\rangle; T_0$ *and* $G_0 \Vdash_{\mathbf{p}} T_0$.
- *If* $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \{l_i : G_i\}_{i\in I}$, *then* $\mathrm{unfold}(T) = \mathbf{q} \oplus \{l_i : T_i\}_{i\in I}$ *and* $G_i \Vdash_{\mathbf{p}} T_i$ *for all* $i \in I$.
- *If* $\mathrm{unfold}(G) = \mathbf{q} \to \mathbf{p}\ \{l_j : G_j\}_{j\in J}$, *then there exists* $I \supseteq J$ *such that* $\mathrm{unfold}(T) = \mathbf{q} \& \{l_i : T_i\}_{i\in I}$ *and* $G_j \Vdash_{\mathbf{p}} T_j$ *for all* $j \in J$.
- *If* $\mathrm{unfold}(G) = \mathbf{q} \to \mathbf{r}\ \langle B\rangle; G_0$ *with* $\mathbf{p} \notin \{\mathbf{q},\mathbf{r}\}$, *then* $G_0 \Vdash_{\mathbf{p}} T$.
- *If* $\mathrm{unfold}(G) = \mathbf{q} \to \mathbf{r}\ \{l_i : G_i\}_{i\in I}$ *with* $\mathbf{p} \notin \{\mathbf{q},\mathbf{r}\}$, *then* $G_i \Vdash_{\mathbf{p}} T$ *for all* $i \in I$.
- *If* $\mathrm{unfold}(G) = \mathsf{end}$, *then* $\mathrm{unfold}(T) = \mathsf{end}$.

Proof of Lemma B.9. Since $\Vdash_{\mathbf{p}}$ is defined coinductively (as a greatest fixpoint), every pair $(G, T) \in \Vdash_{\mathbf{p}}$ is justified by a rule application whose premises also lie in $\Vdash_{\mathbf{p}}$. We proceed by induction on the total number of μ-unfolding steps from G to $\mathrm{unfold}(G)$ plus from T to $\mathrm{unfold}(T)$ (well-founded by guardedness of types).

If G *is a* μ*-type*, say $G = \mu \mathbf{t}.G_1$. The rule justifying $(G, T) \in \Vdash_{\mathbf{p}}$ is either [SPR9] (if $\mathbf{p} \in \mathrm{pt}(G_1)$) or [SPR12] (if $\mathbf{p} \notin \mathrm{pt}(G_1)$). For [SPR12]: $T = \mathsf{end}$ and $\mathbf{p} \notin \mathrm{pt}(G)$. Since $\mathbf{p}$ does not appear as a participant in $\mathrm{unfold}(G)$, cases 1–4 cannot arise; for cases 5–6 the conclusion $G_0 \Vdash_{\mathbf{p}} T$ (resp. $G_i \Vdash_{\mathbf{p}} T$ for all i) holds because $\mathbf{p} \notin \mathrm{pt}(G)$ implies $\mathbf{p} \notin \mathrm{pt}(G_0)$ (resp. $\mathbf{p} \notin \mathrm{pt}(G_i)$), and a straightforward coinductive argument shows $G' \Vdash_{\mathbf{p}} \mathsf{end}$ whenever $\mathbf{p} \notin \mathrm{pt}(G')$; case 7 gives $\mathrm{unfold}(T) = \mathsf{end}$ immediately. For [SPR9]: the premise $G_1[\mu \mathbf{t}.G_1/\mathbf{t}] \Vdash_{\mathbf{p}} T$ holds, and $\mathrm{unfold}(G) = \mathrm{unfold}(G_1[\mu \mathbf{t}.G_1/\mathbf{t}])$ with strictly fewer unfolding steps on the G-side. The result follows by the induction hypothesis.

If G *is not a* μ*-type but* T *is*, say $T = \mu \mathbf{t}.T_1$. The only applicable rule is [SPR10], with premise $G \Vdash_{\mathbf{p}} T_1[\mu \mathbf{t}.T_1/\mathbf{t}]$. Since $\mathrm{unfold}(T) = \mathrm{unfold}(T_1[\mu \mathbf{t}.T_1/\mathbf{t}])$, the induction hypothesis (strictly fewer

unfolding steps on the T-side) gives the desired conclusions about unfold(T) and the sub-relations for cases 1–4 and 7. For cases 5–6, the hypothesis gives $\mathsf{G}_0 \Vdash_{\mathsf{p}} \mathsf{T}_1[\mu \mathbf{t}.\mathsf{T}_1/\mathbf{t}]$ (resp. $\mathsf{G}_i \Vdash_{\mathsf{p}} \mathsf{T}_1[\mu \mathbf{t}.\mathsf{T}_1/\mathbf{t}]$ for all i). Applying [SPR10] yields $\mathsf{G}_0 \Vdash_{\mathsf{p}} \mathsf{T}$ (resp. $\mathsf{G}_i \Vdash_{\mathsf{p}} \mathsf{T}$).

If neither G nor T is a μ-type, then G = unfold(G) and T = unfold(T). Rules [SPR9], [SPR10], and [SPR12] do not apply. The remaining rules [SPR1]–[SPR6] and [SPR11] each match exactly one structural form of G for a given $\mathbf{p}$: [SPR1] matches case 1, [SPR2] matches case 2, [SPR4] matches case 3, [SPR5] matches case 4, [SPR3] matches case 5, [SPR6] matches case 6, and [SPR11] matches case 7. In each case the premises of the rule directly yield the stated conclusions. □

B.2.3 *Finiteness Lemmas.*

Lemma B.10 (Finiteness of reachable local types). *For every T, the set $\{\mathsf{T}' \mid \mathsf{T} \longrightarrow^* \mathsf{T}'\}$ is finite.*

Proof of Lemma B.10. Because local types are guarded, each use of [LR5] only unfolds a guarded recursion and exposes one of finitely many constructors already present in the syntax of T. Up to identifying each recursive variable with its unique binder, every reachable local state is a syntactic subterm of T. Since T has finitely many subterms, $\{\mathsf{T}' \mid \mathsf{T} \longrightarrow^* \mathsf{T}'\}$ is finite. □

Lemma B.11 (Finiteness of reachable contexts). *For every Δ, the set $\{\Delta' \mid \Delta \longrightarrow^* \Delta'\}$ is finite.*

Proof of Lemma B.11. Write $\Delta = \{\mathbf{p} : \mathsf{T}_{\mathbf{p}}\}_{\mathbf{p} \in \mathrm{dom}(\Delta)}$. By Lemma B.10, each $\{\mathsf{T}' \mid \mathsf{T}_{\mathbf{p}} \longrightarrow^* \mathsf{T}'\}$ is finite. Every synchronous context step ([LEnv]) updates only two participants and each updated local type follows one local transition. Hence any reachable Δ' satisfies $\Delta'(\mathbf{p}) \in \{\mathsf{T}' \mid \mathsf{T}_{\mathbf{p}} \longrightarrow^* \mathsf{T}'\}$ for each $\mathbf{p} \in \mathrm{dom}(\Delta)$. Therefore

$$\{\Delta' \mid \Delta \longrightarrow^* \Delta'\} \subseteq \prod_{\mathbf{p} \in \mathrm{dom}(\Delta)} \{\mathsf{T}' \mid \mathsf{T}_{\mathbf{p}} \longrightarrow^* \mathsf{T}'\},$$

and the right-hand side is finite because $\mathrm{dom}(\Delta)$ is finite. □

Lemma B.12 (Finiteness of reachable context-participant spaces). *For any context Δ, the set $\{(\Delta', \mathbf{p}) \mid \Delta \longrightarrow^* \Delta'$ and $\mathbf{p} \in \mathrm{dom}(\Delta)\}$ is finite.*

Proof of Lemma B.12. By Lemma B.11, $\{\Delta' \mid \Delta \longrightarrow^* \Delta'\}$ is finite, and $\mathrm{dom}(\Delta)$ is finite. Their product is finite. □

Lemma B.13 (Finiteness of branching). *For every Δ, the set $\{(\ell, \Delta') \mid \Delta \xrightarrow{\ell} \Delta'\}$ is finite.*

Proof of Lemma B.13. The number of participants is finite. For each participant, the enabled one-step local actions are finite by inspection of rules [LR1]– [LR5] (payload send/receive, finite label choices, and guarded unfolding). A context transition is formed by pairing one send and one matching receive via [LEnv]. Therefore only finitely many synchronous transitions are possible from Δ. □

Lemma 6.3 (Termination of Synthesis on Safe-and-Live Inputs). *For any context Δ satisfying* slive(Δ) *and any priority $\mathbf{p}_i$, the computation synth($\varnothing, \mathbf{p}_i, \Delta$) terminates.*

Proof of Lemma 6.3. Fix Δ and participant $\mathbf{p}_i$. Define

$$S = \{(\Delta', \mathbf{p}) \mid \Delta \longrightarrow^* \Delta' \text{ and } \mathbf{p} \in \mathrm{dom}(\Delta)\}.$$

By Lemma B.12, S is finite.

We prove a stronger statement: for every Σ, $\mathbf{p}_{i'}$, and Δ' such that $\Delta \longrightarrow^* \Delta'$ and $\mathrm{dom}(\Sigma) \subseteq S$, the computation synth($\Sigma, \mathbf{p}_{i'}, \Delta'$) terminates.

Use well-founded induction on the natural number

$$m(\Sigma) = |S \setminus \mathrm{dom}(\Sigma)|\,.$$

Consider one call $\mathsf{synth}(\Sigma, \mathbf{p_{i'}}, \Delta')$.

(1) If $\mathsf{stuck}(\Delta')$, it returns end immediately.
(2) Otherwise let $\mathbf{p_j} = \text{next-active}(\mathbf{p_{i'}})$. If $(\Delta', \mathbf{p_j}) \in \mathrm{dom}(\Sigma)$, it returns $\Sigma(\Delta', \mathbf{p_j})$ immediately.
(3) Otherwise it enters the recursive branch. Let $\Sigma' = \Sigma \cup \{(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}\}$. Since $\Delta \longrightarrow^* \Delta'$ and $\mathbf{p_j} \in \mathrm{dom}(\Delta)$, we have $(\Delta', \mathbf{p_j}) \in S$, and by case assumption $(\Delta', \mathbf{p_j}) \notin \mathrm{dom}(\Sigma)$, therefore

$$m(\Sigma') = m(\Sigma) - 1.$$

For each $l \in L$, choose Δ_l with $\Delta' \xrightarrow{\mathbf{p_j}\,\mathbf{q}:\langle l \rangle} \Delta_l$. Then $\Delta \longrightarrow^* \Delta_l$, and still $\mathrm{dom}(\Sigma') \subseteq S$. Hence each recursive call $\mathsf{synth}(\Sigma', \mathbf{p_{(j+1)\bmod n}}, \Delta_l)$ satisfies the induction hypothesis (strictly smaller measure), so it terminates. By Lemma B.13, L is finite, so there are only finitely many such recursive calls.

All branches terminate, so $\mathsf{synth}(\Sigma, \mathbf{p_{i'}}, \Delta')$ terminates. Instantiating $\Sigma = \varnothing$, $\mathbf{p_{i'}} = \mathbf{p_i}$, and $\Delta' = \Delta$ yields the claim. □

Remark B.1 (Induction principle of the synthesis procedure). The termination proof above establishes a general induction principle. Let $S = \{(\Delta', \mathbf{p}) \mid \Delta \longrightarrow^* \Delta',\ \mathbf{p} \in \mathrm{dom}(\Delta)\}$. To prove a property $P(\Sigma, \mathbf{p_i}, \Delta')$ for all Σ with $\mathrm{dom}(\Sigma) \subseteq S$, all $\mathbf{p_i}$, and all Δ' reachable from Δ, it suffices to assume P holds for every recursive sub-call of synth (which has a strictly smaller measure m) and verify P in each of the three cases of Definition 6.1.

B.2.4 *Substitution and Structural Properties.*

Lemma 6.4 (Substitution for Synth). [Proof in Appendix B.2] *For any environment* Σ*, contexts* Δ *and* Δ'*, priorities* $\mathbf{p_i}$ *and* $\mathbf{p_j}$*, and type variable* $\mathbf{t}$*,*

$$\mathit{synth}\big(\Sigma \cup \{(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}\}, \mathbf{p_i}, \Delta\big)\,[\mathit{synth}(\Sigma, \mathbf{p_j}, \Delta')/\mathbf{t}] = \mathit{synth}(\Sigma, \mathbf{p_i}, \Delta).$$

Proof of Lemma 6.4. Let $\Sigma' = \Sigma \cup \{(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}\}$. By the induction principle of Remark B.1, we induct on $m(\Sigma)$ and case-split on $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta)$ following the three cases of Definition 6.1. Let $\mathbf{p_k} = \text{next-active}(\mathbf{p_i})$.

Case (1): $\mathsf{stuck}(\Delta)$. Both $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta)$ and $\mathsf{synth}(\Sigma', \mathbf{p_i}, \Delta)$ return end, so $\mathsf{end}[\cdots/\mathbf{t}] = \mathsf{end}$.

Case (2): $(\Delta, \mathbf{p_k}) \in \mathrm{dom}(\Sigma)$, say $\Sigma(\Delta, \mathbf{p_k}) = \mathbf{t}'$. Then also $(\Delta, \mathbf{p_k}) \in \mathrm{dom}(\Sigma')$ with the same value $\mathbf{t}'$ (the binding $(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}$ is distinct since $(\Delta, \mathbf{p_k}) \in \mathrm{dom}(\Sigma)$ already). Hence $\mathsf{synth}(\Sigma', \mathbf{p_i}, \Delta) = \mathbf{t}'$, which does not contain $\mathbf{t}$ (it was bound before $\mathbf{t}$ was introduced), so $\mathbf{t}'[\cdots/\mathbf{t}] = \mathbf{t}' = \mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta)$.

Case (3): $\neg\mathsf{stuck}(\Delta)$ and $(\Delta, \mathbf{p_k}) \notin \mathrm{dom}(\Sigma)$. Two sub-cases arise for $\mathsf{synth}(\Sigma', \mathbf{p_i}, \Delta)$:

Sub-case $(\Delta, \mathbf{p_k}) = (\Delta', \mathbf{p_j})$: Then $\Sigma'(\Delta, \mathbf{p_k}) = \mathbf{t}$, so $\mathsf{synth}(\Sigma', \mathbf{p_i}, \Delta) = \mathbf{t}$. On the other side, $\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta) = \mathsf{synth}(\Sigma, \mathbf{p_j}, \Delta')$ (since $\mathbf{p_k} = \mathbf{p_j}$ and $\Delta = \Delta'$). Therefore $\mathbf{t}[\mathsf{synth}(\Sigma, \mathbf{p_j}, \Delta')/\mathbf{t}] = \mathsf{synth}(\Sigma, \mathbf{p_j}, \Delta')$.

Sub-case $(\Delta, \mathbf{p_k}) \neq (\Delta', \mathbf{p_j})$: Both sides enter the recursive branch with the same transition set L and fresh variable $\mathbf{t_s}$. The RHS introduces $\Sigma_1 = \Sigma \cup \{(\Delta, \mathbf{p_k}) \mapsto \mathbf{t_s}\}$ and the LHS $\Sigma'_1 = \Sigma' \cup \{(\Delta, \mathbf{p_k}) \mapsto \mathbf{t_s}\} = \Sigma_1 \cup \{(\Delta', \mathbf{p_j}) \mapsto \mathbf{t}\}$. For each $l \in L$, $m(\Sigma_1) < m(\Sigma)$, so by the induction hypothesis, $\mathsf{synth}(\Sigma'_1, \ldots, \Delta_l)[\mathsf{synth}(\Sigma_1, \mathbf{p_j}, \Delta')/\mathbf{t}] = \mathsf{synth}(\Sigma_1, \ldots, \Delta_l)$. The outer $\mu\mathbf{t_s}$ and branching structure are identical on both sides, yielding the result. □

Lemma 6.5 (Non-Terminated Participant Appears in Synthesised Type). *For any context* Δ *satisfying* $\mathsf{slive}(\Delta)$*, any priority* $\mathbf{p_i}$*, and any participant* $\mathbf{p} \in \mathit{dom}(\Delta)$*, if* $\mathsf{unfold}(\Delta(\mathbf{p})) \neq \mathsf{end}$*, then* $\mathbf{p} \in \mathsf{pt}(\mathit{synth}(\varnothing, \mathbf{p_i}, \Delta))$.

Proof of Lemma 6.5. Fix Δ, $\mathbf{p_i}$, and $\mathbf{p}$ such that slive(Δ) and unfold($\Delta(\mathbf{p})$) $\neq$ end. Write

$$\mathsf{G} = \mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta),$$

and assume, for contradiction, that $\mathbf{p} \notin \mathsf{pt}(\mathsf{G})$.

By Lemma 6.3, the computation of synth($\varnothing$, $\mathbf{p_i}$, Δ) terminates, so its call-unfolding gives a finite recursion tree. Consider an arbitrary root-to-leaf path in this tree:

$$(\Delta_0, \mathbf{p'_0}) \overset{\ell_0}{\rightsquigarrow} (\Delta_1, \mathbf{p'_1}) \overset{\ell_1}{\rightsquigarrow} \cdots \overset{\ell_{m-1}}{\rightsquigarrow} (\Delta_m, \mathbf{p'_m}),$$

where $\mathbf{p'_j}$ denotes the prioritised participant at step j, with $(\Delta_0, \mathbf{p'_0}) = (\Delta, \mathbf{p_i})$.

Since $\mathbf{p} \notin \mathsf{pt}(\mathsf{G})$, no constructor generated along this path involves $\mathbf{p}$. Hence each ℓ_k is between participants different from $\mathbf{p}$. By Lemma B.2, $\Delta_{k+1}(\mathbf{p}) = \Delta_k(\mathbf{p})$ for all $k < m$. Therefore, for all $k \le m$:

$$\Delta_k(\mathbf{p}) = \Delta(\mathbf{p}), \qquad \mathsf{unfold}(\Delta_k(\mathbf{p})) = \mathsf{unfold}(\Delta(\mathbf{p})) \neq \mathsf{end}.$$

So each Δ_k has a one-sided local action at $\mathbf{p}$.

If the leaf is a case (1) leaf, the projected context path is finite maximal and has no interaction involving $\mathbf{p}$, hence it is not live (Definition 5.7). By Lemma B.4, the projection of any maximal round-robin path is fair, so this would be a fair but non-live path from Δ, contradicting slive(Δ).

So the leaf must be a case (2) leaf. Then there is an ancestor on the same path with the same context-participant pair:

$$(\Delta_r, \mathbf{p'_r}) = (\Delta_m, \mathbf{p'_m}), \quad r < m.$$

Let σ be the finite segment from index r to m. Repeating σ forever yields an infinite reduction sequence ρ from Δ in which no interaction involves $\mathbf{p}$, and $\Delta_k(\mathbf{p}) = \Delta(\mathbf{p})$ at every position. Thus ρ is not live (again by Definition 5.7).

Since Δ is live, every fair sequence from Δ must be live. Therefore ρ must be non-fair. But ρ is the projection of an infinite maximal round-robin sequence obtained by looping the same cycle in the recursion tree, and Lemma B.4 implies that projection is fair and so Δ is not live, contradicting our assumption.

Hence $\mathbf{p} \in \mathsf{pt}(\mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta))$. □

Lemma B.14 (Terminated Participant Does Not Appear in Synthesised Type). *For any context Δ satisfying* slive(Δ)*, any priority $\mathbf{p_i}$, and any participant $\mathbf{p} \in \mathit{dom}(\Delta)$, if* unfold($\Delta(\mathbf{p})$) = end*, then* $\mathbf{p} \notin \mathsf{pt}(\mathit{synth}(\varnothing, \mathbf{p_i}, \Delta))$.

Proof of Lemma B.14. We prove the stronger statement: for all Σ, $\mathbf{p_i}$, and Δ' reachable from Δ, if slive(Δ') and unfold($\Delta'(\mathbf{p})$) = end, then $\mathbf{p} \notin \mathsf{pt}(\mathsf{synth}(\Sigma, \mathbf{p_i}, \Delta'))$. By the induction principle of Remark B.1, we case-split on synth(Σ, $\mathbf{p_i}$, Δ'). Let $\mathbf{p_k}$ = next-active($\mathbf{p_i}$).

Case (1): the result is end, and pt(end) = $\emptyset$, so $\mathbf{p} \notin \mathsf{pt}(\mathsf{end})$.

Case (2): $\Sigma(\Delta', \mathbf{p_k}) = \mathbf{t}$, so the result is $\mathbf{t}$ and $\mathsf{pt}(\mathbf{t}) = \emptyset$, hence $\mathbf{p} \notin \mathsf{pt}(\mathbf{t})$.

Case (3): the result is $\mu\mathbf{t}.\mathsf{G}_0$ where G_0 is either $\mathbf{p_k} \to \mathbf{q}\ \langle \mathsf{B} \rangle ; \mathsf{G}'$ (sub-case (a)) or $\mathbf{p_k} \to \mathbf{q}\ \{l : \mathsf{G}_l\}_{l \in L}$ (sub-case (b)). Since unfold($\Delta'(\mathbf{p})$) = end, Lemma B.16 gives $\Delta'(\mathbf{p}) \not\twoheadrightarrow$, so $\mathbf{p}$ cannot be a sender or receiver of any context transition. Hence $\mathbf{p} \neq \mathbf{p_k}$ and $\mathbf{p} \neq \mathbf{q}$. For each successor context Δ_l reached via $\Delta' \xrightarrow{\mathbf{p_k}\mathbf{q}:\langle \mathsf{U} \rangle} \Delta_l$, by Lemma B.2(3), $\Delta_l(\mathbf{p}) = \Delta'(\mathbf{p})$, so unfold($\Delta_l(\mathbf{p})$) = end. By Lemma 6.6, slive(Δ_l). By the induction hypothesis, $\mathbf{p} \notin \mathsf{pt}(\mathsf{G}_l)$ for each successor. Since $\mathbf{p} \neq \mathbf{p_k}$ and $\mathbf{p} \neq \mathbf{q}$, $\mathbf{p} \notin \mathsf{pt}(\mu\mathbf{t}.\mathsf{G}_0)$. □

Lemma B.15 (No-Transition Safe Live Contexts Have Only End Locals). *For any context Δ satisfying* slive(Δ)*, if $\Delta \not\twoheadrightarrow$, then for every $\mathbf{p} \in \mathit{dom}(\Delta)$ we have* unfold($\Delta(\mathbf{p})$) = end.

Proof of Lemma B.15. Suppose for contradiction that $\mathsf{unfold}(\Delta(\mathbf{p})) \neq \mathsf{end}$ for some $\mathbf{p} \in \mathrm{dom}(\Delta)$. By Lemma B.16 (contrapositive), $\Delta(\mathbf{p})$ has at least one local transition, so $\Delta \xrightarrow{\mathbf{p}:\zeta}$ for some action ζ. Since $\Delta \not\twoheadrightarrow$, the only maximal reduction sequence from Δ is the empty (stuck) sequence, which is trivially fair. However, this sequence is not live: the one-sided action at $\mathbf{p}$ is enabled at Δ but no interaction involving $\mathbf{p}$ is ever taken (Definition 5.7). This contradicts $\mathsf{live}(\Delta)$. □

Lemma 6.6 (Preservation of Safety and Liveness under Reduction). *If Δ is safe and live, and $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$, then Δ' is safe and live.*

Proof of Lemma 6.6. *Safety.* By Definition 5.3, $\mathsf{safe}(\Delta)$ requires that for all $\Delta \longrightarrow^* \Delta''$, if $\Delta'' \xrightarrow{\mathbf{r}:\mathbf{s}!\langle \mathsf{U}' \rangle}$ and $\Delta'' \xrightarrow{\mathbf{s}:\mathbf{r}?\langle \mathsf{U}' \rangle}$, then $\Delta'' \xrightarrow{\mathbf{rs}:\langle \mathsf{U}' \rangle}$. Since $\Delta \longrightarrow \Delta'$, every $\Delta' \longrightarrow^* \Delta''$ is also $\Delta \longrightarrow^* \Delta''$, so the condition is inherited by Δ'.

Liveness. Let $(\Delta' = \Delta_0 \xrightarrow{\ell_0} \Delta_1 \xrightarrow{\ell_1} \cdots)$ be a fair reduction sequence from Δ'. Prepending the step $\Delta \xrightarrow{\mathbf{pq}:\langle \mathsf{U} \rangle} \Delta'$ gives a fair reduction sequence from Δ (any enabled pair in the tail was already enabled or becomes enabled after the prefix, and the prefix is finite so fairness is preserved). By $\mathsf{live}(\Delta)$ (Definition 5.8), this extended sequence is live. Hence the original sequence from Δ' is live, so $\mathsf{live}(\Delta')$. □

Lemma B.16 (End Empty). *If $\mathsf{T} \not\twoheadrightarrow$ then $\mathsf{unfold}(\mathsf{T}) = \mathsf{end}$.*

Proof of Lemma B.16. By inspection of rules [LR1]–[LR4], every local type in constructor form other than end (i.e. output, input, selection, or branching) has at least one enabled transition. Rule [LR5] unfolds μ-types, so if T is a μ-type its transitions coincide with those of $\mathsf{unfold}(\mathsf{T})$. Hence if T has no transitions, $\mathsf{unfold}(\mathsf{T})$ is not a μ-type and has no transitions, so $\mathsf{unfold}(\mathsf{T}) = \mathsf{end}$. □

B.2.5 *Backward Closure.*

Lemma 6.7 (Backward Closure of Candidate Relation). *For any participant $\mathbf{p}$, the relation $\widehat{R}_{\mathbf{p}}$ is closed backwards under the rules defining $\Vdash_{\mathbf{p}}$.*

Proof of Lemma 6.7. We show that for every $(\mathsf{G}, \mathsf{T}) \in \widehat{R}_{\mathbf{p}}$, there exists a rule of $\Vdash_{\mathbf{p}}$ whose conclusion matches $\mathsf{G} \Vdash_{\mathbf{p}} \mathsf{T}$ and whose premises all lie in $\widehat{R}_{\mathbf{p}}$.

By definition, every $(\mathsf{G}, \mathsf{T}) \in \widehat{R}_{\mathbf{p}}$ originates from some $(\mathsf{G}', \mathsf{T}') \in R_{\mathbf{p}}$, meaning $\mathsf{G}' = \mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta)$ and $\mathsf{T}' = \Delta(\mathbf{p})$ for some $\mathsf{slive}(\Delta)$ with $\mathbf{p} \in \mathrm{dom}(\Delta)$, with G either equal to G' or its one-step unfolding via [RUnfL] (which applies from $R_{\mathbf{p}}$ only), and T obtained from T' by finitely many applications of [RUnfR] (which applies from $\widehat{R}_{\mathbf{p}}$, chaining through nested μ-binders). We organise the proof by the form of G.

Case $\mathsf{T} = \mu \mathbf{t}'.\mathsf{T}'$. Rule [SPR10] applies; its premise $(\mathsf{G}, \mathsf{T}'[\mu \mathbf{t}'.\mathsf{T}'/\mathbf{t}'])$ lies in $\widehat{R}_{\mathbf{p}}$ by [RUnfR]. Henceforth assume T is not a μ-type.

Case $\mathsf{G} = \mathsf{end}$. Then $\mathsf{stuck}(\Delta)$. By Lemma B.15, $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathsf{end}$, so $\mathsf{T} = \mathsf{end}$. Rule [SPR11].

Case $\mathsf{G} = \mu \mathbf{t}.\mathsf{G}_0$. Then synthesis applied Definition 6.1(3). If $\mathbf{p} \in \mathrm{pt}(\mathsf{G}_0)$: rule [SPR9] applies. Its premise is $(\mathsf{G}_0[\mu \mathbf{t}.\mathsf{G}_0/\mathbf{t}], \mathsf{T})$. Since $(\mu \mathbf{t}.\mathsf{G}_0, \Delta(\mathbf{p})) \in R_{\mathbf{p}}$, [RUnfL] gives $(\mathsf{G}_0[\mu \mathbf{t}.\mathsf{G}_0/\mathbf{t}], \Delta(\mathbf{p})) \in \widehat{R}_{\mathbf{p}}$; if $\mathsf{T} \neq \Delta(\mathbf{p})$ (obtained via repeated [RUnfR]), the same chain of [RUnfR] steps applied to $(\mathsf{G}_0[\mu \mathbf{t}.\mathsf{G}_0/\mathbf{t}], \Delta(\mathbf{p}))$ yields the premise in $\widehat{R}_{\mathbf{p}}$.

If $\mathbf{p} \notin \mathrm{pt}(\mathsf{G}_0)$: since $\mathrm{pt}(\mu \mathbf{t}.\mathsf{G}_0) = \mathrm{pt}(\mathsf{G}_0)$, the contrapositive of Lemma 6.5 gives $\mathsf{unfold}(\Delta(\mathbf{p})) = \mathsf{end}$, so $\mathsf{T} = \mathsf{end}$. Rule [SPR12].

Case G *is in communication form.* This arises via [RUnfL]: synthesis hit Definition 6.1(3) with sender $\mathbf{p_j}$ = next-active($\mathbf{p_i}$) and receiver $\mathbf{q}$. Write $\mathsf{G}' = \mu\mathbf{t}.\mathsf{G}_0 = \mathsf{synth}(\varnothing, \mathbf{p_i}, \Delta)$. In both sub-cases (a) and (b), for each successor context Δ_l reached via $\Delta \xrightarrow{\mathbf{p_j}\,\mathbf{q}:\langle \mathsf{U} \rangle} \Delta_l$, the synthesis sub-call gives $\mathsf{G}_l = \mathsf{synth}(\{(\Delta, \mathbf{p_j}) \mapsto \mathbf{t}\},\ \mathbf{p_{(j+1)\bmod n}},\ \Delta_l)$. By Lemma 6.4 (noting $\mathsf{synth}(\{(\Delta, \mathbf{p_j}) \mapsto \mathbf{t}\}, \mathbf{p_i}, \Delta) = \mathbf{t}$ via Definition 6.1(2), hence $\mathsf{synth}(\varnothing, \mathbf{p_j}, \Delta) = \mathsf{G}'$):

$$\mathsf{G}_l[\mathsf{G}'/\mathbf{t}] \;=\; \mathsf{synth}(\varnothing,\ \mathbf{p_{(j+1)\bmod n}},\ \Delta_l).$$

By Lemma 6.6, $\mathsf{slive}(\Delta_l)$, so

$$(\star) \qquad \big(\mathsf{G}_l[\mathsf{G}'/\mathbf{t}],\ \Delta_l(\mathbf{p})\big) \;\in\; R_{\mathbf{p}} \;\subseteq\; \widehat{R}_{\mathbf{p}} \qquad \text{for each successor } \Delta_l.$$

We case-split on the form of the enabled action and the role of $\mathbf{p}$.

Sub-case (a): value-passing ($\mathsf{U} = \mathsf{B}$, *single continuation).* Then $\mathsf{G} = \mathbf{p_j} \to \mathbf{q}\ \langle \mathsf{B} \rangle; \mathsf{G}_l[\mathsf{G}'/\mathbf{t}]$. By Lemma B.2: $\mathsf{unfold}(\Delta(\mathbf{p_j})) = \mathbf{q}!\langle \mathsf{B} \rangle; \Delta_l(\mathbf{p_j})$, $\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p_j}?\langle \mathsf{B} \rangle; \Delta_l(\mathbf{q})$, and $\Delta_l(\mathbf{r}) = \Delta(\mathbf{r})$ for $\mathbf{r} \notin \{\mathbf{p_j}, \mathbf{q}\}$.

If $\mathbf{p} = \mathbf{p_j}$: $\mathsf{T} = \mathbf{q}!\langle \mathsf{B} \rangle; \Delta_l(\mathbf{p_j})$. [SPR1]; premise in $\widehat{R}_{\mathbf{p}}$ by $(\star)$. If $\mathbf{p} = \mathbf{q}$: $\mathsf{T} = \mathbf{p_j}?\langle \mathsf{B} \rangle; \Delta_l(\mathbf{q})$. [SPR2]; premise in $\widehat{R}_{\mathbf{p}}$ by $(\star)$.

If $\mathbf{p} \notin \{\mathbf{p_j}, \mathbf{q}\}$: $\Delta_l(\mathbf{p}) = \Delta(\mathbf{p})$. [SPR3]; premise $(\mathsf{G}_l[\mathsf{G}'/\mathbf{t}], \mathsf{T})$ lies in $\widehat{R}_{\mathbf{p}}$: if $\mathsf{T} = \Delta(\mathbf{p})$ by $(\star)$ directly; if $\mathsf{T} \neq \Delta(\mathbf{p})$ by repeated [RUnfR] on $(\star)$ (since $\Delta_l(\mathbf{p}) = \Delta(\mathbf{p})$, the same unfolding chain applies).

Sub-case (b): labelled branching ($L = I$). Then $\mathsf{G} = \mathbf{p_j} \to \mathbf{q}\ \{l_i : \mathsf{G}_i[\mathsf{G}'/\mathbf{t}]\}_{i \in I}$. By Lemma B.2: $\mathsf{unfold}(\Delta(\mathbf{p_j})) = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$ with $\Delta_i(\mathbf{p_j}) = \mathsf{T}_i$; $\mathsf{unfold}(\Delta(\mathbf{q})) = \mathbf{p_j} \,\&\, \{l_i : \mathsf{T}'_i\}_{i \in I_r}$ with $I \subseteq I_r$ and $\Delta_i(\mathbf{q}) = \mathsf{T}'_i$; and $\Delta_i(\mathbf{r}) = \Delta(\mathbf{r})$ for $\mathbf{r} \notin \{\mathbf{p_j}, \mathbf{q}\}$.

If $\mathbf{p} = \mathbf{p_j}$: $\mathsf{T} = \mathbf{q} \oplus \{l_i : \mathsf{T}_i\}_{i \in I}$. [SPR4] with $J = I$; each premise $(\mathsf{G}_i[\mathsf{G}'/\mathbf{t}], \mathsf{T}_i)$ lies in $\widehat{R}_{\mathbf{p}}$ by $(\star)$ (since $\mathsf{T}_i = \Delta_i(\mathbf{p})$).

If $\mathbf{p} = \mathbf{q}$: $\mathsf{T} = \mathbf{p_j} \,\&\, \{l_i : \mathsf{T}'_i\}_{i \in I_r}$. [SPR5] with $J = I \subseteq I_r$; each premise $(\mathsf{G}_i[\mathsf{G}'/\mathbf{t}], \mathsf{T}'_i)$ lies in $\widehat{R}_{\mathbf{p}}$ by $(\star)$ (since $\mathsf{T}'_i = \Delta_i(\mathbf{p})$).

If $\mathbf{p} \notin \{\mathbf{p_j}, \mathbf{q}\}$: $\Delta_i(\mathbf{p}) = \Delta(\mathbf{p})$ for every $i \in I$ (Lemma B.2(3)). [SPR6] applies; *every premise asks for* $(\mathsf{G}_i[\mathsf{G}'/\mathbf{t}], \mathsf{T})$ *with the same* T. By $(\star)$, $(\mathsf{G}_i[\mathsf{G}'/\mathbf{t}], \Delta_i(\mathbf{p})) \in R_{\mathbf{p}}$ for each i, and $\Delta_i(\mathbf{p}) = \Delta(\mathbf{p})$ is the same type across all branches. If $\mathsf{T} = \Delta(\mathbf{p})$, the premises are directly $(\star)$; if $\mathsf{T} \neq \Delta(\mathbf{p})$, apply repeated [RUnfR] on $(\star)$ (since $\Delta_i(\mathbf{p}) = \Delta(\mathbf{p})$, the same unfolding chain applies to every branch). □

B.2.6 *Association Proofs.*

Lemma B.17 (Merge Preserves Lower Bounds). *If* $\mathsf{T} \leqslant \mathsf{T}_i$ *for all* $i \in I$ *and* $\{\mathsf{T}_i\}_{i \in I} \sqcap \mathsf{T}'$, *then* $\mathsf{T} \leqslant \mathsf{T}'$.

Proof of Lemma B.17. By coinduction. Define the candidate relation $R = \{(\mathsf{T}, \mathsf{T}') \mid \exists \{\mathsf{T}_i\}_{i \in I}.\ \mathsf{T} \leqslant \mathsf{T}_i \text{ for all } i \in I \wedge \{\mathsf{T}_i\}_{i \in I} \sqcap \mathsf{T}'\}$ and show that R is consistent with the subtyping rules, i.e. for every $(\mathsf{T}, \mathsf{T}') \in R$ there is a subtyping rule whose conclusion matches $\mathsf{T} \leqslant \mathsf{T}'$ and whose premises lie in R. Take $(\mathsf{T}, \mathsf{T}') \in R$ with witnesses $\{\mathsf{T}_i\}_{i \in I}$ such that $\mathsf{T} \leqslant \mathsf{T}_i$ for all i and $\{\mathsf{T}_i\}_{i \in I} \sqcap \mathsf{T}'$. We case-split on the merge rule used.

Case [M1]. $\mathsf{T}_i = \mathsf{end}$ for all i and $\mathsf{T}' = \mathsf{end}$. By Lemma B.7, $\mathsf{unfold}(\mathsf{T}) = \mathsf{end}$. Rule [S7] gives $\mathsf{T} \leqslant \mathsf{end}$.

Case [M2]. All $\mathsf{T}_i = \mathbf{p}!\langle \mathsf{B} \rangle; \mathsf{T}_{0,i}$ and $\mathsf{T}' = \mathbf{p}!\langle \mathsf{B} \rangle; \mathsf{T}''$ where $\{\mathsf{T}_{0,i}\}_{i \in I} \sqcap \mathsf{T}''$. By Lemma B.7, $\mathsf{unfold}(\mathsf{T}) = \mathbf{p}!\langle \mathsf{B} \rangle; \mathsf{T}_0$ with $\mathsf{T}_0 \leqslant \mathsf{T}_{0,i}$ for all i. Hence $(\mathsf{T}_0, \mathsf{T}'') \in R$. Rule [S1] (with [S5] if T is a μ-type).

Case [M3]. Symmetric to [M2], using [S2].

Case [M4]. All $\mathsf{T}_i = \mathbf{p} \oplus \{l_k : \mathsf{T}_{k,i}\}_{k \in K}$ (same K) and $\mathsf{T}' = \mathbf{p} \oplus \{l_k : \mathsf{T}''_k\}_{k \in K}$ where $\{\mathsf{T}_{k,i}\}_{i \in I} \sqcap \mathsf{T}''_k$ for each k. By Lemma B.7, $\mathsf{unfold}(\mathsf{T}) = \mathbf{p} \oplus \{l_k : \mathsf{T}_k\}_{k \in K'}$ with $K' \subseteq K$ and $\mathsf{T}_k \leqslant \mathsf{T}_{k,i}$ for all $k \in K'$ and all i. Hence $(\mathsf{T}_k, \mathsf{T}''_k) \in R$ for each $k \in K'$. Rule [S3] with $K' \subseteq K$.

Case [M5]. $\mathtt{T}_j = \mathbf{p}\,\&\,\{l_k : \mathtt{T}_{k,j}\}_{k\in K_j}$ and $\mathtt{T}' = \mathbf{p}\,\&\,\{l_k : \mathtt{T}''_k\}_{k\in\bigcup_j K_j}$ where $\{\mathtt{T}_{k,j}\}_{j\in J\mid k\in K_j} \sqcap \mathtt{T}''_k$ for each k. By Lemma B.7, $\mathrm{unfold}(\mathtt{T}) = \mathbf{p}\,\&\,\{l_k : \mathtt{T}_k\}_{k\in K_T}$ with $K_j \subseteq K_T$ for all j, hence $\bigcup_j K_j \subseteq K_T$, and $\mathtt{T}_k \leqslant \mathtt{T}_{k,j}$ for all j with $k \in K_j$. Hence $(\mathtt{T}_k, \mathtt{T}''_k) \in R$ for each $k \in \bigcup_j K_j$. Rule [S4] with $\bigcup_j K_j \subseteq K_T$.

Case [M6]. Some $\mathtt{T}_j = \mu\mathbf{t}.\mathtt{T}_0$ is unfolded to $\mathtt{T}_0[\mu\mathbf{t}.\mathtt{T}_0/\mathbf{t}]$ in the premise. By Lemma B.6, $\mathtt{T} \leqslant \mathtt{T}_j \leqslant \mathtt{T}_0[\mu\mathbf{t}.\mathtt{T}_0/\mathbf{t}]$. Hence the premise of the merge still has $\mathtt{T}$ as a lower bound of each input, and $(\mathtt{T}, \mathtt{T}') \in R$ via the inner derivation.

Case [M7]. $\{\mathtt{T}_i\}_{i\in I} \sqcap \mathtt{T}'[\mu\mathbf{t}.\mathtt{T}'/\mathbf{t}]$ and $\mathtt{T}' = \mu\mathbf{t}.\mathtt{T}''$ for some $\mathtt{T}''$. Then $(\mathtt{T}, \mathtt{T}'[\mu\mathbf{t}.\mathtt{T}'/\mathbf{t}]) \in R$, so by the coinductive hypothesis $\mathtt{T} \leqslant \mathtt{T}'[\mu\mathbf{t}.\mathtt{T}'/\mathbf{t}]$. Rule [S6] gives $\mathtt{T} \leqslant \mu\mathbf{t}.\mathtt{T}'' = \mathtt{T}'$. □

Lemma 6.9 (Pointwise Composite Relation Implies Association). [Proof in Appendix B.2] *If* $\mathtt{G} \Vdash \Delta$, *then* $\Delta \sqsubseteq \mathtt{G}$.

Proof of Lemma 6.9. By coinduction. Given $\mathtt{G} \Vdash_{\mathbf{p}} \mathtt{T}$, we construct $\mathtt{T}'$ and show $\mathtt{G} \upharpoonright_{\mathbf{p}} \mathtt{T}'$ and $\mathtt{T} \leqslant \mathtt{T}'$, by case-splitting on the rule of $\Vdash_{\mathbf{p}}$ and appealing to the coinductive hypothesis for the premises.

Cases [SPR1]/[SPR2]. $\mathtt{G} = \mathbf{p} \to \mathbf{q}\ \langle \mathtt{B}\rangle;\mathtt{G}_0$ and $\mathtt{T} = \mathbf{q}!\langle \mathtt{B}\rangle;\mathtt{T}_0$ (resp. $\mathbf{q}?\langle \mathtt{B}\rangle;\mathtt{T}_0$), with $\mathtt{G}_0 \Vdash_{\mathbf{p}} \mathtt{T}_0$. By coinductive hypothesis, there exists $\mathtt{T}'_0$ with $\mathtt{G}_0 \upharpoonright_{\mathbf{p}} \mathtt{T}'_0$ and $\mathtt{T}_0 \leqslant \mathtt{T}'_0$. Set $\mathtt{T}' = \mathbf{q}!\langle \mathtt{B}\rangle;\mathtt{T}'_0$ (resp. $\mathbf{q}?\langle \mathtt{B}\rangle;\mathtt{T}'_0$). Rule [PR1] (resp. [PR2]) for projection; rule [S1] (resp. [S2]) for subtyping.

Case [SPR3]. $\mathtt{G} = \mathbf{q} \to \mathbf{r}\ \langle \mathtt{B}\rangle;\mathtt{G}_0$, $\mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}$, $\mathtt{G}_0 \Vdash_{\mathbf{p}} \mathtt{T}$. By c.h., $\exists \mathtt{T}'$: $\mathtt{G}_0 \upharpoonright_{\mathbf{p}} \mathtt{T}'$ and $\mathtt{T} \leqslant \mathtt{T}'$. Rule [PR3] gives $\mathtt{G} \upharpoonright_{\mathbf{p}} \mathtt{T}'$. Subtyping inherited from the premise.

Case [SPR4]. $\mathtt{G} = \mathbf{p} \to \mathbf{q}\ \{l_i : \mathtt{G}_i\}_{i\in I}$, $\mathtt{T} = \mathbf{q} \oplus \{l_i : \mathtt{T}_i\}_{i\in I}$, with $\mathtt{G}_i \Vdash_{\mathbf{p}} \mathtt{T}_i$ for each i. By c.h., for each i: $\mathtt{G}_i \upharpoonright_{\mathbf{p}} \mathtt{T}'_i$ and $\mathtt{T}_i \leqslant \mathtt{T}'_i$. Set $\mathtt{T}' = \mathbf{q} \oplus \{l_i : \mathtt{T}'_i\}_{i\in I}$. Rule [PR4]; rule [S3] with $I \subseteq I$.

Case [SPR5]. $\mathtt{G} = \mathbf{q} \to \mathbf{p}\ \{l_j : \mathtt{G}_j\}_{j\in J}$, $\mathtt{T} = \mathbf{q}\,\&\,\{l_i : \mathtt{T}_i\}_{i\in I}$ with $J \subseteq I$, and $\mathtt{G}_j \Vdash_{\mathbf{p}} \mathtt{T}_j$ for each $j \in J$. By c.h., for each $j \in J$: $\mathtt{G}_j \upharpoonright_{\mathbf{p}} \mathtt{T}'_j$ and $\mathtt{T}_j \leqslant \mathtt{T}'_j$. Set $\mathtt{T}' = \mathbf{q}\,\&\,\{l_j : \mathtt{T}'_j\}_{j\in J}$. Rule [PR5]; rule [S4] with $J \subseteq I$.

Case [SPR6]. $\mathtt{G} = \mathbf{q} \to \mathbf{r}\ \{l_i : \mathtt{G}_i\}_{i\in I}$, $\mathbf{p} \notin \{\mathbf{q}, \mathbf{r}\}$, $\mathtt{G}_i \Vdash_{\mathbf{p}} \mathtt{T}$ for all i (same $\mathtt{T}$). By c.h., for each i: $\mathtt{G}_i \upharpoonright_{\mathbf{p}} \mathtt{T}'_i$ and $\mathtt{T} \leqslant \mathtt{T}'_i$. The types $\{\mathtt{T}'_i\}_{i\in I}$ are mergeable: since $\mathtt{T} \leqslant \mathtt{T}'_i$ for each i, inversion of subtyping (Lemma B.7) ensures the $\mathtt{T}'_i$ have compatible outermost form (up to unfolding), and a coinductive argument on the merge rules verifies that the merge exists. Let $\{\mathtt{T}'_i\}_{i\in I} \sqcap \mathtt{T}'$. Rule [PR6] for projection. By Lemma B.17, $\mathtt{T} \leqslant \mathtt{T}'$.

Case [SPR9]. $\mathtt{G} = \mu\mathbf{t}.\mathtt{G}_0$, $\mathbf{p} \in \mathrm{pt}(\mathtt{G}_0)$, $\mathtt{G}_0[\mu\mathbf{t}.\mathtt{G}_0/\mathbf{t}] \Vdash_{\mathbf{p}} \mathtt{T}$. By c.h., $\exists \mathtt{T}'$: $\mathtt{G}_0[\mu\mathbf{t}.\mathtt{G}_0/\mathbf{t}] \upharpoonright_{\mathbf{p}} \mathtt{T}'$ and $\mathtt{T} \leqslant \mathtt{T}'$. Rule [PR9] gives $\mu\mathbf{t}.\mathtt{G}_0 \upharpoonright_{\mathbf{p}} \mathtt{T}'$. Subtyping inherited.

Case [SPR10]. $\mathtt{T} = \mu\mathbf{t}.\mathtt{T}_0$, with $\mathtt{G} \Vdash_{\mathbf{p}} \mathtt{T}_0[\mu\mathbf{t}.\mathtt{T}_0/\mathbf{t}]$. By c.h., $\exists \mathtt{T}'$: $\mathtt{G} \upharpoonright_{\mathbf{p}} \mathtt{T}'$ and $\mathtt{T}_0[\mu\mathbf{t}.\mathtt{T}_0/\mathbf{t}] \leqslant \mathtt{T}'$. Projection: $\mathtt{G} \upharpoonright_{\mathbf{p}} \mathtt{T}'$ directly. By Lemma B.6, $\mu\mathbf{t}.\mathtt{T}_0 \leqslant \mathtt{T}_0[\mu\mathbf{t}.\mathtt{T}_0/\mathbf{t}] \leqslant \mathtt{T}'$.

Cases [SPR11]/[SPR12]. $\mathtt{T} = \mathtt{end}$. Set $\mathtt{T}' = \mathtt{end}$. Rules [PR11]/[PR12] and rule [S7].

Collecting: for each $\mathbf{p} \in \mathrm{pt}(\mathtt{G})$, the construction yields $\mathtt{T}'_{\mathbf{p}}$ with $\mathtt{G} \upharpoonright_{\mathbf{p}} \mathtt{T}'_{\mathbf{p}}$ and $\Delta(\mathbf{p}) \leqslant \mathtt{T}'_{\mathbf{p}}$. Define Δ' by $\Delta'(\mathbf{p}) = \mathtt{T}'_{\mathbf{p}}$. Then $\mathrm{dom}(\Delta') = \mathrm{pt}(\mathtt{G})$, $\mathtt{G} \upharpoonright_{\mathbf{p}} \Delta'(\mathbf{p})$ for each $\mathbf{p}$, and $\Delta \leqslant \Delta'$ (componentwise by rule [S8]). By Definition 6.8, $\Delta \sqsubseteq \mathtt{G}$. □

B.2.7 *Balancedness.*

Lemma 6.11 (Synthesised Global Type is Balanced). [Proof in Appendix B.2] *For any context* Δ *satisfying* $\mathrm{slive}(\Delta)$ *and any priority* $\mathbf{p_i}$, *let* $\mathtt{G} = \mathit{synth}(\varnothing, \mathbf{p_i}, \Delta)$. *Then* $\mathtt{G}$ *is balanced.*

Proof of Lemma 6.11. We show that for every subterm $\mathtt{G}' \in \mathrm{sub}(\mathtt{G})$, $\mathrm{pt}(\mathtt{G}') \subseteq \mathrm{unav}(\mathtt{G}')$ (the reverse inclusion is immediate from the definition). Equivalently, we show that every $\mathbf{p} \in \mathrm{pt}(\mathtt{G}')$ appears on every path through the unfolded tree of $\mathtt{G}'$.

Each subterm $\mathtt{G}'$ of $\mathtt{G}$ corresponds to a synthesis call $\mathrm{synth}(\Sigma, \mathbf{p_k}, \Delta')$ for some reachable context Δ' (i.e. $\Delta \longrightarrow^* \Delta'$). By repeated application of Lemma 6.6, $\mathrm{slive}(\Delta')$.

Consider any such subterm $\mathtt{G}'$ with corresponding context Δ'. Let $\mathbf{p} \in \mathrm{pt}(\mathtt{G}')$ and consider any path π through the unfolded tree of $\mathtt{G}'$. We must show $\mathbf{p}$ appears on π.

The path π fixes a specific label at each branching node. Together with the round-robin schedule, this determines a maximal round-robin reduction sequence from Δ'. By Lemma B.4, this projects onto a *fair* reduction sequence from Δ'. Since $\mathrm{live}(\Delta')$, this fair sequence is also *live* (Definition 5.8).

We claim that every participant $\mathbf{p}$ with $\Delta'(\mathbf{p}) \neq \mathsf{end}$ communicates on this path. Since $\Delta'(\mathbf{p}) \neq \mathsf{end}$, participant $\mathbf{p}$ has a one-sided action (send or receive) at Δ'. By liveness (Definition 5.7), the matching partner eventually has the corresponding complementary action, so the two-sided interaction involving $\mathbf{p}$ eventually fires. After $\mathbf{p}$ communicates, its type may change; if $\mathbf{p}$ is still active, liveness again guarantees eventual communication, and so on until $\mathbf{p}$ reaches end.

It remains to connect $\mathbf{p} \in \mathrm{pt}(G')$ with $\mathbf{p}$ being active. If $\mathbf{p} \in \mathrm{pt}(G')$, then $\mathbf{p}$ appears as sender or receiver at some node of G'. At the root of G', the sender and receiver $\mathbf{p_j}, \mathbf{q}$ are active in Δ' (since the synthesis only fires enabled transitions). For $\mathbf{p}$ appearing deeper in G': $\mathbf{p}$ is active in some context reachable from Δ' along the path π. By the argument above, every active participant communicates on every fair, live path from its context. In particular, $\mathbf{p}$ appears on π.

Since $\mathbf{p}$ appears on every path through G', we have $\mathbf{p} \in \mathrm{unav}(G')$. As this holds for all $\mathbf{p} \in \mathrm{pt}(G')$ and all $G' \in \mathrm{sub}(G)$, the type G is balanced. □

B.2.8 *Main Theorems.*

Lemma 6.15 (Soundness of the Combined Relation). [Proof in Appendix B.2] *If* $G \Vdash \Delta$ *and* $G \xrightarrow{\ell} G'$, *then there exists* Δ' *such that* $\Delta \xrightarrow{\ell} \Delta'$ *and* $G' \Vdash \Delta'$.

Proof of Lemma 6.15. Assume $G \Vdash \Delta$ and $G \xrightarrow{\mathbf{pq}:\langle U \rangle} G'$. We construct Δ' with $\Delta \xrightarrow{\mathbf{pq}:\langle U \rangle} \Delta'$ and $G' \Vdash \Delta'$. The proof is by induction on the derivation of $G \xrightarrow{\mathbf{pq}:\langle U \rangle} G'$. By Lemma B.8, one of four cases holds.

Cases 1 and 2 (head interaction fires).

Case 1: $U = B$, $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \langle B \rangle; G_0$, and $G' = G_0$.

For the sender $\mathbf{p}$: from $G \Vdash_{\mathbf{p}} \Delta(\mathbf{p})$, Lemma B.9(1) gives $\mathrm{unfold}(\Delta(\mathbf{p})) = \mathbf{q}!\langle B \rangle; T_{\mathbf{p}}$ with $G_0 \Vdash_{\mathbf{p}} T_{\mathbf{p}}$. Hence $\Delta(\mathbf{p}) \xrightarrow{\mathbf{q}!\langle B \rangle} T_{\mathbf{p}}$.

For the receiver $\mathbf{q}$: from $G \Vdash_{\mathbf{q}} \Delta(\mathbf{q})$, Lemma B.9(2) gives $\mathrm{unfold}(\Delta(\mathbf{q})) = \mathbf{p}?\langle B \rangle; T_{\mathbf{q}}$ with $G_0 \Vdash_{\mathbf{q}} T_{\mathbf{q}}$. Hence $\Delta(\mathbf{q}) \xrightarrow{\mathbf{p}?\langle B \rangle} T_{\mathbf{q}}$.

Combining via rules [LTag] and [LEnv], $\Delta \xrightarrow{\mathbf{pq}:\langle B \rangle} \Delta'$ where $\Delta'(\mathbf{p}) = T_{\mathbf{p}}$, $\Delta'(\mathbf{q}) = T_{\mathbf{q}}$, and $\Delta'(\mathbf{r}) = \Delta(\mathbf{r})$ for $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$.

For each uninvolved $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$: Lemma B.9(5) gives $G_0 \Vdash_{\mathbf{r}} \Delta(\mathbf{r}) = \Delta'(\mathbf{r})$. Hence $G' \Vdash \Delta'$.

Case 2: $U = l_j$, $\mathrm{unfold}(G) = \mathbf{p} \to \mathbf{q}\ \{l_i : G_i\}_{i \in I}$, $j \in I$, $G' = G_j$.

For the sender $\mathbf{p}$: Lemma B.9(3) gives $\mathrm{unfold}(\Delta(\mathbf{p})) = \mathbf{q} \oplus \{l_i : T_i\}_{i \in I}$ with $G_i \Vdash_{\mathbf{p}} T_i$ for all $i \in I$. Since $j \in I$: $\Delta(\mathbf{p}) \xrightarrow{\mathbf{q}!\langle l_j \rangle} T_j$ and $G_j \Vdash_{\mathbf{p}} T_j$.

For the receiver $\mathbf{q}$: Lemma B.9(4) gives $J' \supseteq I$ with $\mathrm{unfold}(\Delta(\mathbf{q})) = \mathbf{p} \,\&\, \{l_i : T'_i\}_{i \in J'}$ and $G_j \Vdash_{\mathbf{q}} T'_j$ (since $j \in I \subseteq J'$). Hence $\Delta(\mathbf{q}) \xrightarrow{\mathbf{p}?\langle l_j \rangle} T'_j$.

Combining via [LTag] and [LEnv], $\Delta \xrightarrow{\mathbf{pq}:\langle l_j \rangle} \Delta'$ where $\Delta'(\mathbf{p}) = T_j$, $\Delta'(\mathbf{q}) = T'_j$, $\Delta'(\mathbf{r}) = \Delta(\mathbf{r})$ for $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$.

For uninvolved $\mathbf{r}$: Lemma B.9(6) gives $G_i \Vdash_{\mathbf{r}} \Delta(\mathbf{r})$ for all i; in particular $G_j \Vdash_{\mathbf{r}} \Delta(\mathbf{r}) = \Delta'(\mathbf{r})$. Hence $G' \Vdash \Delta'$.

Case 3 (base-type prefix pass-through). $\mathrm{unfold}(G) = \mathbf{r} \to \mathbf{s}\ \langle B \rangle; G_0$ with $\{\mathbf{p}, \mathbf{q}\} \cap \{\mathbf{r}, \mathbf{s}\} = \emptyset$, $G_0 \xrightarrow{\mathbf{pq}:\langle U \rangle} G'_0$, and $G' = \mathbf{r} \to \mathbf{s}\ \langle B \rangle; G'_0$.

For each participant $\mathbf{t} \in \mathrm{dom}(\Delta)$, apply Lemma B.9 to $G \Vdash_{\mathbf{t}} \Delta(\mathbf{t})$:

- $\mathbf{t} = \mathbf{r}$: case (1) gives $\mathrm{unfold}(\Delta(\mathbf{r})) = \mathbf{s}!\langle \mathrm{B}\rangle;\mathrm{T}_{\mathbf{r}}$ with $\mathrm{G}_0 \Vdash_{\mathbf{r}} \mathrm{T}_{\mathbf{r}}$.
- $\mathbf{t} = \mathbf{s}$: case (2) gives $\mathrm{unfold}(\Delta(\mathbf{s})) = \mathbf{r}?\langle \mathrm{B}\rangle;\mathrm{T}_{\mathbf{s}}$ with $\mathrm{G}_0 \Vdash_{\mathbf{s}} \mathrm{T}_{\mathbf{s}}$.
- $\mathbf{t} \notin \{\mathbf{r}, \mathbf{s}\}$: case (5) gives $\mathrm{G}_0 \Vdash_{\mathbf{t}} \Delta(\mathbf{t})$.

Define Δ_0 by $\Delta_0(\mathbf{r}) = \mathrm{T}_{\mathbf{r}}$, $\Delta_0(\mathbf{s}) = \mathrm{T}_{\mathbf{s}}$, and $\Delta_0(\mathbf{t}) = \Delta(\mathbf{t})$ for $\mathbf{t} \notin \{\mathbf{r}, \mathbf{s}\}$. Then $\mathrm{G}_0 \Vdash \Delta_0$. Since $\{\mathbf{p}, \mathbf{q}\} \cap \{\mathbf{r}, \mathbf{s}\} = \emptyset$, we have $\Delta_0(\mathbf{p}) = \Delta(\mathbf{p})$ and $\Delta_0(\mathbf{q}) = \Delta(\mathbf{q})$.

By the induction hypothesis (the derivation of $\mathrm{G}_0 \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \mathrm{G}'_0$ is strictly smaller), there exists Δ'_0 with $\Delta_0 \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \Delta'_0$ and $\mathrm{G}'_0 \Vdash \Delta'_0$.

The step $\Delta_0 \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \Delta'_0$ only modifies $\mathbf{p}$ and $\mathbf{q}$. Since $\Delta_0(\mathbf{p}) = \Delta(\mathbf{p})$ and $\Delta_0(\mathbf{q}) = \Delta(\mathbf{q})$, the same one-sided transitions exist in Δ, giving $\Delta \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \Delta'$ where $\Delta'(\mathbf{p}) = \Delta'_0(\mathbf{p})$, $\Delta'(\mathbf{q}) = \Delta'_0(\mathbf{q})$, and $\Delta'(\mathbf{t}) = \Delta(\mathbf{t})$ for $\mathbf{t} \notin \{\mathbf{p}, \mathbf{q}\}$.

It remains to show $\mathrm{G}' \Vdash \Delta'$ where $\mathrm{G}' = \mathbf{r} \twoheadrightarrow \mathbf{s}\ \langle \mathrm{B}\rangle;\mathrm{G}'_0$.

- For $\mathbf{t} \notin \{\mathbf{r}, \mathbf{s}\}$: by [SPR3], $\mathrm{G}'_0 \Vdash_{\mathbf{t}} \Delta'(\mathbf{t})$ implies $\mathrm{G}' \Vdash_{\mathbf{t}} \Delta'(\mathbf{t})$. From $\mathrm{G}'_0 \Vdash \Delta'_0$: if $\mathbf{t} \in \{\mathbf{p}, \mathbf{q}\}$, then $\mathrm{G}'_0 \Vdash_{\mathbf{t}} \Delta'_0(\mathbf{t}) = \Delta'(\mathbf{t})$; if $\mathbf{t} \notin \{\mathbf{p}, \mathbf{q}\}$, then $\Delta'(\mathbf{t}) = \Delta(\mathbf{t}) = \Delta_0(\mathbf{t}) = \Delta'_0(\mathbf{t})$, and $\mathrm{G}'_0 \Vdash_{\mathbf{t}} \Delta'_0(\mathbf{t}) = \Delta'(\mathbf{t})$.
- For $\mathbf{t} = \mathbf{r}$: since $\mathbf{r} \notin \{\mathbf{p}, \mathbf{q}\}$, we have $\Delta'(\mathbf{r}) = \Delta(\mathbf{r})$ and $\Delta'_0(\mathbf{r}) = \Delta_0(\mathbf{r}) = \mathrm{T}_{\mathbf{r}}$. From $\mathrm{G}'_0 \Vdash \Delta'_0$: $\mathrm{G}'_0 \Vdash_{\mathbf{r}} \mathrm{T}_{\mathbf{r}}$. By [SPR1], $\mathbf{r} \twoheadrightarrow \mathbf{s}\ \langle \mathrm{B}\rangle;\mathrm{G}'_0 \Vdash_{\mathbf{r}} \mathbf{s}!\langle \mathrm{B}\rangle;\mathrm{T}_{\mathbf{r}}$. Since $\mathrm{unfold}(\Delta(\mathbf{r})) = \mathbf{s}!\langle \mathrm{B}\rangle;\mathrm{T}_{\mathbf{r}}$, by closure under [SPR10]: $\mathrm{G}' \Vdash_{\mathbf{r}} \Delta(\mathbf{r}) = \Delta'(\mathbf{r})$.
- For $\mathbf{t} = \mathbf{s}$: symmetric, using [SPR2] and [SPR10].

Case 4 (branching prefix pass-through). $\mathrm{unfold}(\mathrm{G}) = \mathbf{r} \twoheadrightarrow \mathbf{s}\ \{l_i : \mathrm{G}_i\}_{i \in I}$ with $\{\mathbf{p}, \mathbf{q}\} \cap \{\mathbf{r}, \mathbf{s}\} = \emptyset$, $\mathrm{G}_i \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \mathrm{G}'_i$ for all i, and $\mathrm{G}' = \mathbf{r} \twoheadrightarrow \mathbf{s}\ \{l_i : \mathrm{G}'_i\}_{i \in I}$. The argument is analogous to Case 3, using Lemma B.9 cases (3), (4), and (6) instead of (1), (2), and (5), and rules [SPR4]/[SPR5]/[SPR6] instead of [SPR1]/[SPR2]/[SPR3]. The induction hypothesis is applied to each sub-derivation $\mathrm{G}_i \xrightarrow{\mathbf{pq}:\langle \mathrm{U}\rangle} \mathrm{G}'_i$, yielding compatible context steps that agree on $\mathbf{p}$ and $\mathbf{q}$ (since the combined relation determines the local types of $\mathbf{p}$ and $\mathbf{q}$ independently of the branch index i). □

C FURTHER DISCUSSIONS ON "A SYNTHETIC RECONSTRUCTION OF MULTIPARTY SESSION TYPES"

This section reports potential technical issues on [22]. They claim in **Theorem 5.9**, *any global type* G *satisfies the well-behaved conditions in Definition 5.3.* However there exists a global type which does not satisfy "Conditional Commutativity" in Definition 5.3, the key definition for stating Theorem 5.9. Consider the following global type:

$$\mathrm{G} \quad = \quad \mathbf{p} \twoheadrightarrow \mathbf{q}\ \big\{l\!:\mathbf{a} \twoheadrightarrow \mathbf{b}\ \{l_1\!:\mathrm{end},\ l_2\!:\mathrm{end}\},\ \mathbf{a} \twoheadrightarrow \mathbf{b}\ \{l_1\!:\mathrm{end}\}\big\}$$

where $\mathbf{a}, \mathbf{b}, \mathbf{p}, \mathbf{q}$ are pairwise distinct.

Then $\mathrm{G} \xrightarrow{\mathbf{ab}:\langle l_1\rangle} \mathrm{G}'$ and $\mathrm{G} \xrightarrow{\mathbf{pq}:\langle l\rangle} \mathrm{G}_1 \xrightarrow{\mathbf{ab}:\langle l_2\rangle} \mathrm{G}_2$ but $\neg(\mathrm{G} \xrightarrow{\mathbf{ab}:\langle l_2\rangle})$.

Hence, Conditional Commutativity fails. Consequently, we believe that Theorem 5.9 and Corollary 5.10 do not hold. Although none of our results depends on their results, we report this issue to avoid confusion for reviewers. In particular, the authors [22] use the term "liveness" to refer to deadlock-freedom in [99], and the authors appear to be unaware of the projections in [8, 112].

D BENCHMARK GLOBAL TYPE TABLE

Rows (d) and (e) show the MapReduce and Independent Workers families schematically; rows (m)–(t) reproduce the remaining uncopied Haskell catalogue entries, including the concrete Coinductive Full instances. Entries are labelled (a)–(t) for reference from Tables 1 and 2.

Table 3. Benchmark global type table used in the evaluation.

(a) Odd-Even [77, Ex. 2.1]

$$\mathsf{G}_{oe} = \mathbf{p} \to \mathbf{q} \left\{ \begin{array}{l} o : \mathbf{q} \to \mathbf{r} \, \{ o : \mu \mathbf{t}_1 . \, \mathbf{p} \to \mathbf{q} \left\{ \begin{array}{l} o : \mathbf{q} \to \mathbf{r} \, \{ o : \mathbf{q} \to \mathbf{r} \, \{ o : \mathbf{t}_1 \} \} \\ b : \mathbf{q} \to \mathbf{r} \, \{ b : \mathbf{r} \to \mathbf{p} \, \{ o : \mathsf{end} \} \} \} \end{array} \right. \\ m : \mu \mathbf{t}_2 . \, \mathbf{p} \to \mathbf{q} \left\{ \begin{array}{l} o : \mathbf{q} \to \mathbf{r} \, \{ o : \mathbf{q} \to \mathbf{r} \, \{ o : \mathbf{t}_2 \} \} \\ b : \mathbf{q} \to \mathbf{r} \, \{ b : \mathbf{r} \to \mathbf{p} \, \{ m : \mathsf{end} \} \} \end{array} \right. \end{array} \right.$$

Three participants **p**, **q**, **r**. The outer choice (o/m) selects between two recursive modes. In each mode, **q** relays to **r** and the protocol either continues (o) or terminates (b). Requires coinductive full-merging projection.

(b) OAuth [99, Ex. 1]

$$\mathsf{G}_{oa} = \mathbf{s} \to \mathbf{c} \left\{ \begin{array}{l} \mathit{login} : \mathbf{c} \to \mathbf{a} \, \{ \mathit{password} : \\ \quad \mathbf{c} \to \mathbf{a} \, \langle \mathsf{string} \rangle ; \mathbf{a} \to \mathbf{s} \, \{ \mathit{auth} : \\ \quad\quad \mathbf{a} \to \mathbf{s} \, \langle \mathsf{bool} \rangle ; \mathsf{end} \} \} \\ \mathit{auth} : \mathbf{c} \to \mathbf{a} \, \{ \mathit{quit} : \mathsf{end} \} \end{array} \right.$$

Three participants: **s** (server), **c** (client), **a** (auth. provider). Server offers login or direct auth. On login, client sends password to auth provider, who replies with a boolean to the server.

(c) Recursive Two-Buyer [99, Ex. 2]

$$\mathsf{G}_{tb} = \mathbf{a} \to \mathbf{s} \, \{ \mathit{query} : \mathbf{a} \to \mathbf{s} \, \langle \mathsf{string} \rangle ; \; \mathbf{s} \to \mathbf{a} \, \{ \mathit{price} : \mathbf{s} \to \mathbf{a} \, \langle \mathsf{int} \rangle ; \; \mu \mathbf{t} . \, \mathbf{a} \to \mathbf{b} \left\{ \begin{array}{l} \mathit{split} : \mathbf{b} \to \mathbf{a} \left\{ \begin{array}{l} \mathit{yes} : \mathbf{a} \to \mathbf{s} \, \{ \mathit{buy} : \mathsf{end} \} \\ \mathit{no} : \mathbf{t} \end{array} \right. \\ \mathit{cancel} : \mathbf{a} \to \mathbf{s} \, \{ \mathit{no} : \mathsf{end} \} \end{array} \right. \} \}$$

Three participants: **a** (buyer A), **b** (buyer B), **s** (seller). A queries the seller, who quotes a price; A asks B to split. B accepts or declines (recurse). A may also cancel outright. Not balanced, but projectable.

(d) MapReduce [99, Ex. 3]

$$\mathsf{G}_{mr} = \mu \mathbf{t} . \, \mathbf{m} \to \mathbf{w}_1 \, \{ \mathit{datum} : \mathbf{m} \to \mathbf{w}_1 \, \langle \mathsf{int} \rangle ; \cdots \; \mathbf{m} \to \mathbf{w}_n \, \{ \mathit{datum} : \mathbf{m} \to \mathbf{w}_n \, \langle \mathsf{int} \rangle ; \; \mathbf{w}_1 \to \mathbf{r} \, \langle \mathsf{int} \rangle ; \cdots \; \mathbf{w}_n \to \mathbf{r} \, \langle \mathsf{int} \rangle ; \; \mathbf{r} \to \mathbf{m} \left\{ \begin{array}{l} \mathit{continue} : \mathbf{r} \to \mathbf{m} \, \langle \mathsf{int} \rangle ; \mathbf{t} \\ \mathit{stop} : \mathbf{m} \to \mathbf{w}_1 \, \{ \mathit{stop} : \cdots \\ \quad \mathbf{m} \to \mathbf{w}_n \, \{ \mathit{stop} : \mathsf{end} \} \cdots \} \} \cdots \} \end{array} \right.$$

n+2 participants: **m** (manager), $\mathbf{w}_1, \ldots, \mathbf{w}_n$ (workers), **r** (reducer). Manager dispatches data, workers return results, reducer continues or stops. Running example (§2); the concrete catalogue instances are `MapReduce(5)`, `MapReduce(6)`, and `MapReduce(7)`.

(e) Independent Workers [99, Ex. 4]

Continued on next page

Benchmark Global Type Table (continued)

(f) Semantic Recursion [107]

$G_{itp} = \mu t.\ \mathbf{a} \to \mathbf{b}\ \langle \mathtt{string} \rangle;$

$$\mu t'.\ \mathbf{c} \to \mathbf{d} \begin{Bmatrix} \mathit{left}: \mathbf{t} \\ \mathit{right}: \mathbf{a} \to \mathbf{b}\ \langle \mathtt{string} \rangle; \mathbf{t}' \end{Bmatrix}$$

Four participants: **a**, **b**, **c**, **d**. Two nested recursions: the inner loop (**t′**) between **c** and **d** interleaves with sends from **a** to **b**. Choosing *left* restarts the outer loop. Requires coinductive projection.

(g) Simple Travel Agency [117, Fig. 1(a)]

$G_{sta} = \mathbf{c} \to \mathbf{a}\ \langle \mathtt{string} \rangle; \mathbf{a} \to \mathbf{c}\ \langle \mathtt{int} \rangle;$

$$\mathbf{c} \to \mathbf{a} \begin{Bmatrix} \mathit{accept}: \mathbf{c} \to \mathbf{a}\ \langle \mathtt{string} \rangle; \mathbf{a} \to \mathbf{c}\ \langle \mathtt{int} \rangle; \mathsf{end} \\ \mathit{reject}: \mathsf{end} \end{Bmatrix}$$

Two participants: **c** (client) and **a** (agency). The client requests a quote and receives a reply, then either accepts with booking details or rejects.

(h) Better Travel Agency [117, Fig. 1(b)]

$G_{bta} = \mu t.\ \mathbf{c} \to \mathbf{a}\ \langle \mathtt{string} \rangle; \mathbf{a} \to \mathbf{c}\ \langle \mathtt{int} \rangle;$

$$\mathbf{c} \to \mathbf{a} \begin{Bmatrix} \mathit{accept}: \mathbf{c} \to \mathbf{a}\ \langle \mathtt{string} \rangle; \mathbf{a} \to \mathbf{c}\ \langle \mathtt{int} \rangle; \mathsf{end} \\ \mathit{retry}: \mathbf{t} \\ \mathit{reject}: \mathsf{end} \end{Bmatrix}$$

The recursive variant of (g). After the initial request and quote, the client may accept, reject, or retry and return to the start of the protocol.

(i) Inductive Plain [112, Ex. 4.8]

$$G_{ip} = \mu t.\ \mathbf{q} \to \mathbf{r} \begin{Bmatrix} l_1: \mathbf{r} \to \mathbf{p}\ \{l_1: \mathbf{t}\} \\ l_2: \mathbf{r} \to \mathbf{p}\ \{l_1: \mathbf{t}\} \end{Bmatrix}$$

Three participants: **q** chooses and **r** relays to **p**. Both branches present the same label to **p**, so plain merge already succeeds.

(j) Inductive Full [112, Ex. 4.8]

$$G_{if} = \mu t.\ \mathbf{q} \to \mathbf{r} \begin{Bmatrix} l_1: \mathbf{r} \to \mathbf{p}\ \{l_1: \mathbf{t}\} \\ l_2: \mathbf{r} \to \mathbf{p}\ \{l_2: \mathsf{end}\} \end{Bmatrix}$$

The same overall shape as (i), but the second branch relays a different label to **p**. This is the smallest benchmark in the catalogue where full merge matters.

(k) Ring [22]

$G_{iw} = \mathbf{s} \to \mathbf{wa_1}\ \{\mathit{datum}: \mathbf{s} \to \mathbf{wa_1}\ \langle \mathtt{int} \rangle; \cdots$
$\mathbf{s} \to \mathbf{wa_n}\ \{\mathit{datum}: \mathbf{s} \to \mathbf{wa_n}\ \langle \mathtt{int} \rangle;$
$\mu t.\ \mathbf{wa_i} \to \mathbf{wb_i}\ \{\mathit{datum}: \mathbf{wa_i} \to \mathbf{wb_i}\ \langle \mathtt{int} \rangle;$
$\mathbf{wb_i} \to \mathbf{wc_i}\ \{\mathit{datum}: \mathbf{wb_i} \to \mathbf{wc_i}\ \langle \mathtt{int} \rangle;$
$\mathbf{wc_i} \to \mathbf{wa_i}\ \langle \mathtt{int} \rangle; \mathbf{t}\},$
$\mathit{stop}: \mathbf{wb_i} \to \mathbf{wc_i}\ \{\mathit{stop}: \mathsf{end}\}\}$

$3n$+1 participants: **s** (server) and n independent pipelines, each with $\mathbf{wa_i}$, $\mathbf{wb_i}$, $\mathbf{wc_i}$. Server dispatches integers; each pipeline loops with datum/stop until termination. Displayed schematically: in the Haskell catalogue, the intended parallel composition is realised by interleaving the pipeline-local choices, since the syntax has no explicit parallel operator; the concrete catalogue instances are `Independent Workers(7)` and `Independent Workers(10)`.

Continued on next page

Benchmark Global Type Table (continued)

$G_{ring} = \mathbf{a} \to \mathbf{b} \left\{ \begin{array}{l} \mathit{AppThenGet}: \mathbf{b} \to \mathbf{c} \ \{\mathit{AppThenGet}: \mathbf{c} \to \mathbf{a} \ \{\mathit{Val}: \mathsf{end}\}\} \\ \mathit{App}: \mathbf{b} \to \mathbf{c} \ \{\mathit{App}: \mathbf{a} \to \mathbf{c} \ \{\mathit{Get}: \mathbf{c} \to \mathbf{a} \ \{\mathit{Val}: \mathsf{end}\}\}\} \end{array} \right\}$

Three participants arranged in a ring. One branch sends an application then retrieves a value, while the other performs the application and retrieval in a different order.

(l) Adder [59, Fig. 1(a)]

$G_{ad} = \mu \mathbf{t}.\ \mathbf{c} \to \mathbf{s} \left\{ \begin{array}{l} \mathit{Add}: \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \\ \quad \mathbf{s} \to \mathbf{c} \ \{\mathit{Res}: \mathbf{s} \to \mathbf{c} \ \langle \mathsf{int} \rangle; \mathbf{t}\} \\ \mathit{Bye}: \mathbf{s} \to \mathbf{c} \ \{\mathit{Bye}: \mathsf{end}\} \end{array} \right\}$

Two participants: client **c** and server **s**. The client either submits two integers and receives their sum, or terminates the session.

(m) Company Communication [45, Fig. 9]

$G_{cc} = \mathbf{d} \to \mathbf{ad} \left\{ \mathit{prod}: \mathbf{d} \to \mathbf{s} \left\{ \mathit{prod}: \right. \right.$

$\mathbf{d} \to \mathbf{f}_1 \ \{\mathit{prod}: \mathbf{f}_1 \to \mathbf{f}_2 \ \{\mathit{prod}: \mu \mathbf{t}.$

$\left. \left. \mathbf{f}_2 \to \mathbf{f}_1 \left\{ \begin{array}{l} \mathit{price}: \mathbf{f}_1 \to \mathbf{d} \ \{\mathit{ok}: \mathbf{d} \to \mathbf{ad} \ \{\mathit{go}: \\ \quad \mathbf{f}_1 \to \mathbf{s} \ \{\mathit{price}: \mathbf{s} \to \mathbf{w} \ \{\mathit{publish}: \mathsf{end}\}\}\}\} \\ \mathit{wait}: \mathbf{f}_1 \to \mathbf{d} \ \{\mathit{wait}: \mathbf{d} \to \mathbf{ad} \ \{\mathit{wait}: \\ \quad \mathbf{f}_1 \to \mathbf{s} \ \{\mathit{wait}: \mathbf{s} \to \mathbf{w} \ \{\mathit{wait}: \mathbf{t}\}\}\}\} \end{array} \right\} \right\} \right\} \right\}$

Six participants: **d** (director), **ad** (admin), **s** (seller), $\mathbf{f}_1, \mathbf{f}_2$ (factories), **w** (warehouse). Loops on `wait` until a price is accepted.

(n) Online Wallet [89, Fig. 1]

$G_{ow} = \mathbf{c} \to \mathbf{a} \ \{\mathit{login}: \mathbf{c} \to \mathbf{a} \ \langle \mathsf{string} \rangle; \mathbf{c} \to \mathbf{a} \ \langle \mathsf{string} \rangle;$

$\mathbf{a} \to \mathbf{c} \left\{ \begin{array}{l} \mathit{login\_ok}: \mathbf{a} \to \mathbf{s} \ \{\mathit{login\_ok}: \mu \mathbf{t}. \\ \quad \mathbf{s} \to \mathbf{c} \ \{\mathit{account}: \mathbf{s} \to \mathbf{c} \ \langle \mathsf{int} \rangle; \mathbf{s} \to \mathbf{c} \ \langle \mathsf{int} \rangle; \\ \quad \mathbf{c} \to \mathbf{s} \left\{ \begin{array}{l} \mathit{pay}: \mathbf{c} \to \mathbf{s} \ \langle \mathsf{string} \rangle; \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \mathbf{t} \\ \mathit{quit}: \mathsf{end} \end{array} \right\} \} \} \\ \mathit{login\_fail}: \mathbf{a} \to \mathbf{c} \ \langle \mathsf{string} \rangle; \\ \quad \mathbf{a} \to \mathbf{s} \ \{\mathit{login\_fail}: \mathbf{a} \to \mathbf{s} \ \langle \mathsf{string} \rangle; \mathsf{end}\} \end{array} \right\} \}$

Three participants: **c** (client), **a** (authenticator), **s** (service). After login, the service reports account state and accepts payments until quit; login failure terminates.

(o) Distributed Logging [70, Fig. 16]

$G_{dl} = \mathbf{c} \to \mathbf{s} \ \{\mathit{Start}: \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \mu \mathbf{t}.$

$\mathbf{s} \to \mathbf{c} \left\{ \begin{array}{l} \mathit{Success}: \mathbf{s} \to \mathbf{c} \ \langle \mathsf{int} \rangle; \mathbf{t} \\ \mathit{Failure}: \mathbf{s} \to \mathbf{c} \ \langle \mathsf{int} \rangle; \mathbf{c} \to \mathbf{s} \left\{ \begin{array}{l} \mathit{Restart}: \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \mathbf{t} \\ \mathit{Stop}: \mathbf{c} \to \mathbf{s} \ \langle \mathsf{int} \rangle; \mathsf{end} \end{array} \right\} \end{array} \right\} \}$

Two participants: **c** (client) and **s** (server). After starting, the server reports success (recur) or failure (client restarts or stops).

(p) E-Voting [70]

$G_{ev} = \mathbf{v} \to \mathbf{s} \ \{\mathit{Authenticate}: \mathbf{v} \to \mathbf{s} \ \langle \mathsf{string} \rangle;$

$\mathbf{s} \to \mathbf{v} \left\{ \begin{array}{l} \mathit{Ok}: \mathbf{s} \to \mathbf{v} \ \langle \mathsf{string} \rangle; \mathbf{v} \to \mathbf{s} \left\{ \begin{array}{l} \mathit{Yes}: \mathbf{v} \to \mathbf{s} \ \langle \mathsf{string} \rangle; \mathbf{s} \to \mathbf{v} \ \{\mathit{Result}: \mathbf{s} \to \mathbf{v} \ \langle \mathsf{int} \rangle; \mathsf{end}\} \\ \mathit{No}: \mathbf{v} \to \mathbf{s} \ \langle \mathsf{string} \rangle; \mathbf{s} \to \mathbf{v} \ \{\mathit{Result}: \mathbf{s} \to \mathbf{v} \ \langle \mathsf{int} \rangle; \mathsf{end}\} \end{array} \right\} \\ \mathit{Reject}: \mathbf{s} \to \mathbf{v} \ \langle \mathsf{string} \rangle; \mathsf{end} \end{array} \right\}$

Two participants: **v** (voter) and **s** (server). The voter authenticates; on success casts a Yes/No vote and receives the result. On reject, terminates.

(q)–(t) Coinductive Full(n) [112, Thm. 4.24], $n \in \{2, 3, 4, 5\}$

Let $A_1(\mathbf{t}) = \mathbf{p} \to \mathbf{q} \ \{a: \mathbf{t}\}$ and $A_{m+1}(\mathbf{t}) = \mathbf{p} \to \mathbf{q} \ \{a: A_m(\mathbf{t})\}$.

$G_{cf_n} = \mathbf{p} \to \mathbf{r} \ \{l_k : \mathrm{G_k}\}_{1 \le k \le n}$

$\mathrm{G_k} = \mu \mathbf{t}.\ \mathbf{p} \to \mathbf{q} \left\{ \begin{array}{l} a: A_{d_k}(\mathbf{t}) \\ b: \mathbf{p} \to \mathbf{q} \ \{l_k : \mathsf{end}\} \end{array} \right\}$

Three participants: **p**, **q**, **r**. Branch depths $(d_1, \ldots, d_n)$ are the first n primes: $(2, 3)$, $(2, 3, 5)$, $(2, 3, 5, 7)$, $(2, 3, 5, 7, 11)$.

}

Example D.1. The Binary Counter family exhibits the unavoidable exponential growth of the synthesised global type with the number of participants. Let $\Delta_b = \{\mathbf{p_i} : \mathsf{T}_{\mathbf{p_i}}\}_{i \in P}$ where $P = \{0, 1, \ldots, n-1\}$ and each participant $\mathbf{p_i}$ has local type:

$$\mathsf{T}_{\mathbf{p_i}} = \mu \mathbf{t_0}.\ \mathbf{p_{i-1}} \,\&\, \left\{ \begin{array}{l} l_0 : \mathbf{p_{i+1}} \oplus \{ l_0 : \mathbf{t_0} \} \\ l_1 : \mathbf{p_{i+1}} \oplus \{ l_0 : \mu \mathbf{t_1}.\ \mathbf{p_{i-1}} \,\&\, \left\{ \begin{array}{l} l_0 : \mathbf{p_{i+1}} \oplus \{ l_0 : \mathbf{t_1} \} \\ l_1 : \mathbf{p_{i+1}} \oplus \{ l_1 : \mathbf{t_0} \} \end{array} \right. \} \end{array} \right.$$

Each participant acts as one bit of a ripple-carry counter, receiving from $\mathbf{p_{i-1}}$ and sending to $\mathbf{p_{i+1}}$. Labels l_0 and l_1 encode "no carry" and "carry", while $\mathbf{t_0}$ and $\mathbf{t_1}$ record whether the current bit stores 0 or 1; receiving l_1 flips the bit, and only state $\mathbf{t_1}$ propagates the carry onward.